

Unit 1 – Lecture 1

Project Overview and Feasibility Study in IT Project Management

Dr. S.Markkandeyan
Senior Assistant Professor
School of Computing SASTRA
Deemed to be University
Thanjavur - 613005

1. Project Overview

- A project overview provides a high-level summary of an IT project.
- It serves as a foundation for understanding the project's
 - purpose,
 - scope, and
 - stakeholders.

Key Components of Project Overview

1. Project Objectives:

- Defines the purpose of the project.
- Outlines goals and deliverables.
- Example: "Develop a mobile app to streamline e-commerce transactions for small businesses."

2. Scope:

- Specifies boundaries, including what is included and excluded.
- Avoids scope creep by setting clear limits.
- Example: "The project will include app design, development, and deployment but exclude ongoing maintenance."

Key Components of Project Overview

3. Stakeholders:

- Identifies individuals or groups impacted by or involved in the project.
- Example: "Stakeholders include the client, development team, QA team, and end users."

4. Resources:

- Outlines required resources (financial, human, and technological).
- Example: "The project will require a budget of \$50,000, a team of 5 developers, and cloud hosting services."

5. Timeline:

- Establishes key milestones and deadlines.
- Example: "The project will commence on March 1 and conclude on September 1."

Key Components of Project Overview

6. Budget:

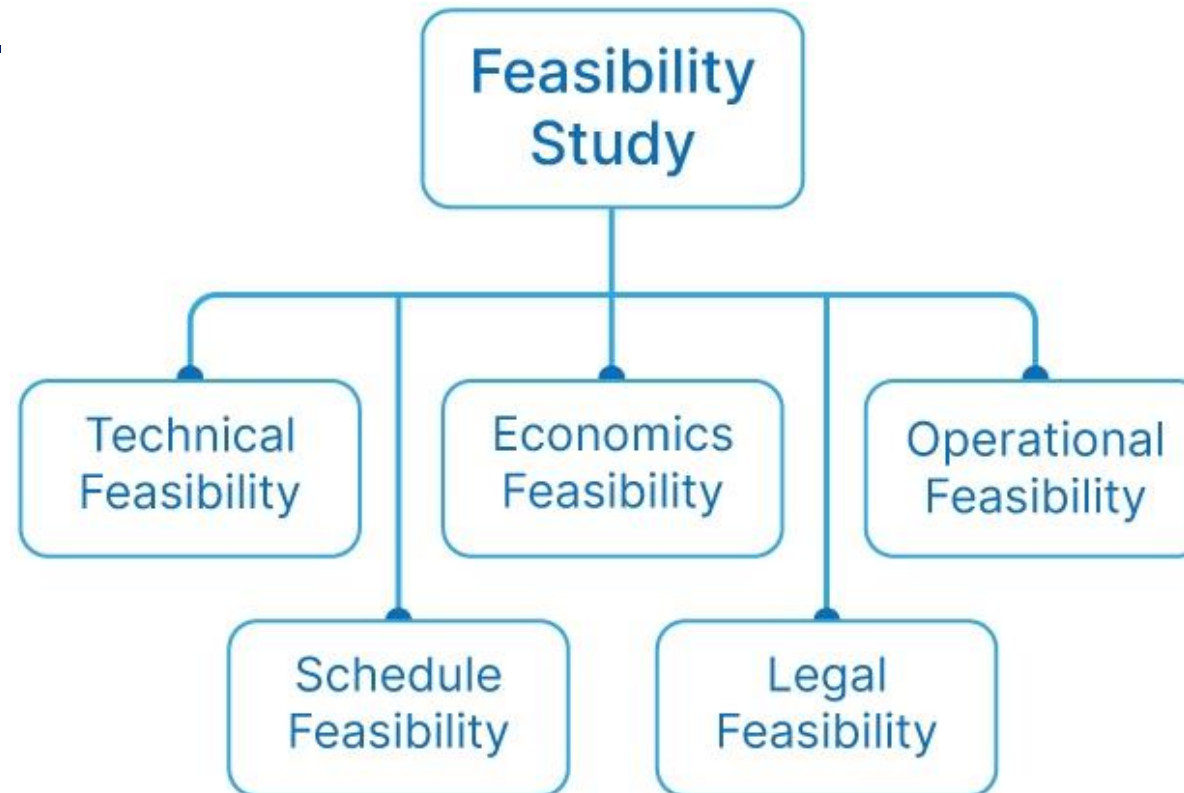
- An estimate of the financial resources required.
- Example: \$25,000

7. Risks and Assumptions:

- Lists potential challenges and assumptions made during planning.
- Example: "Assume 70% of the development team will work remotely, which may affect collaboration."

2. Feasibility Study in ITPM

- A feasibility study **assesses** whether a project is **viable**, **achievable**, and **worth pursuing**.
- It evaluates **technical**, **financial**, **operational**, and **legal** aspects of the project.



Key Aspects of Feasibility Study

➤ **Technical Feasibility:**

- Assesses if the necessary technology and expertise are available.
- Example: "Does the team have expertise in React Native to build a cross-platform mobile app?"

➤ **Economic Feasibility:**

- Analyzes costs versus expected benefits.
- Example: "The project is estimated to cost \$50,000, but it is expected to generate \$200,000 in revenue within the first year."

Key Aspects of Feasibility Study

➤ **Operational Feasibility:**

- Evaluates if the project aligns with organizational processes and end-user needs.
- Example: "Will the app integrate seamlessly with existing inventory management systems?"

➤ **Legal Feasibility:**

- Ensures the project complies with laws and regulations.
- Example: "Does the app comply with GDPR and other data protection regulations?"

➤ **Schedule Feasibility:**

- Assesses if the project can be completed within the proposed timeframe.
- Example: "Can the app be developed and tested within six months?"

Steps to Conduct a Feasibility Study

1. Define the project goals and objectives.
2. Gather and analyze relevant data (e.g., market research, technical requirements).
3. Evaluate each feasibility aspect (technical, economic, operational, etc.).
4. Document findings and provide recommendations.

Unit 1 – Lecture 2

Identification of Market and
Demand Analysis, Project Cost
Estimate, Financial Appraisal for
a Software Project

1. Identification of Market and Demand Analysis

- It involves **determining** the **target market** for the **software project** and **analyzing its demand**.

Components:

1. Market Identification:

- **Example:** "Target market includes SMEs in retail requiring inventory management software."

2. Market Research:

- **Tools:** Google Trends, SWOT analysis, and competitor benchmarking.

3. Demand Analysis:

- **Example:** "60% of SMEs lack affordable inventory management systems, indicating high demand."

4. Competitor Analysis:

- **Example:** "Competitor analysis shows most systems are expensive and lack mobile support."

5. Customer Feedback:

- **Example:** "Pilot tests reveal high demand for real-time stock tracking features."

2. Project Cost Estimate

- ❖ It involves **predicting the financial resources** required to complete a **software project**, including development, deployment, and support.
- ❖ Estimating costs is critical to ensure the project stays within budget.

- ❖ **Key Components:**

- 1. Direct Costs:**

Development Costs:

Example: "\$40,000 for a team of 5 over six months."

Tools and Software:

Example: "\$5,000 for AWS and software licenses."

- 2. Indirect Costs:**

Example: "\$2,000 for overheads."

- 3. Contingency Costs:**

Example: "\$5,000 for contingency."

- 4. Total Cost Estimate:**

Example: "Total estimated cost: \$52,000."

2.1 Methods of Cost Estimation

1. Expert Judgment

Example: "Experienced manager estimates a CRM system at \$100,000."

2. Analogous Estimation (Historical Data)

Example: "Past e-commerce site cost \$80,000; adjust for inflation."

3. Parametric Estimation

Example: "Cost = \$500 per function point; 200 function points = \$100,000."

4. Bottom-Up Estimation(cost/module)

Example: "Frontend = \$30,000, Backend = \$40,000, QA = \$10,000."

5. Top-Down Estimation

Example: "Allocate \$100,000 for a prototype based on company budget."

2.2 Factors Affecting Cost Estimation

1. **Project Size**
2. **Complexity**
3. **Technology Stack**
4. **Team Expertise**
5. **Timeline.**

2.3 Tools for Cost Estimation

1. COCOMO

2. Function Point Analysis (FPA)

3. Microsoft Project

2.4 Importance of Cost Estimation & Example

1. Ensures budget adherence.
2. Facilitates resource planning.
3. Helps in decision-making and project viability assessment.

➤ **Example:** "An estimated cost of \$50,000 for an inventory management system ensures the company allocates sufficient resources before starting the project."

➤ **Outcome:**

➤ **Accurate cost estimation is critical for project success**, minimizing risks of budget overruns and resource shortages.

3. Financial Appraisal

- This involves **evaluating the financial viability** of the software project.

Key Methods:

1. Payback Period
2. Net Present Value (NPV)
3. Internal Rate of Return (IRR)
4. Benefit-Cost Ratio (BCR)
5. Profitability Index (PI)

3.1 Payback Period

❖ Time required to recover the initial Investment.

❖ Formula:

$$\textit{Payback Period (t)} = \frac{\textit{Initial Investment}}{\textit{Annual Cash Inflow}}$$

Example: “Calculate the payback period for a company investing \$50,000, with annual revenue of \$25,000?”

$$\textit{Payback Period (t)} = \frac{\$50,000}{\$25,000} = 2 \textit{ Years}$$

3.2 Net Present Value (NPV)

❖ Measures the **profitability of the project** by discounting future cash flows.

❖ Formula:

$$\text{NPV} = \sum \frac{\text{Cash Inflow}_t}{(1+r)^t} - \text{Initial Investment}$$

Example: Calculate the NPV for a company investing \$50,000 where they gain cashflow of \$10,000 over 3 years at 10% discount rate ?.

$$\text{NPV} = \sum \frac{\text{Cash Inflow}_t}{(1+r)^t} - \text{Initial Investment}$$

3.2 Net Present Value (NPV)

❖ Measures the **profitability of the project** by discounting future cash flows.

❖ Formula:

$$\text{NPV} = \sum \frac{\text{Cash Inflow}_t}{(1+r)^t} - \text{Initial Investment}$$

Example: Calculate the NPV for a company investing \$50,000 where they gain cashflow of \$10,000 over 3 years at 10% discount rate ?.

$$\text{NPV} = \sum \frac{\$10,000_3}{(1 + 0.10)^3} - \$50,000 = \$8,264$$

3.3 Internal Rate of Return (IRR)

- ❖ It represents the **discount rate** at which the **Net Present Value (NPV)** of all cash flows (both inflows and outflows) **equals zero**.
- ❖ Decision Rule:
 - ❖ **IRR** > Required Rate of Return (or Cost of Capital): **The investment is profitable and should be considered.**
 - ❖ **IRR** < **Required Rate of Return**: **The investment is not viable.**
 - ❖ **IRR** = Required Rate of Return: **The investment breaks even.**

Example: IRR = 15%, cost of capital = 10%, project is viable.

3.4 Benefit-Cost Ratio (BCR)

❖ Ratio of benefits to costs. A **BCR > 1** indicates viability.

❖ Formula:

$$BCR = \frac{\text{Total Benefits}}{\text{Total Costs}}$$

Example: Calculate BCR for the company with Total benefits of \$75,000 and Total cost is \$50,000.

$$BCR = \frac{\$75,000}{\$50,000} = 1.5 \text{ [Viable]}$$

3.5 Profitability Index (PI)

- ❖ It **assess** the **feasibility or profitability of a project**.
- ❖ It **compares** the **present value of benefits** to the **present value of costs**.
- ❖ Similar to BCR but considers NPV.
- ❖ Formula:

$$PI = 1 + \frac{NPV}{Initial\ Investment}$$

Example: Financial Appraisal for a Software Project

Project: Development of an Online Learning Platform.

Initial Investment: \$100,000

Projected Revenue: \$50,000/year

Discount Rate: 8%

1. **Payback Period:** $PP = \frac{100,000}{50,000} = 2 \text{ Years}$

2. **NPV:** Using a discount rate of 8%, calculate future cash flows over 5 years:

- $NPV = \sum \frac{50,000}{(1 + 0.08)^t} - 100,000$

- $NPV \approx \$108,660.$

3. **IRR:** $IRR \approx 12\%.$

4. $BCR = \frac{250,000}{100,000} = 2.5$

Conclusion: The project is financially viable.

For Problem solving, please refer lecture materials and solved problems materials.

Lecture 1: Project Overview and Feasibility Study in IT Project Management

1. Project Overview in IT Project Management

A project overview provides a high-level summary of an IT project. It serves as a foundation for understanding the project's purpose, scope, and stakeholders.

Key Components of a Project Overview:

1. Project Objectives:

- Defines the purpose of the project.
- Outlines goals and deliverables.
- **Example:** "Develop a mobile app to streamline e-commerce transactions for small businesses."

2. Scope:

- Specifies boundaries, including what is included and excluded.
- Avoids scope creep by setting clear limits.
- **Example:** "The project will include app design, development, and deployment but exclude ongoing maintenance."

3. Stakeholders:

- Identifies individuals or groups impacted by or involved in the project.
- **Example:** "Stakeholders include the client, development team, QA team, and end users."

4. Resources:

- Outlines required resources (financial, human, and technological).
- **Example:** "The project will require a budget of \$50,000, a team of 5 developers, and cloud hosting services."

5. Timeline:

- Establishes key milestones and deadlines.
- **Example:** "The project will commence on March 1 and conclude on September 1."

6. Budget:

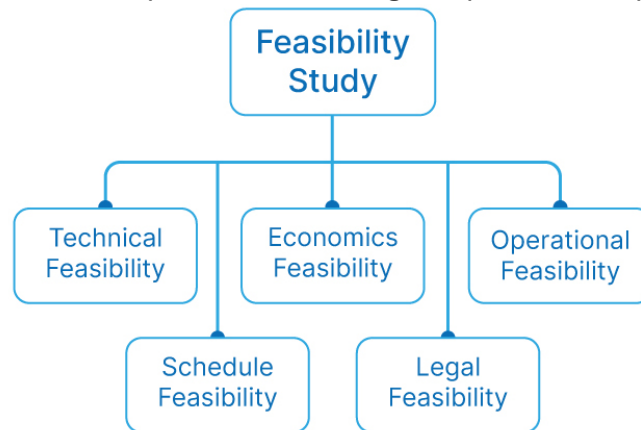
- An estimate of the financial resources required.
- **Example:** \$25,000

7. Risks and Assumptions:

- Lists potential challenges and assumptions made during planning.
- **Example:** "Assume 70% of the development team will work remotely, which may affect collaboration."

2. Feasibility Study in IT Project Management

A feasibility study assesses whether a project is viable, achievable, and worth pursuing. It evaluates technical, financial, operational, and legal aspects of the project.



Key Aspects of a Feasibility Study:

1. Technical Feasibility:

- Assesses if the necessary technology and expertise are available.
- **Example:** "Does the team have expertise in React Native to build a cross-platform mobile app?"

2. Economic Feasibility:

- Analyzes costs versus expected benefits.
- **Example:** "The project is estimated to cost \$50,000, but it is expected to generate \$200,000 in revenue within the first year."

3. Operational Feasibility:

- Evaluates if the project aligns with organizational processes and end-user needs.
- **Example:** "Will the app integrate seamlessly with existing inventory management systems?"

4. Legal Feasibility:

- Ensures the project complies with laws and regulations.
- **Example:** "Does the app comply with GDPR and other data protection regulations?"

5. Schedule Feasibility:

- Assesses if the project can be completed within the proposed timeframe.
- **Example:** "Can the app be developed and tested within six months?"

Steps to Conduct a Feasibility Study

1. Define the project goals and objectives.
2. Gather and analyze relevant data (e.g., market research, technical requirements).
3. Evaluate each feasibility aspect (technical, economic, operational, etc.).
4. Document findings and provide recommendations.

Understanding the project overview and conducting a thorough feasibility study are critical initial steps in IT project management. These processes ensure stakeholders are aligned and that the project's objectives are realistic and achievable. By addressing technical, economic, operational, legal, and schedule factors, organizations can make informed decisions to minimize risks and maximize project success.

Example 1: IT Project Overview

Project Title: Implementation of a Cloud-Based Customer Relationship Management (CRM) System

- **Objective:** To enhance customer management and sales tracking by transitioning from a legacy CRM to a cloud-based solution.
- **Scope:** Includes migrating customer data, training employees, and deploying the new system. Excludes developing custom CRM features.
- **Stakeholders:** IT department, sales team, senior management, CRM vendor.
- **Deliverables:** Fully functional cloud-based CRM, training materials, data migration reports.
- **Timeline:** 6 months, starting from January 2025.
- **Budget:** \$100,000.
- **Risks:** Data migration issues, employee resistance to change, vendor delays.

Example 2 : IT Project Overview

Project Title:
Development of a Cloud-Based Inventory Management System

1. Project Objectives:

- To create a user-friendly, cloud-based inventory management system for small and medium-sized enterprises (SMEs).
- To improve inventory tracking efficiency and reduce errors caused by manual processes.
- To provide real-time stock updates and analytics to help businesses optimize inventory levels.

2. Scope:

- **In-Scope:**
 - Development of a web-based and mobile-friendly application.
 - Integration with barcode scanners and POS systems.
 - Implementation of features like inventory tracking, stock alerts, and sales reports.
 - Cloud hosting to ensure scalability and data security.
- **Out-of-Scope:**
 - Ongoing system maintenance after project completion.
 - Integration with accounting software.

3. Stakeholders:

- **Internal Stakeholders:**
 - Project sponsor: Business Solutions Inc.
 - Project manager: IT Department Lead.
 - Development team, QA team.
- **External Stakeholders:**
 - SMEs using the system.
 - Third-party vendors providing hardware support (e.g., barcode scanners).

4. Resources:

- **Budget:** \$100,000.
- **Team:**
 - 1 project manager, 4 developers, 2 QA specialists, and 1 UI/UX designer.
- **Technological Resources:**
 - Programming tools: React.js for frontend, Node.js for backend.
 - Cloud platform: AWS for hosting.
 - Database: MongoDB.

5. Timeline:

Milestone	Deadline
Project kickoff	February 1, 2025
Requirement gathering	February 15, 2025
Design phase completion	March 15, 2025
Development phase	June 15, 2025
Testing phase	July 15, 2025
Project launch	August 1, 2025

6. Risks and Assumptions:

- **Risks:**
 - Delay in hardware integration due to vendor issues.
 - Security risks associated with handling sensitive client data.
- **Assumptions:**
 - SMEs will provide timely feedback during the design and testing phases.
 - All team members will work 40 hours per week on the project.

This project overview provides a clear roadmap for developing the inventory management system, ensuring that all stakeholders have a shared understanding of the goals, scope, resources, and timeline.

Example 3: Feasibility Study for an IT Project

Project Title: Development of an E-Commerce Mobile Application

- **Technical Feasibility:** Required technology stack (e.g., Flutter, Firebase) and skilled developers are available.
- **Economic Feasibility:** Projected cost is \$50,000, with an expected ROI of 150% within two years.
- **Operational Feasibility:** The app aligns with the company's goal to increase online sales by 20%.
- **Legal and Regulatory Feasibility:** The app complies with data privacy laws (e.g., GDPR, CCPA).
- **Schedule Feasibility:** Development can be completed in 6 months with adequate resources.

Outcome of the Feasibility Study

Based on the analysis, the e-commerce app project is feasible, and the recommendation is to proceed with the planning phase.

Example 4: Feasibility Study for an IT Project

Project: Development of an AI-based Chatbot for Customer Support

1. **Technical Feasibility:**
 - Existing team has experience in Python, NLP libraries, and cloud deployment.
 - Required tools like TensorFlow and AWS are accessible.
2. **Economic Feasibility:**
 - Development cost: \$30,000.
 - Estimated savings: \$60,000/year by reducing customer support staff workload.
3. **Operational Feasibility:**
 - Chatbot will handle 80% of common queries, freeing up human agents for complex cases.
 - Positive feedback from a pilot test with a small user group.
4. **Legal Feasibility:**
 - Ensures compliance with data privacy laws (e.g., GDPR, HIPAA).
 - Includes disclaimers and user consent mechanisms.
5. **Schedule Feasibility:**
 - Project can be completed in four months, with a buffer for unforeseen delays.

Outcome: The study concludes that the project is feasible and should proceed.

Summary

The project overview lays the groundwork for understanding the project, while the feasibility study ensures the project is viable. Together, they help IT managers make informed decisions and set the stage for successful project execution.

Lecture 2: Identification of Market and Demand Analysis, Project Cost Estimate, Financial Appraisal for a Software Project

1. Identification of Market and Demand Analysis

This step involves determining the target market for the software project and analyzing its demand.

Key Components:

1. Market Identification:

- Define the target audience based on demographics, industry, and specific needs.
- **Example:** "Target market includes SMEs in retail requiring inventory management software."

2. Market Research:

- Use surveys, interviews, and competitor analysis to gather insights.
- **Tools:** Google Trends, SWOT analysis, and competitor benchmarking.

3. Demand Analysis:

- Estimate the potential demand for the software based on market size and growth trends.
- Assess unmet needs and pain points that the software aims to solve.
- **Example:** "60% of SMEs lack affordable inventory management systems, indicating high demand."

4. Competitor Analysis:

- Analyze competitors' offerings, pricing, and market share.
- **Example:** "Competitor analysis shows most systems are expensive and lack mobile support."

5. Customer Feedback:

- Engage potential users early for feedback on features and pricing.
- **Example:** "Pilot tests reveal high demand for real-time stock tracking features."

2. Project Cost Estimate

Software project cost estimation involves predicting the financial resources required to complete a software project, including development, deployment, and support.

Estimating costs is critical to ensure the project stays within budget.

Key Components:

1. Direct Costs:

- **Development Costs:**
 - Salaries for developers, designers, and QA testers.
 - **Example:** "\$40,000 for a team of 5 over six months."
- **Tools and Software:**
 - Licenses for development tools and platforms.
 - **Example:** "\$5,000 for AWS and software licenses."

2. Indirect Costs:

- Overheads such as office space, utilities, and administrative expenses.
- **Example:** "\$2,000 for overheads."

3. Contingency Costs:

- Allocate funds for unforeseen expenses (usually 10-15% of total cost).
- **Example:** "\$5,000 for contingency."

4. Total Cost Estimate:

- Combine direct, indirect, and contingency costs.
- **Example:** "Total estimated cost: \$52,000."

Methods of Cost Estimation

1. Expert Judgment:

- Based on experience from similar projects.
- Example: "Experienced manager estimates a CRM system at \$100,000."

2. Analogous Estimation:

- Uses historical data from similar projects.
- Example: "Past e-commerce site cost \$80,000; adjust for inflation."

3. Parametric Estimation:

- Applies mathematical models based on project parameters.
- Example: "Cost = \$500 per function point; 200 function points = \$100,000."

4. Bottom-Up Estimation:

- Breaks the project into smaller tasks and estimates costs for each.
- Example: "Frontend = \$30,000, Backend = \$40,000, QA = \$10,000."

5. Top-Down Estimation:

- Uses overall project goals and constraints to estimate costs.
- Example: "Allocate \$100,000 for a prototype based on company budget."

Factors Affecting Cost Estimation

1. **Project Size:** Larger projects require more resources.
2. **Complexity:** Complex requirements lead to higher costs.
3. **Technology Stack:** Advanced or niche technologies may increase costs.
4. **Team Expertise:** Skilled teams may be more expensive but work faster.
5. **Timeline:** Tight deadlines may necessitate higher spending.

Tools for Cost Estimation

- **COCOMO:** Calculates cost based on lines of code.
- **Function Point Analysis (FPA):** Estimates based on functionality delivered.
- **Microsoft Project:** Tracks costs and timelines.

Importance of Cost Estimation

1. Ensures budget adherence.
2. Facilitates resource planning.
3. Helps in decision-making and project viability assessment.

Example: "An estimated cost of \$50,000 for an inventory management system ensures the company allocates sufficient resources before starting the project."

Outcome: Accurate cost estimation is critical for project success, minimizing risks of budget overruns and resource shortages.

3. Financial Appraisal

This involves evaluating the financial viability of the software project.

Key Methods:

1. Payback Period:

- Time required to recover the initial investment.
- Formula: Payback Period = Initial Investment / Annual Cash Inflow.
- **Example:** "\$50,000 investment, \$25,000 annual revenue = 2 years."

2. Net Present Value (NPV):

- Measures the profitability of the project by discounting future cash flows.
- Formula: $NPV = \sum \frac{C_t}{(1+r)^t} - \text{Initial Investment}$
 - C_t : Cash flow at time t.
 - r : Internal Rate of Return.
 - t: Time period.
- Example: NPV of \$10,000 over 3 years at 10% discount rate = \$8,264.

3. Internal Rate of Return (IRR):

- The **Internal Rate of Return (IRR)** is a financial metric used to evaluate the profitability of an investment or project. It represents the discount rate at which the **Net Present Value (NPV)** of all cash flows (both inflows and outflows) equals zero.
- Rate at which NPV = 0; compares profitability to the cost of capital.
- Example: IRR = 15%, cost of capital = 10%, project is viable.

4. Benefit-Cost Ratio (BCR):

- Ratio of benefits to costs. A BCR > 1 indicates viability.
- Formula: $BCR = \frac{\text{Total Benefits}}{\text{Total Costs}}$
- Example: BCR = \$75,000 / \$50,000 = 1.5.

5. Profitability Index (PI):

- It is a cost-benefit analysis to assess the feasibility or profitability of a project or investment. It compares the present value of benefits to the present value of costs.
- Similar to BCR but considers NPV.
- Formula: $PI = 1 + \frac{NPV}{\text{Initial Investment}}$

Example: Financial Appraisal for a Software Project

Project: Development of an Online Learning Platform.

- **Initial Investment:** \$100,000.
- **Projected Revenue:** \$50,000/year.
- **Discount Rate:** 8%.

1. **Payback Period:** $PP = \frac{100,000}{50,000} = 2 \text{ Years}$

2. **NPV:** Using a discount rate of 8%, calculate future cash flows over 5 years:

- $NPV = \sum \frac{50,000}{(1+0.08)^t} - 100,000$
- NPV ≈ \$108,660.

3. **IRR:** $IRR \approx 12\%$.

4. **BCR** = $\frac{250,000}{100,000} = 2.5$

Conclusion: The project is financially viable.

This structured approach ensures the project is both market-driven and financially sustainable.

Summary:

- **Market and Demand Analysis:** Understand the market size, target audience, and competition.
- **Project Cost Estimate:** Identify all cost components for accurate budgeting.
- **Financial Appraisal:** Use methods like Payback Period, NPV, and IRR to assess financial viability.

Lecture 2: Software Project Scheduling

Software project scheduling is a crucial aspect of project management that involves planning and managing the timeline of software development projects. It helps ensure that a project is completed on time and within budget by breaking down tasks into manageable units and defining the milestones and deadlines required to complete the project successfully. Effective scheduling can lead to better resource management, risk reduction, and overall project success.

Key concepts of software project scheduling:

1. Project Scope and Requirements:

Before creating a schedule, it is important to fully understand the project scope and requirements. This involves gathering detailed information about the project's goals, features, resources, and constraints. A well-defined scope sets clear expectations and serves as the foundation for creating an accurate schedule.

2. Task Breakdown:

Once the project scope is understood, it is broken down into smaller tasks or work packages. These tasks are manageable and can be assigned to different team members. This process is often referred to as **Work Breakdown Structure (WBS)**. A good WBS helps the team focus on specific tasks, ensuring that no important detail is overlooked.

3. Time Estimation:

For each task, time estimations need to be made to determine how long it will take to complete the task. This can be based on historical data, expert judgment, or estimation techniques like **Analogous Estimating, Parametric Estimating, or Expert Judgment**.

4. Resource Allocation:

A key aspect of scheduling is ensuring that tasks are assigned to the appropriate resources (e.g., developers, testers, designers) with the right skills and availability. Resource allocation involves managing people, hardware, software tools, and other resources that are needed for the completion of tasks.

5. Dependencies and Constraints:

In software development projects, tasks are often dependent on one another. For example, development cannot begin until the design phase is completed. Identifying these dependencies is crucial for building a schedule. Constraints like team availability, budget limits, and specific deadlines also play a role in the scheduling process.

6. Gantt Chart:

A commonly used tool for scheduling in software projects is the **Gantt Chart**, a visual representation of the project timeline. It shows the start and end dates for each task, as well as how tasks overlap and their dependencies. This chart helps project managers track progress and make adjustments as needed.

7. Critical Path Method (CPM):

The **Critical Path Method** is a technique used to identify the longest sequence of dependent tasks and determine the minimum project duration. The critical path has no slack, meaning any delay in these tasks will directly affect the project's overall timeline.

8. Milestones:

Milestones represent significant points in the project lifecycle, such as the completion of a major phase (e.g., design, coding, testing). These are often used to mark progress and provide an opportunity for review and adjustment.

9. Monitoring and Adjustments:

Once the project is underway, the schedule needs to be monitored regularly to ensure that the project stays on track. If issues arise (e.g., delays, resource shortages, scope changes), the schedule may need to be adjusted. Techniques such as **Earned Value Management (EVM)** and **Burndown Charts** help track project progress against the baseline schedule.

10. Risk Management:

Software projects are often subject to risks, such as changes in requirements, technological challenges, and unexpected events. A good project schedule accounts for potential risks and includes buffer time for unforeseen circumstances.

Key Tools and Techniques in Software Project Scheduling:

- **Project Management Software:** Tools like Microsoft Project, Jira, Trello, or Asana help manage tasks, track progress, and maintain communication.
- **Agile Methodologies:** In Agile development, schedules are often more flexible, and tasks are planned in smaller increments (sprints). Sprint planning and daily standups are used to manage and adjust timelines.
- **Kanban and Scrum:** These Agile methodologies help manage workflows and keep the team focused on the most important tasks at any given time.

Summary of Project Scheduling:

Software project scheduling is essential for delivering projects on time and within scope. It involves breaking down tasks, estimating time, assigning resources, managing dependencies, and continuously monitoring progress. By using techniques like Gantt charts, the Critical Path Method, and Agile methodologies, project managers can improve their

chances of project success and avoid common pitfalls like scope creep and missed deadlines.

Example 1: Software Project Scheduling

Let's assume the project is to develop a basic web application for a small business. We'll walk through the phases, task breakdown, time estimates, dependencies, and scheduling.

Project: Development of a Web Application for a Small Business

Phases and Tasks:

The project is divided into four phases: **Planning**, **Development**, **Testing**, and **Deployment**.

1. Planning Phase:

- **Task 1.1: Gather Requirements** (Duration: 2 days)
- **Task 1.2: Define Scope and Features** (Duration: 2 days)

2. Development Phase:

- **Task 2.1: Design the User Interface** (Duration: 3 days)
- **Task 2.2: Develop Frontend** (Duration: 5 days)
- **Task 2.3: Develop Backend** (Duration: 6 days)
- **Task 2.4: Integrate Frontend and Backend** (Duration: 4 days)

3. Testing Phase:

- **Task 3.1: Unit Testing** (Duration: 3 days)
- **Task 3.2: Integration Testing** (Duration: 3 days)
- **Task 3.3: User Acceptance Testing (UAT)** (Duration: 2 days)

4. Deployment Phase:

- **Task 4.1: Prepare Deployment Environment** (Duration: 2 days)
- **Task 4.2: Deploy the Application** (Duration: 1 day)

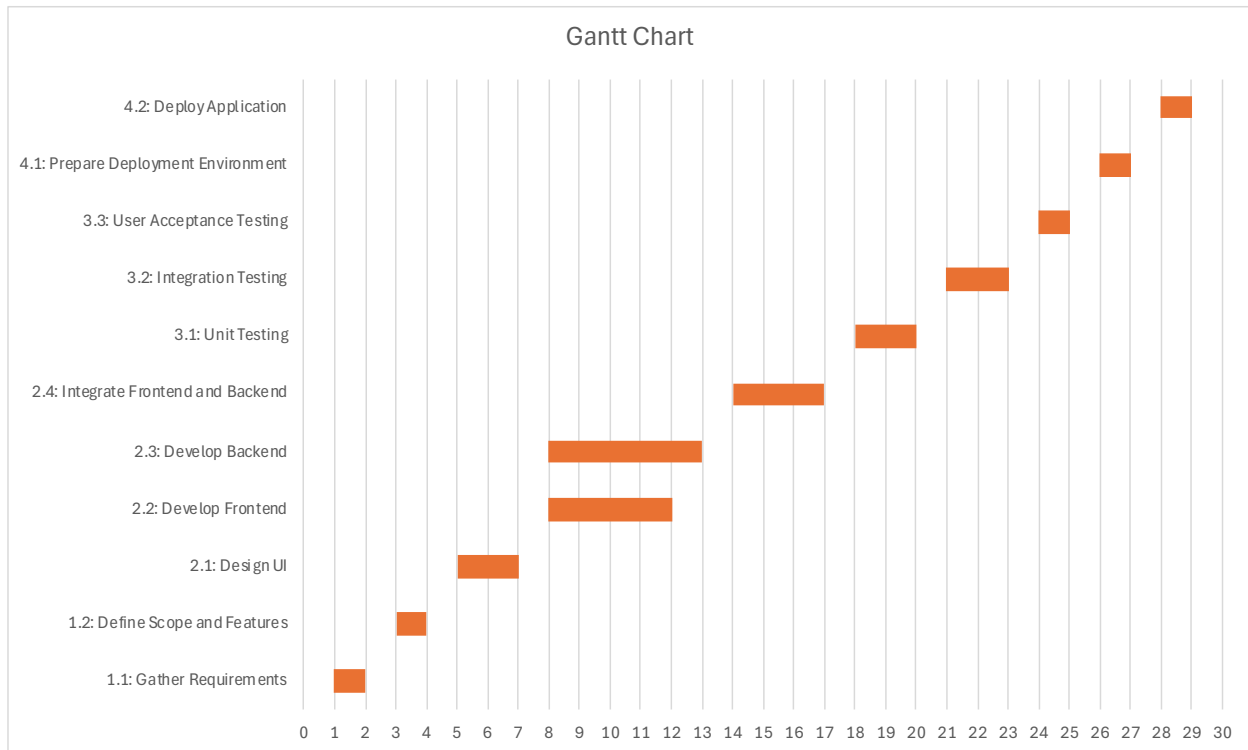
Task Dependencies:

- **Task 1.1** (Gather Requirements) must be completed before **Task 1.2** (Define Scope and Features).
- **Task 1.2** (Define Scope and Features) must be completed before **Task 2.1** (Design the User Interface).
- **Task 2.1** (Design the User Interface) must be completed before **Task 2.2** (Develop Frontend) and **Task 2.3** (Develop Backend).
- **Task 2.2** (Develop Frontend) and **Task 2.3** (Develop Backend) must be completed before **Task 2.4** (Integrate Frontend and Backend).
- **Task 2.4** (Integrate Frontend and Backend) must be completed before **Task 3.1** (Unit Testing).
- **Task 3.1** (Unit Testing) must be completed before **Task 3.2** (Integration Testing).
- **Task 3.2** (Integration Testing) must be completed before **Task 3.3** (User Acceptance Testing).
- **Task 3.3** (User Acceptance Testing) must be completed before **Task 4.1** (Prepare Deployment Environment).
- **Task 4.1** (Prepare Deployment Environment) must be completed before **Task 4.2** (Deploy the Application).

Gantt Chart Representation (Simplified Timeline):

Let's assume the project starts on January 1st. Here's how the timeline might look:

Task	Start Date	Duration	End Date
1.1: Gather Requirements	Jan 1	2 days	Jan 2
1.2: Define Scope and Features	Jan 3	2 days	Jan 4
2.1: Design UI	Jan 5	3 days	Jan 7
2.2: Develop Frontend	Jan 8	5 days	Jan 12
2.3: Develop Backend	Jan 8	6 days	Jan 13
2.4: Integrate Frontend and Backend	Jan 14	4 days	Jan 17
3.1: Unit Testing	Jan 18	3 days	Jan 20
3.2: Integration Testing	Jan 21	3 days	Jan 23
3.3: User Acceptance Testing	Jan 24	2 days	Jan 25
4.1: Prepare Deployment Environment	Jan 26	2 days	Jan 27
4.2: Deploy Application	Jan 28	1 day	Jan 28

**Key Points:**

- Dependencies:** Tasks depend on the completion of prior tasks. For example, frontend and backend development cannot start until the UI design is completed.
- Time Estimation:** Each task has an estimated duration. Total project duration is calculated by adding the durations of all tasks while considering dependencies.
- Resource Allocation:** The development team will work on frontend, backend, and UI design tasks simultaneously, as long as those tasks are independent of each other.

4. **Project Monitoring:** The project manager will track the progress of each task and make adjustments if delays or risks are encountered.

This simple scheduling example highlights how a software project can be planned and scheduled using task breakdowns, dependencies, and time estimates. Tools like Gantt charts help visualize the schedule, ensuring that the project stays on track and is completed within the expected timeline.

Problem 1: Investment in a New Machine

A company is considering purchasing a new machine that costs **\$50,000**. The expected cash inflows from the machine over the next 5 years are:

- Year 1: **\$12,000**
- Year 2: **\$15,000**
- Year 3: **\$18,000**
- Year 4: **\$10,000**
- Year 5: **\$8,000**

If the required discount rate is **10%**, calculate the **Net Present Value (NPV)** of the investment and determine whether the company should proceed with the purchase.

Year 0	-50,000
Year 1	10,909
Year 2	12,396
Year 3	13,524
Year 4	6,830
Year 5	4968
NPV	-1373

Problem 2: New Project Evaluation

A company is evaluating a new project that requires an initial investment of **\$100,000**. The project is expected to generate cash flows of:

- Year 1: **\$30,000**
- Year 2: **\$40,000**
- Year 3: **\$35,000**
- Year 4: **\$50,000**

If the discount rate is **8%**, compute the **NPV** of the project. Should the company accept or reject the project?

Problem 3: Real Estate Investment

An investor is considering purchasing a rental property that costs **\$200,000**. The property is expected to generate rental income of:

- Year 1: **\$25,000**
- Year 2: **\$30,000**
- Year 3: **\$35,000**
- Year 4: **\$40,000**
- Year 5: **\$50,000**

At the end of year 5, the investor expects to sell the property for **\$220,000**. If the required rate of return is **7%**, calculate the **NPV** of the investment and advise whether the investor should proceed.

Year 0	-200,000
Year 1	23,365
Year 2	26,203
Year 3	28,571
Year 4	30,511
Year 5	35,637
NPV	-55,711

9/1/25

2

Page ①

Unit 1 Problem Solving

1. Net Present Value (NPV) Examples

Problem 1:

Assumptions:

→ Initial Investment: \$ 200,000 (0 year)

→ Expected Cash Inflows:

⊗ Year 1: \$ 80,000

⊗ Year 2: \$ 100,000

⊗ Year 3: \$ 120,000

→ Discount Rate: 10%.

Solution to Problem 1:

Step 1. Formula for NPV:

$$NPV = \sum_{t=0}^n \frac{C_t}{(1+r)^t}$$

where

C_t - cash flow @ t

r - discount rate

t - year (0, 1, ..., n)

Breakdown NPV Calculation

□ Year 0 (Initial Investment)

Cash flow: - \$ 200,000

$$\text{Present value (PV)} = \frac{-200,000}{(1+0.10)^0} = -\$200,000$$

□ Year 1

Cash Flow: \$ 80,000

$$PV = \frac{80,000}{(1+0.10)^1} = \frac{80,000}{1.10} = \$72,727$$

□ year 2:

Cash flow: \$ 100,000

$$PV = \frac{100,000}{(1+0.10)^2} = \frac{100,000}{1.21} = \$ 82,645$$

□ year 3:

Cash Flow: \$120,000

$$PV = \frac{120,000}{(1+0.10)^3} = \frac{120,000}{1.331} = \$ 90,190$$

Step 2: Sum the Present values

$$\begin{aligned} NPV &= -200,000 + 72,727 + 82,645 + 90,190 \\ &= \$ 45,562 \end{aligned}$$

Step 3: Interpretation

$$NPV = \$ 45,562 \text{ (Positive)}$$

This shows Project is considered financially Viable.
 This is indicating the Project generates more value than
 its cost, and thus it is a good investment.

Problem 2:

A Company is Considering the development of a new mobile APP. The Project requires an initial investment and the Company expects to receive cash inflows over the next 4 years. We will use the Net Present Value (NPV) Method to evaluate the Project.

Assumptions:

→ Initial Investment (year 0) = \$ 500,000

→ Expected Cash inflows:

⊗ Year 1 = \$ 150,000

⊗ Year 2 = \$ 180,000

⊗ Year 3 = \$ 220,000

⊗ Year 4 = \$ 250,000

→ Discount Rate = 12%.

Solution:

step 1: Calculate NPV

$$NPV = \sum_{t=0}^n \frac{C_t}{(1+r)^t}$$

$$\begin{aligned} t &\rightarrow 4 \\ r &\rightarrow 12\% \end{aligned}$$

For Year 0: (Initial Investment)

Cash Flow: - \$ 500,000

$$\text{Present value (PV)} = \frac{-500,000}{(1+0.12)^0} = -500,000$$

For Year 1:

Cash flow: \$150,000

$$PV = \frac{150,000}{(1+0.12)^1} = \frac{150,000}{1.12} = 133,929$$

For Year 2:

Cash flow: \$180,000

$$PV = \frac{180,000}{(1+0.12)^2} = \frac{180,000}{1.2544} = 143,479$$

For Year 3:Cash flow: ~~\$220,000~~

$$PV = \frac{220,000}{(1+0.12)^3} = \frac{220,000}{1.4049} = 156,179$$

For year 4:

Cash flow: \$250,000

$$PV = \frac{250,000}{(1+0.12)^4} = \frac{250,000}{1.5735} = 158,354$$

Step 2: Sum the PVs

$$NPV = -500,000 + 133,929 + 143,479 + 156,179 + 158,354 = \$91,941 //$$

Step 3: Interpretation.

$$NPV = \$91,941 \text{ (Positive)}$$

- The Project is viable and it can generate more value than its cost.
- The Project is Profitability ~~not~~ & financially viable.

Problems based on Benefit-Cost Ratio (BCR)

Problem 1:

Assumptions:

- Initial Investment = \$200,000
- Expected Benefits (Cash Inflows over 3 years)
 - year 1 = \$80,000
 - year 2 = \$100,000
 - year 3 = \$120,000
- Discount Rate = 10%.

Solution:

Step 1: Formula for BCR

$$BCR = \frac{\text{Present Value of Benefits}}{\text{Present value of costs}}$$

Step 2: Calculate the Present Value of Benefits
(As we did for NPV)

$$NPV = \sum_{t=0}^n \frac{C_t}{(1+r)^t}$$

For Year 1:

$$\text{Cash Inflow} = \$80,000$$

$$PV = \frac{80,000}{(1+0.10)^1} = \frac{80,000}{1.10} = 72,727$$

For Year 2:

$$\text{Cash Inflow} = \$100,000$$

$$PV = \frac{100,000}{(1+0.10)^2} = \frac{100,000}{1.21} = 82,645$$

For Year 3:

Cash Inflow: \$120,000

$$PV = \frac{120,000}{(1+0.10)^3} = \frac{120,000}{1.331} = \$90,190$$

$$\therefore PV \text{ of Benefits} = 72,727 + 82,645 + 90,190 \\ = 245,562 //$$

Step 3: Calculate Present Value of Costs

$$PV \text{ of Costs} = \text{Initial Investment}$$

$$\therefore PV \text{ of Costs} = \$200,000 \text{ (already provided)}$$

Step 4: Calculate BCR

$$BCR = \frac{245,562}{200,000} = 1.23$$

Step 5: Interpretation

$$BCR = 1.23$$

$\therefore BCR > 1$, means the benefits are greater than the costs, indicating that the software project is beneficial.

$\therefore$ For every (\$) Dollar spent on the Project, it will generate \$1.23 in benefits.

Problem 2:

Calculate the BCR for the Project given in NPV example 2. Refer the Problem 2 in NPV examples.

Outline

Basics Of PERT/CPM

PERT=Project Evaluation Review Technique

CPM = Critical Path Method

Precedence relationship

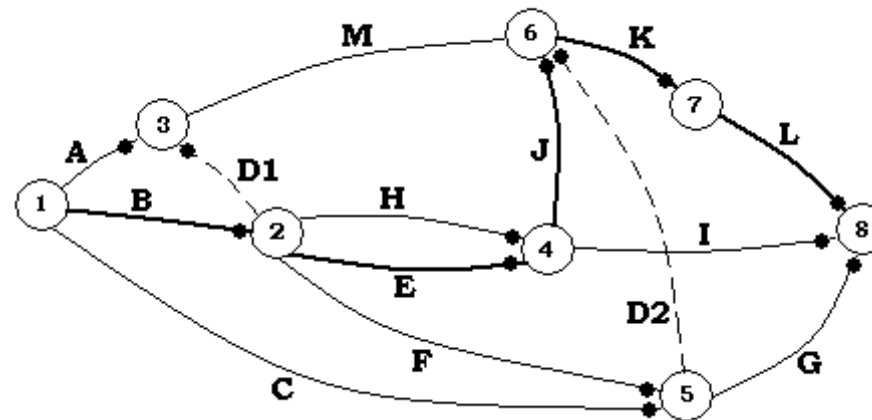
Difference between PERT and CPM

Float calculation and its importance

Why PERT/CPM?

- Prediction of deliverables
- Planning resource requirements
- Controlling resource allocation
- Internal program review
- External program review
- Performance evaluation
- Uniform wide acceptance

The CPM Diagram



- “Tasks” are Arrows
- “Events” are Circles
- “Critical Tasks” are Thick Arrows
- “Dummy Tasks” are Dashed Arrows

Critical path method: How to use CPM for project management

- Building out a project roadmap can help you visualize what needs to be done to reach your end goal.
- The critical path method helps you do exactly that. It's a project management technique that involves mapping out key tasks or critical tasks necessary to complete a project.

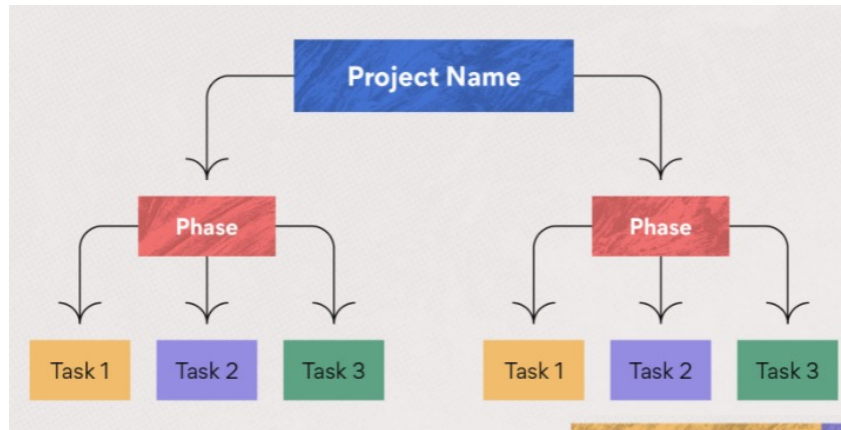
What is the critical path method (CPM)?

- The critical path method (CPM) is a technique where you identify tasks that are necessary for project completion and determine scheduling flexibilities.
- CPM revolves around discovering the most important tasks in the project timeline, identifying task dependencies, and calculating task durations.
- It helps you break down complex projects into individual tasks and gain a better understanding of the project's flexibility.

Why use the critical path method?

- CPM can provide valuable insight on how to plan projects, allocate resources, and schedule tasks.
- Improves future planning
- Facilitates more effective resource management
- Helps avoid bottlenecks

List activities



Task ID	Task	Duration (days)
A	Create outline	1
B	Write draft	5
C	Edit and create final draft	2
D	Design post visuals	4
E	Add animations to visuals	2
F	Upload post	1

Identify dependencies

Here are the task dependencies based on the example above:

- Task B is dependent on A
- Task C is dependent on B
- Tasks C and D can run in parallel
- Task E is dependent on D
- Task F is dependent on C, D, and E

The list of dependent tasks is referred to as an activity sequence, which will be used to determine the critical path.

Calculate the float

- Float, or slack, refers to the amount of flexibility of a given task. It indicates how much the task can be delayed without impacting subsequent tasks or the project end date.
- Finding the float is useful in gauging how much flexibility the project has. Float is a resource that should be used to cover project risks or unexpected issues that come up.
- Critical tasks have zero float, which means their dates are set. Tasks with positive float numbers belong in the non-critical path, meaning they may be delayed without affecting the project completion date. If you're short on time or resources, non-critical tasks may be skipped.
- Calculating the float can be done with an algorithm or manually. Use the calculations from the section below to determine the total float and free float.

Total float vs. free float

- Total float: This is the amount of time that an activity can be delayed from the early start date without delaying the project finish date or violating a schedule constraint. Total float = $LS - ES$ or $LF - EF$
- Free float: This refers to how long an activity can be delayed without impacting the following activity. There can only be free float when two or more activities share a common successor. On a network diagram, this is where activities converge. Free float = $ES \text{ (next task)} - EF \text{ (current task)}$

Calculate the critical path

Here are the steps to calculate the critical path manually:

Step 1: Write down the start and end time next to each activity.

The first activity has a start time of 0, and the end time is the duration of the activity.

The next activity's start time is the end time of the previous activity, and the end time is the start time plus the duration.

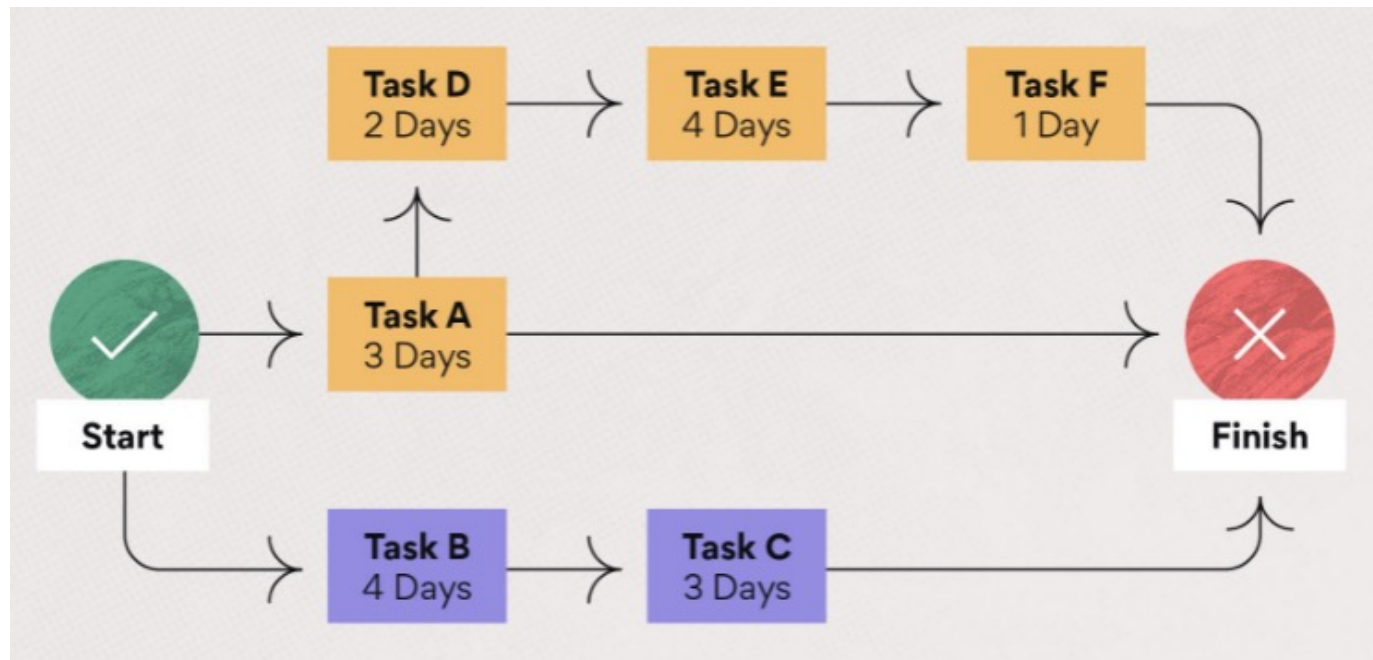
Do this for all the activities.

Step 2: Look at the end time of the last activity in the sequence to determine the duration of the entire sequence.

Step 3: The sequence of activities with the longest duration is the critical path.

Using the same example above, here's what the critical path diagram might look like:

Calculate the critical path



Finding the critical path



1. List project tasks and details



2. Identify task dependencies for the project



3. Create a network diagram of tasks



4. Estimate each task duration

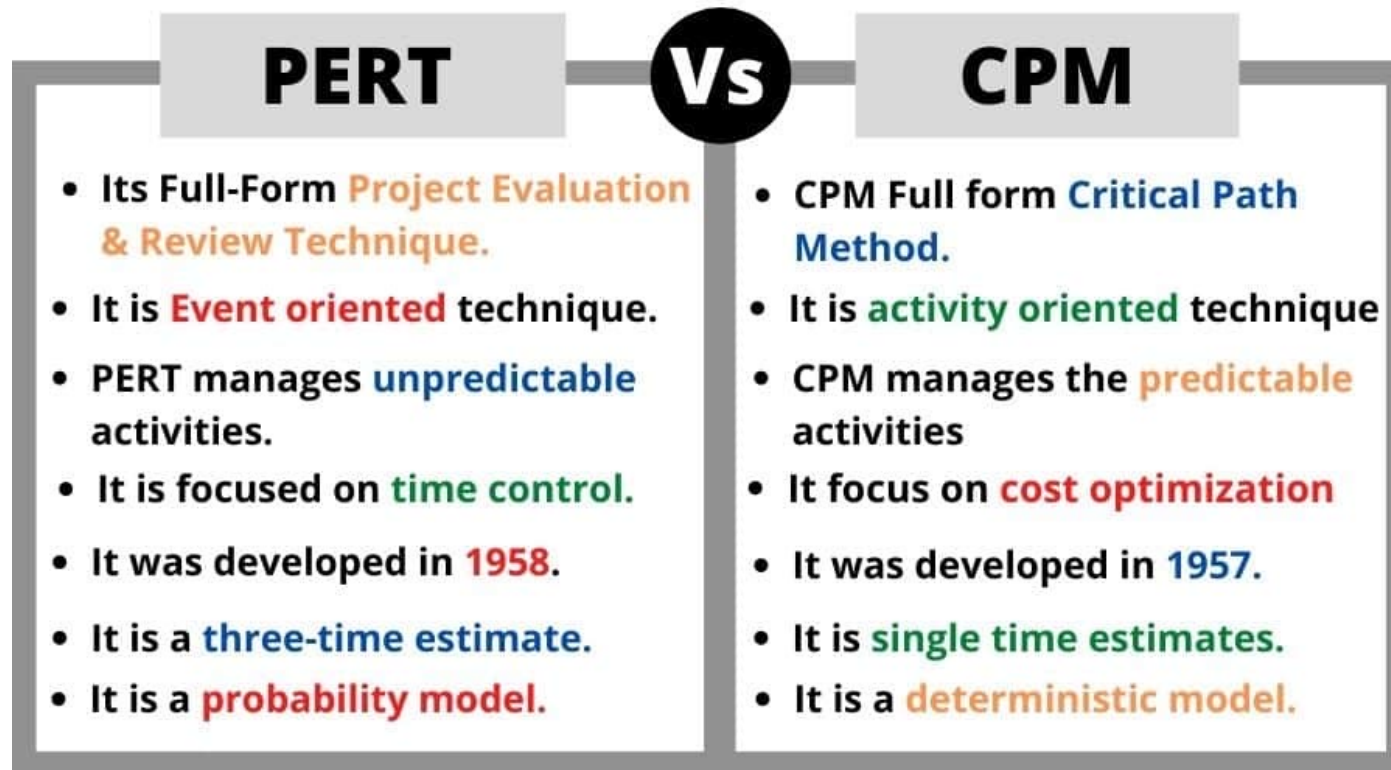


5. Find the critical path based on longest sequence



6. Calculate the total float

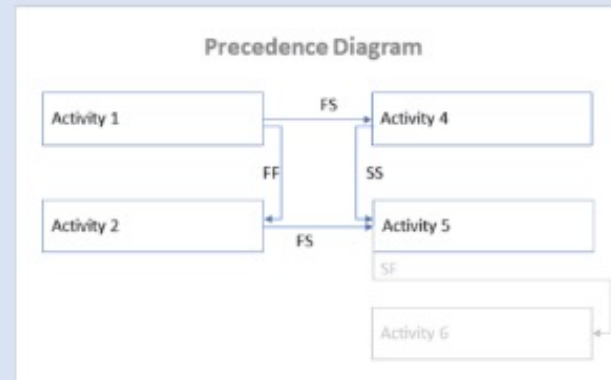
PERT vs CPM



Precedence Relationship

Precedence Diagramming Method (PDM)

- Definition
- Types of Logical Relationships
i.e. FS, FF, SS, SF
- Example of Sequencing Activities with PDM

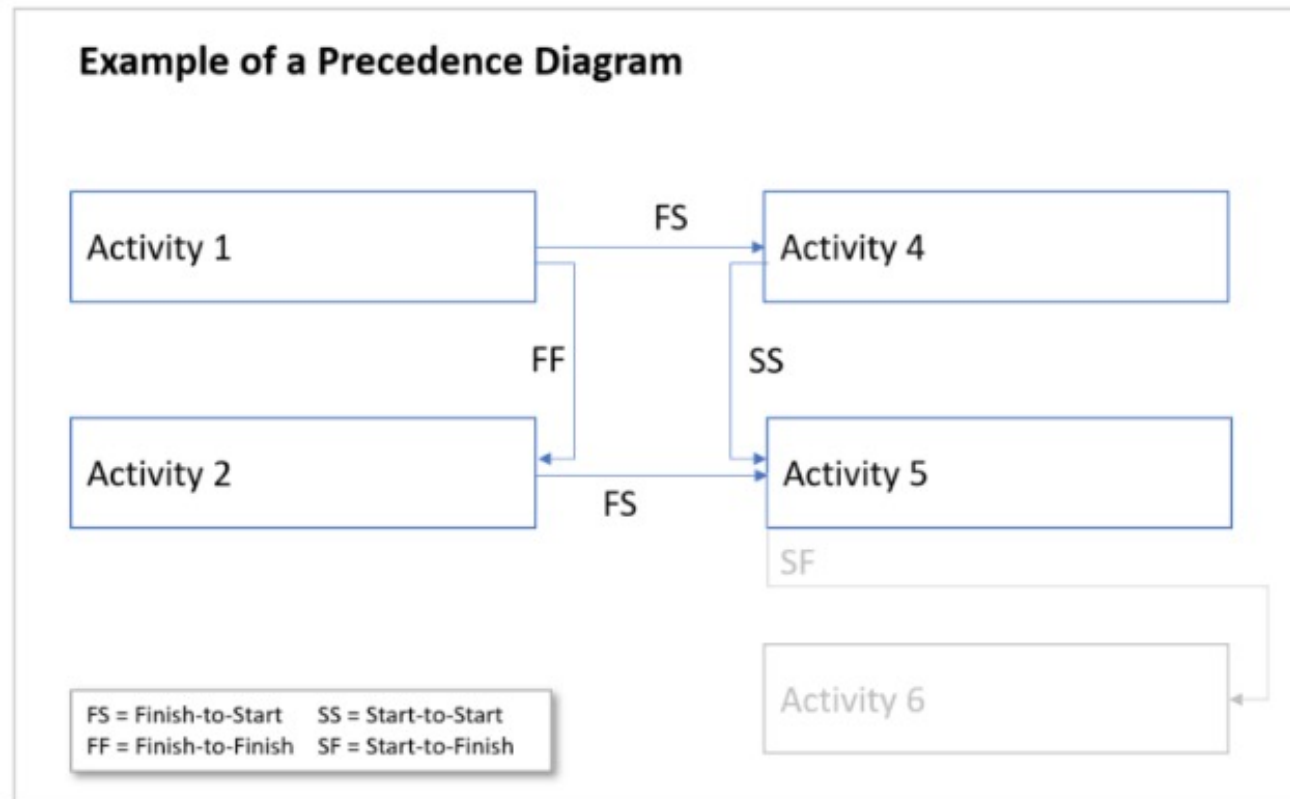


PDM – Precedence Diagramming Method [FS, FF, SS, SF]

What Is the Precedence Diagramming Method (PDM)?

- PDM is a scheduling technique of the 'sequence activities' process in PMI methodology.
- The precedence diagramming method can be used for creating a project schedule network diagram which is an output of this process that is used as an input for the 'develop schedule' process.
- The method involves the identification and visualization of the sequence and dependencies of activities in a project or a part of a project, such as a workstream or a work package.

What Is the Precedence Diagramming Method (PDM)?



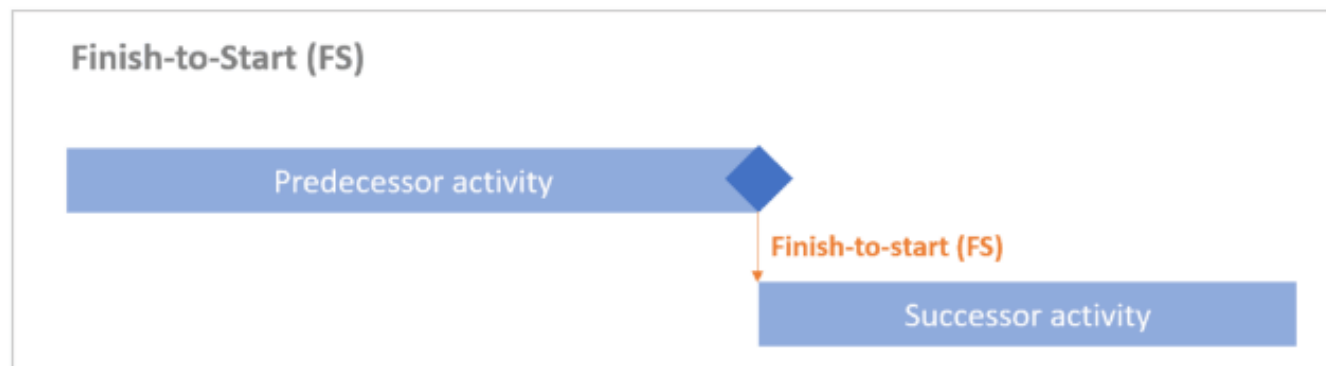
What Are the Types of Dependencies in PDM?

The 4 types of logical relationships in the precedence diagramming method are:

- Finish-to-Start (FS) dependency,
- Finish-to-Finish (FF) dependency,
- Start-to-Start (SS) dependency, and
- Start-to-Finish (SF).

Finish-to-Start Relationship (FS)

If two activities are linked by a finish-to-start dependency, this means that the predecessor activity must be completed before the successor activity can start.



Finish-to-Finish Relationship (FF)

Under this type of logical relationship, a successor activity requires the predecessor activity to be finished before it can be completed.

This type of dependency can also coincide with start-to-start relationships of the same activities.



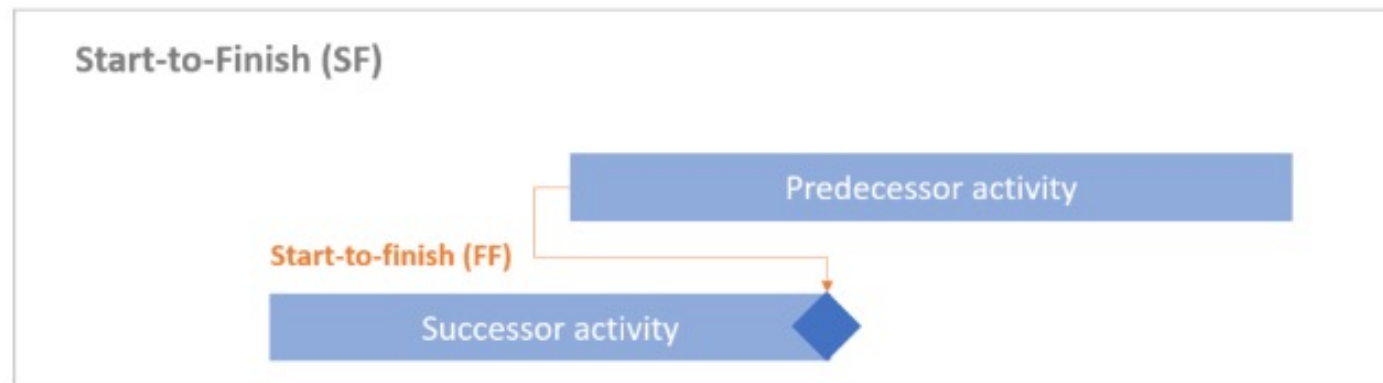
Start-to-Start Relationship (SS)

If two activities are connected through a start-to-start dependency, this means that the predecessor activity must have started before the successor activity can start.



Start-to-Finish Relationship (SF)

This logical relationship requires that a predecessor activity must have started before the successor activity can be finished. In practice, this type of dependency does not occur very often which is also acknowledged by the PMBOK.



Starting Point: Task Primary Properties

- **Prerequisite task set (may be empty)**
- **Optimal Staffing**
- **Duration at Optimal Staffing Level**
- **Crash Limit**
- **Fixed and Variable costs**

If a task seems too complex or
involved to easily determine primary
properties . . .

Break the task up into simpler tasks . .
.

Or create a CPM sub-project.

We will use PERT/CPM Analysis to determine Task Secondary properties:

- Tail Event and Head Event
- Earliest Start, Earliest Complete
- Latest Start, Latest Complete
- Critical / Non-Critical Status
- Total Float, Free Float
- Scheduled Start, Scheduled Complete
- Actual Staffing, Duration, and Variable Costs

**We will then use Task Secondary Properties
to generate Project Management Tools:**

- **Gantt Chart (Project Schedule)**
- **Manpower Chart**
- **Expenditure Curves**
- **Project Completion (PC)**

Generate Initial CPM Diagram

- Must strictly enforce all prerequisite relationships.
- Number of events is initially unknown
- Critical path is initially unknown
- Iterative Process
- Try to minimize number of Dummy Tasks

CPM Hint #1

- **Add or remove events at your pleasure.**
- **Do not number events until last.**

CPM Hint #2

- **The initial event is the Tail Event for all tasks which have empty prerequisite sets (Initial Tasks).**
- **The Final Event is the Head Event for all tasks which are not members of any prerequisite set (Final Tasks).**

CPM Hint #3

- **Tasks which have identical prerequisite sets have the same Tail Event**

CPM Hint #4

- Starting with the Final Tasks, work backwards, enforcing the smallest prerequisite sets first.
- Use Dummy Tasks to enforce any prerequisites in large sets which have already been enforced in a smaller set.

Finish CPM Diagram

- Remove all redundant Dummy Tasks
- Remove all redundant Events
- Number all remaining events
- Not really finished . . haven't identified critical tasks yet.

Generate PERT Chart: Enter Data for Each Task

- **Task Symbol**
- **Tail Event**
- **Head Event**
- **Task Duration (TD)**

**Forward Pass:
Determine Earliest Start (ES) and Earliest Complete
(EC) for each Task**

- For all Initial Tasks, $ES = 0$
- Once ES is Determined, EC equals ES plus TD.
- The ES for all tasks with tail [i] is equal to the largest value of EC for all tasks with head [i].
- PC is the largest value of EC for all Final Tasks.

Backward Pass:
Determine Latest Start (LS) and Latest
Complete (LC) for each Task

- For all Final Tasks, $LC = PC$
- Once LC is Determined, LS equals LC minus TD.
- The LC for all tasks with head [j], is equal to the smallest value of LS for all tasks with tail [j].
- At least one Initial Task must have $LS = 0$; none may be negative.

Determine Total Float (TF):
Allowable delay in start of task which will not
delay Project Completion

- For task with tail [i] and head [j],
$$TF[i,j] = (LC[j] - ES[i]) - TD[i,j]$$
- ES[i] is earliest start for all tasks with tail [i].
- LC[j] is latest complete for all tasks with head [j].

Determine Free Float (FF):
Allowable delay in start of task which
will not delay start of any other task.

- For task with tail [i] and head [j],
$$FF[i,j] = ES[j] - ES[i] - TD[i, j]$$
$$= ES[j] - EC[i,j]$$
- If [j] is the final event, use PC for ES[j]

Determine Critical Path

- All Tasks with zero Total Float are Critical.
- Any delay in these Tasks will delay Project Completion.
- Darken these Tasks to finish CPM Diagram.

Lecture on

Network Diagram Construction for PERT/CPM

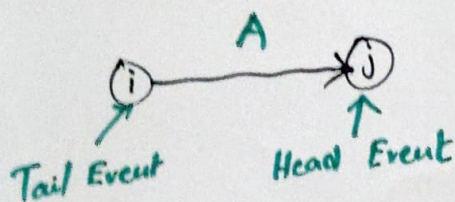
1. Types of Activities: Start, Finish, Parallel, Dummy, & Dangling
2. PERT/CPM networks contains TWO Major Components.

1. Activity (Arrows)

2. Events (Circles)

3. Activity Duration

4. Completion.

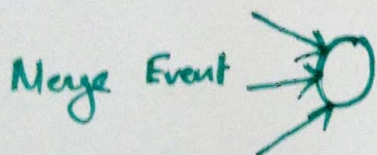


A - Activity

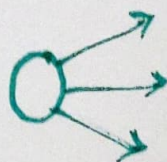
i, j - Events

→ Merge & Burst Events

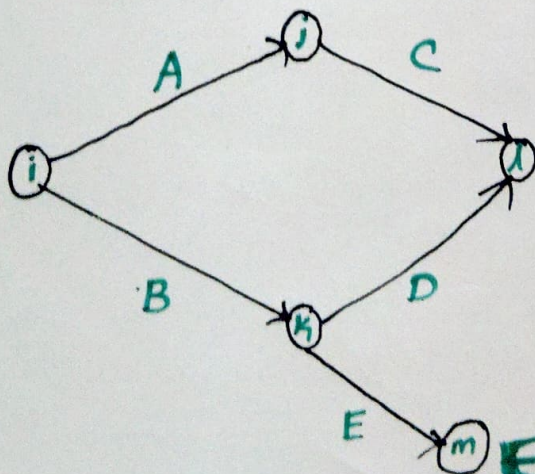
One or more activities can start and end simultaneously.



Merge Event



Burst Event

→ Preceding and Succeeding Activities

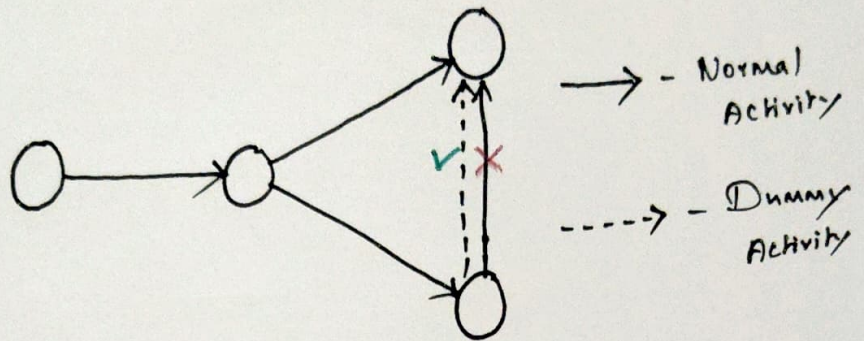
A, B - Preceding

C, D - Succeeding Activities

E Dangling Activity

→ Dummy Activity

⊗ An Imaginary activity does not consume any resource and time is called dummy activity.



2. Rules For Constructing a Network Diagram.

1. An activity can be represented only once.
 → Length of arrow has no significance.

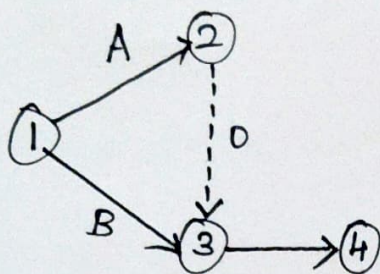
2. Event number 1 is the START EVENT and
 Highest event number is END EVENT

3. Before starting an ~~activity~~ activity, all preceding activities must have finished.
 → Activities must follow LOGICAL SEQUENCE.

4. Dummy Activity must be used only if it is necessary. (To reduce Complexity of Network)

5. A network should have ONE START EVENT
 and ONE END EVENT.

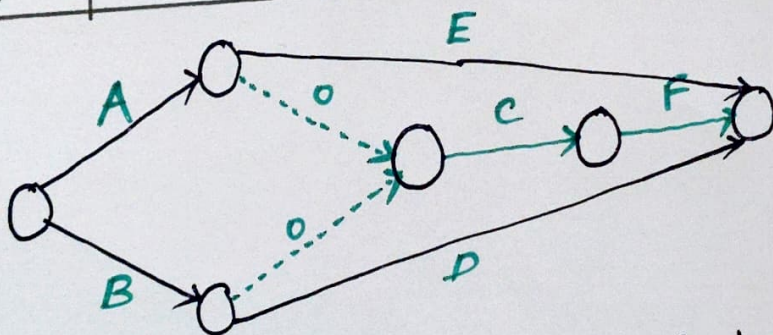
Example Dummy Activity ①



Activity	Preceding / Succeeding	Duration
A	1 → 2	5
B	1 → 3	5

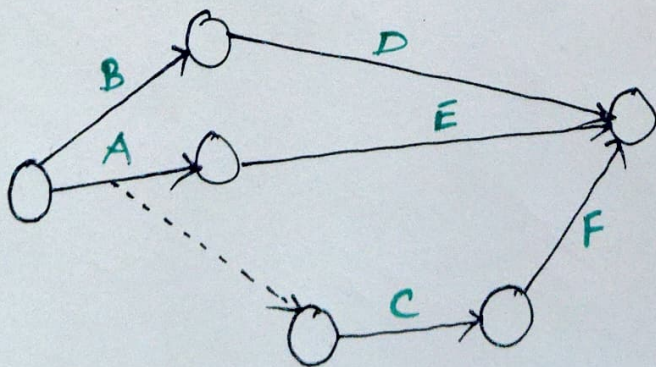
Example Dummy Activity ② → Used for Precedence Relationships

Activity	A	B	C	D	E	F
Preceding Activity	-	-	A, B	B	A	C



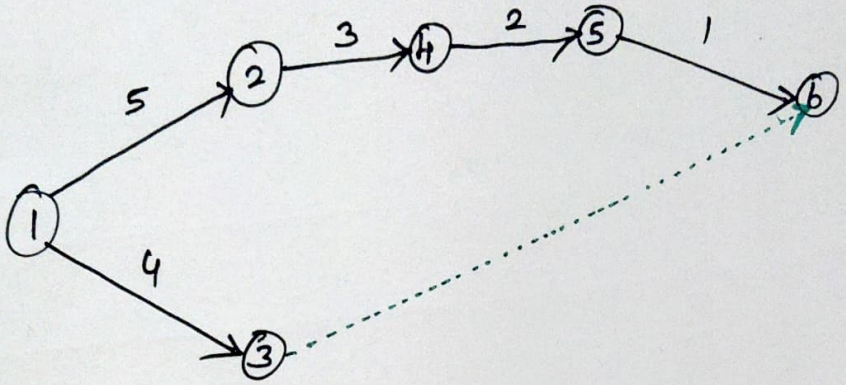
Example Dummy Activity ③ [If Second activity can be started after part of first activity]

Activity	A	B	C	D	E	F
Preceding Activity	-	-	sox. Completion of A	B	A	C



Example Dummy Activity (4) [To Connect Dangling Activities]

Activity	1-2	1-3	2-4	4-5	5-6
Preceding Activity	5	4	3	2	1



CPM AND PERT

Critical Path Method (CPM) – Basics:

The CPM is a technique for analyzing, planning and scheduling large complex projects. The technique aims at completing the project in time and within the budget. The technique can be applied only if the project can be divided into a number of well defined activities.

Some Basic Points:

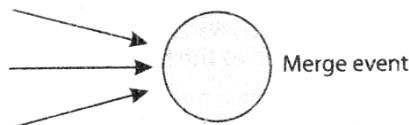
- (I) A project consists of different types of activities:
 - (a) Start activity: it is one which does not have any preceding activity.
 - (b) Finish or terminal activity: it is one which does not have any succeeding activity.
 - (c) Concurrent or parallel activity: these activities can be started simultaneously.
 - (d) Dummy activity: it is used merely to show the logical dependencies between the activities. It is an imaginary activity that does not consume any time. It is represented by a dotted arrow.
 - (e) Dangling activity: an activity (other than the terminal activity) that does not have a succeeding activity is termed as a dangling activity. It is connected to the last node through a dummy activity.

The two major components of PERT/CPM network are as follows:

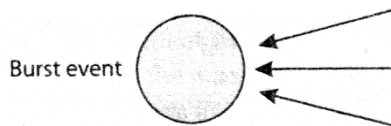
- (II) **Events:** Events in the network diagram represent project milestones. It represents a point in time satisfying the completion of some activities and the beginning of new ones. This is generally represented by a circle 'O' in a network which is also called a node or connector.

The events can be further classified into following two categories:

- (a) **Merge Events:** When more than one activity comes and joins an event. Such event is known as merge events.



- (b) **Burst event :** When more than one activity leaves an event, such event is known as burst event.



- (III) **Activity Duration:** This can be defined as time taken to complete a given activity.

- (IV) **Completion:** The project is said to be completed if and only if all the activities of the project are complete.

Networking:

CPM is used to coordinate and schedule the sequential activities of a project. It presents the activities of a project as a net work. Network graphically presents the how each activity is related to the others. When all the activities are connected with each other in a logical manner, it gives rise to a network.

The network is drawn using the NODE -ARC principle. Nodes are circles indicating either the start of an activity or finish of an activity or start of one activity and finish of the other activity. An arrow joining any two nodes is called as ARC and each ARC represents a given activity on the network. The networks are not drawn to scale. While drawing the networks, as far as possible, the crisscrossing should be avoided.

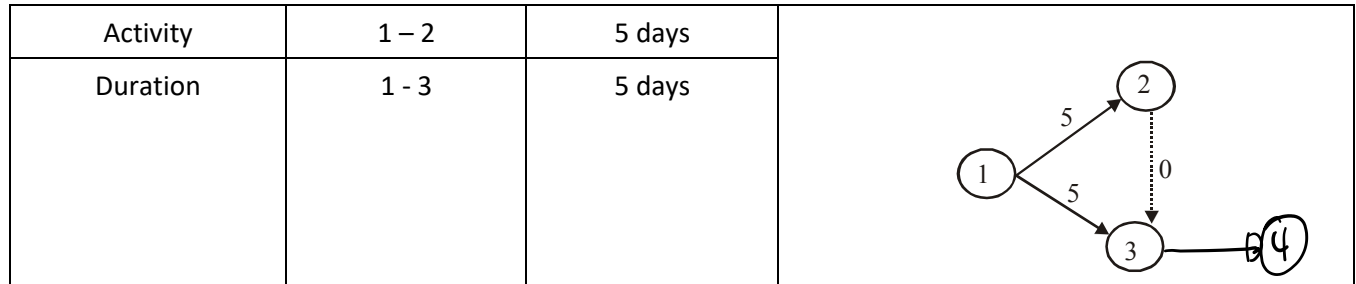
Dummy activity:

It is used merely to show the logical dependencies between the activities. It is an imaginary activity that does not consume any time. It is represented by a dotted line.

Dummy activities are inserted in following cases:

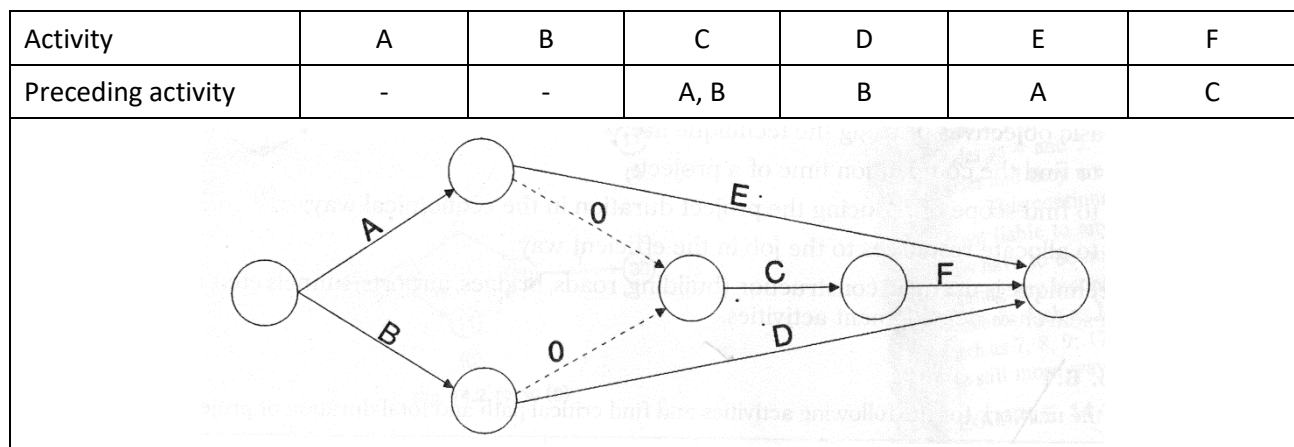
- (i) In network, two or more activities are not allowed to have the same starting ending nodes. Hence, if two activities begin and end at the same time, a dummy activity is inserted into the network to distinguish the two activities.

Example:



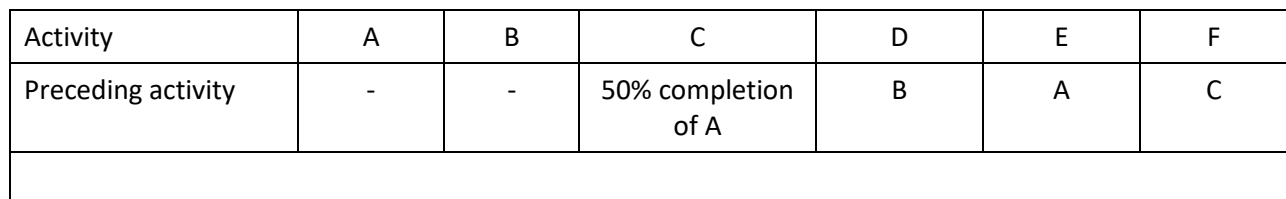
- (ii) Dummy activity is used to identify precedence relationships correctly.

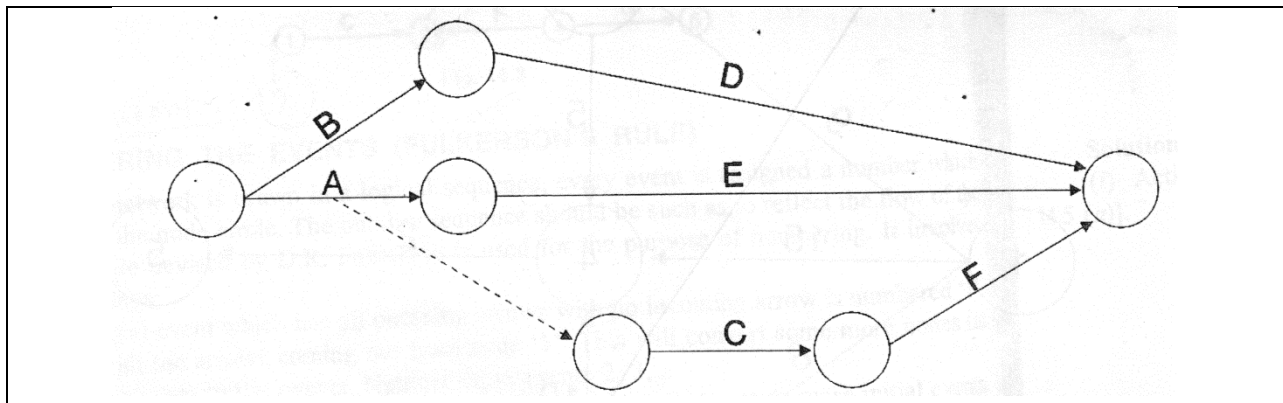
Example:



- (iii) If a second activity can be started after part of a first activity is complete, a dummy activity is inserted.

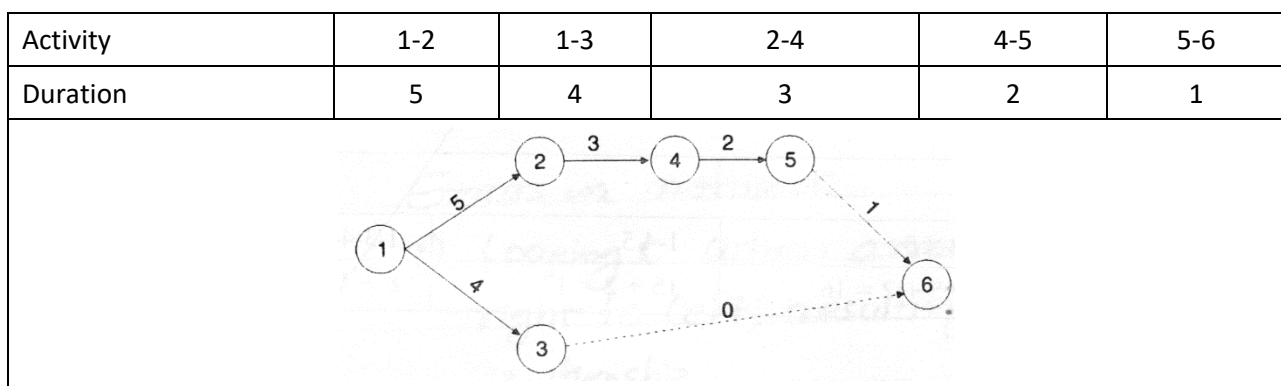
Example:





(iv) Dummy is inserted to connected the dangling activities

Examples:



Critical Path:

The longest-duration path through the network is known as critical path (It is the minimum time required to complete the project). Activities that lie on the critical path cannot be delayed without delaying the completion time for the whole project; such activities are known as critical activities.

Basic Objectives:

The basic objectives of using the technique are:

- to find the competition time of a project;
- to find scope of reducing the project duration in the economical way;
- to allocate resources to the job in the efficient way.

The technique is useful in construction (building, roads, bridges, airports, tunnels etc.), maintenance and research & development activities.

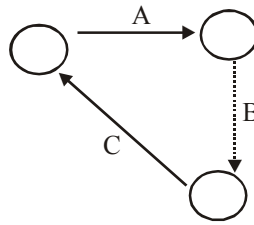
NUMBERING THE EVENTS (FULKERSON'S RULE)

After the network is drawn in a logical sequence, every event is assigned a number which is placed inside the node circle. The number sequence should be such as to reflect the flow of the network. The rule devised by D.R. Fulkerson is used for the purpose of numbering. It involves the following steps:

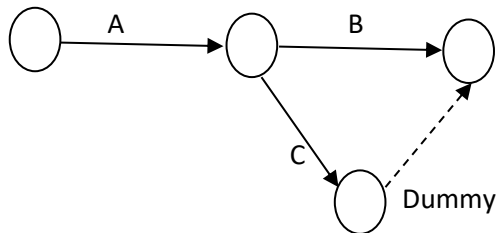
- The initial event which has all outgoing arrows with no incoming arrow is numbered '1'.
- Delete all the arrows coming out from node '1'. This will convert some more nodes (at least one) into initial events. Number these events 2, 3,...
- Delete all the arrows going out from these numbered events to create more initial events. Assign the next numbers to these events.
- Continue until the final or terminal node, which has all arrows coming in with no arrow going out, is numbered.

(3) Errors in Network :

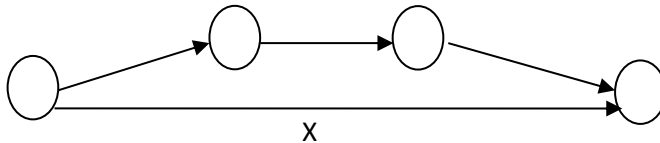
- 1) Looping: When arrows are drawn from right to left, resulting into error in relationship



- 2) Dangling: When an activity (other than terminal activity) is not connected to any succeeding activity. It should be connected to terminal activity by a dummy activity.



- 3) Redundancy: When unnecessary predecessors are used. It should be eliminated.



NETWORK CONSTRUCTION

Q1. Draw a network for the simple project of erection of steel works for a shed. The various activities of the project are as under:

<u>Activity</u>	<u>Description</u>	<u>Preceded by</u>
A	Erect site workshop	—
B	Fence site	—
C	Bend reinforcement	A
D	Dig foundation	B
E	Fabricate steel work	A
F	Install concrete pillars	B
G	Place reinforcement	C, D
H	Concrete foundation	G, F
I	Erect steel work	E
J	Paint steel work	H, I
K	Give finishing touch	J

Q2. Draw the network diagram for the following list of activities :

Activity	Immediate predecessor	Activity	Immediate predecessor
A	—	L	K
B	A	M	K
C	B	N	K
D	C	O	D
E	D	P	O
F	E	Q	B
G	E	R	N
H	C	S	L,M
I	C,F	T	S
J	G,H,I	U	P,Q
K	J	V	U

Q3. Depict the following dependency relationships by means of network diagrams. The alphabets stand for activities.

- (i) A and B control F; B and C control G
- (ii) A and B control F; B controls G while C controls G and H.
- (iii) A controls F and G; B controls G while C controls G and H.
- (iv) F and G are controlled by A; G and H are controlled by B with H controlled by and C.
- (v) A controls F, G and H, B controls G and H with H controlled by C.

Q4. The activities along with their dependency relationship are given below. Draw an arrow diagram.

Activity:	A	B	C	D	E	F	G	H	I
Immediate Predecessor:	—	—	—	C	A, B	C	E, F	D	F, H

Q5. The activities along with the dependency relationship are given below. Draw the arrow diagram.

Activity	Immediate predecessor	Activity	Immediate predecessor
A	—	G	B, C
B	—	H	C
C	—	I	E,F
E	A	J	G,H
F	A,B	K	H

CRITICAL PATH

Q6. Draw the network for the following activities and find critical path the total duration of project:

Activity	Duration (months)	Activity	Duration (months)
1-2	1	3-6	1.5
2-3	4	4-7	3.5
2-4	5	5-7	2.5
3-4	3	6-7	1
4-5	1.5	7-8	4

Q7. Draw the network for the following activities and find critical path and total duration of project:

Activity	Duration (months)	Activity	Duration (months)
1-2	2.5	4-5	2.0
2-3	2.5	5-6	3.0
2-4	1.5	6-7	1.5
3-4	1.0	5-7	1.5
3-5	1.0		

Q8. Draw the network for the following activities and find critical path and total duration of project:

Activity	Dependence	Duration (Days)	Activity	Dependence	Duration (Days)
A	—	6	F	C	3
B	—	3	G	D	5
C	A	5	H	B, E	5
D	A	4	I	D, H	2
E	A	3	J	F, G, I	3

ANALYSIS OF TIMING

This is to be studied in two parts:

- Computing Earliest start (this is referred as forward numbering process): Earliest time at which the activity can start.
- Computing Latest finish (this is referred as backward numbering process): Latest time by which the activity should be finished without delaying the project completion.

Forward Numbering Process: (Earliest start)

- Start putting zero in the first node. It is designated as 'E'.
- Proceed sequentially to the next adjacent node.
- If there is more than one way of reaching to the next adjacent node, find the maximum time of reaching and write on top at the next node as 'E'.
- Number on the top of node indicates the earliest possible time to start the activity.
- Proceed in this way till terminal node is reached.

Backward Numbering Process: (Latest finish)

- (i) Write the number obtained in the terminal node in the forward process as latest finish. It is designated by L.
- (ii) Work sequentially backward by travelling through head to tail of the arrow.
- (iii) If there are more than one ways of reaching back, find the maximum time of reaching back.

$L = \text{critical path duration} - \text{the maximum time of reaching back.}$

- (iv) Proceed in this way, till start node is reached.

Connected the nodes in a sequential order which has got both – forward and backward numbers identical. The activities so connected are known as critical activities. The path is known as critical path.

Head event slack: It is the difference between L and S at the end node of an activity.

Tail event slack: It is the difference between L and S at the starting node of an activity.

Float Analysis

Float is defined as the maximum permissible delay that can be tolerated without delaying the total project completion. Float can be associated only with non-critical activities. Float of a critical activity is zero. The critical path has no float.

Writing the Float Table:

- (i) Draw the net work diagram and note the E and L
- (ii) Write the activities and their respective durations in columnar form.
- (iii) Write the respective earliest start time of different activities (with the help of diagram prepared under (i))
- (iv) Write the respective earliest finish time of different activities. Earliest finish time is $E + \text{activity duration}$.
- (v) Write the respective latest finish time of different activities (with the help of diagram prepared under (i))
- (vi) Write the respective latest start time of different activities. Latest start time is $L - \text{activity duration}$.

Total float: The total float of an activity is the amount of time by which it may be extended or delayed without delaying completion of the project.

A critical activity can be defined as one which has a total float of zero.

Total Float is the difference of LS and ES or LF and EF, i.e. $LS - ES$ or $LF - EF$.

Free float Free float is an amount of time that an activity can be delayed without delaying the start of its succeeding activity. It is computed as total float minus head event slack.

Independent float

It is that amount of time within which an activity can be delayed without delaying the completion of the project even if the preceding activities have consumed their total float.

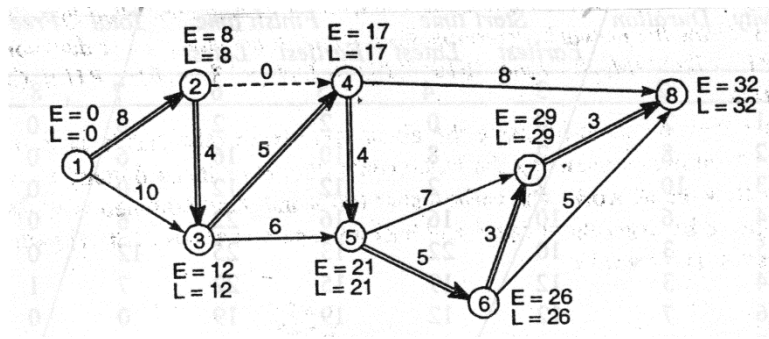
It is computed as free float minus tail event slack. (If negative, it is taken as zero).

Interference float is the difference between total float and free float.

Q9. The utility data for a network are given below. Determine the total, free, independent and interfering floats and identify the critical path.

Activity:	0-1	1-2	1-3	2-4	2-5	3-4	3-6	4-7	5-7	6-7
Duration:	2	8	10	6	3	3	7	5	2	8

Q10. For the network given in fig. determine the total, free, independent and interfering floats for each activity. Times for activities are in months.



INT422

IT Project Management

PERT and CPM

Program Evaluation and Review Technique

Definition:

PERT, which stands for Program Evaluation and Review Technique, is a **project management method used for planning, scheduling, and coordinating tasks** within a project.

It is particularly **useful** in situations **where the activities involved are uncertain, complex, or have interdependencies.**

Key Components and Features of PERT

1. Probabilistic Time Estimates:

- PERT incorporates three time estimates for each activity:
 - **Optimistic Time (O)**: The **shortest time** an activity can be completed under **favorable conditions**.
 - **Pessimistic Time (P)**: The **longest time** the activity might take under **unfavorable conditions**.
 - **Most Likely Time (M)**: The **best estimate of the time** required under **normal conditions**.
- These estimates are used to calculate the **expected duration (TE)** using the formula
$$TE = (O + 4M + P) / 6$$

Key Components and Features of PERT

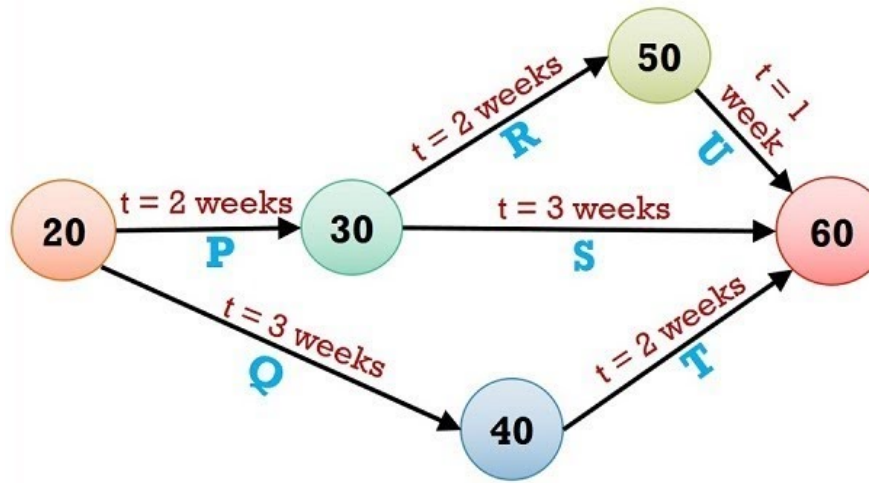
2. Network Diagrams:

- PERT uses **network diagrams** (PERT charts) to **represent** the **sequence of activities and their dependencies**. **Nodes** represent **activities**, and **directed arrows** show the flow and dependencies between them.

Key Components and Features of PERT

3. Critical Path Analysis:

The critical path is the **longest path through the network**, determining the **minimum project duration**. Activities on the critical path have no float or slack, meaning **any delay** in these activities **directly impacts the project's completion time**.



Key Components and Features of PERT

4. Probability Analysis (Optional):

PERT allows for the **calculation of the probability of completing the entire project within a specified time frame**. This involves using probability distributions to account for uncertainties in activity durations.

How PERT Works ?

1. Activity Identification:

- Identify and list all the activities required for the project.

2. Time Estimation:

- For each activity, determine optimistic, pessimistic, and most likely time estimates. Calculate the expected duration using the formula.

3. Network Construction:

- Develop a network diagram that visually represents the sequence and dependencies of activities.

How PERT Works ?

4. Critical Path Analysis:

- Identify the critical path by calculating the expected duration for each path in the network. The critical path is the sequence of activities with the longest duration.

5. Probability Analysis (Optional):

- Optionally, perform probability analysis to **assess the likelihood of completing the entire project** within a specified time frame.

6. Float/Slack Calculation:

- Determine the **float or slack for non-critical activities**, representing the flexibility in their start and finish times.

How PERT Works ?

7. Project Control and Monitoring:

- **Monitor** the progress of **activities**, especially those **on the critical path**, and **take corrective actions** to **prevent delays**. **Adjust project timelines** based on actual progress.

8. Periodic Review and Adjustment:

- Periodically **review the project's progress** and **adjust time estimates** or **schedules** based on **real-world developments** or **changes** in project requirements.

Critical Path Method (CPM)

- The Critical Path Method (CPM) is a **project management technique** used for **planning, scheduling, and managing complex projects**. **CPM** is particularly **effective** when the activities within a project are well-defined and have known durations.

Key Components and Features in CPM

1. Deterministic Time Estimates:

- Unlike PERT, CPM uses deterministic time estimates, where a single time estimate is assigned to each activity. This estimate represents the expected time for the activity to be completed.

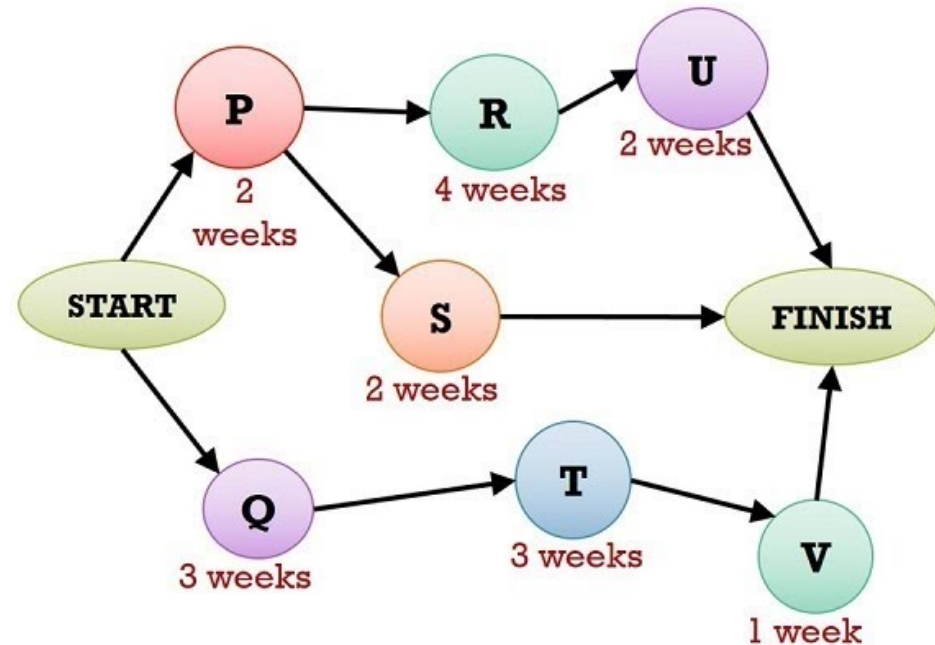
2. Network Diagrams:

- Similar to PERT, CPM uses network diagrams to represent the sequence of activities and their dependencies. Nodes represent activities, and directed arrows show the logical flow and dependencies between them.

Key Components and Features in CPM

3. Critical Path Analysis:

- The critical path in CPM is the sequence of activities that **determines the minimum project duration**. Unlike PERT, **CPM focuses on deterministic estimates and identifies the longest path through the network**.



Key Components and Features in CPM

4. Float or Slack Calculation:

- CPM calculates the float or slack for each activity, representing the amount of time an activity can be delayed without delaying the project's overall completion. Activities on the critical path have zero float.

How CPM works ?

1. Activity Identification:

- **Identify and list all the activities** required for the project.

2. Time Estimation:

- **Assign a deterministic time estimate to each activity.** This is the expected time it will take for the activity to be completed.

3. Network Construction:

- **Develop a network diagram** that visually represents the **sequence and dependencies of activities.**

4. Critical Path Analysis:

- **Identify the critical path by calculating the total duration for each path in the network.** The critical path is the sequence of activities with the longest total duration.

How CPM works ?

5. Float/Slack Calculation:

- Determine the float or slack for each activity by subtracting its expected duration from the duration of the critical path. Activities on the critical path have zero float.

6. Project Control and Monitoring:

- Monitor the progress of activities, especially those on the critical path. Focus on managing and controlling activities to ensure they are completed on time.

7. Periodic Review and Adjustment:

- Periodically review the project's progress and adjust schedules based on actual achievements and changes in project requirements.

PERT vs CPM

Technique

PERT is a project management technique used to measure uncertain activities in a project and focuses on time planning and control.

CPM is a statistical project management technique that manages the well-defined activities of a project, such as time and cost management.

Focus

PERT focuses on minimizing the time required to complete a project by the designated deadline.

CPM focuses on a trade-off between cost and time with a significant emphasis on minimizing costs.

PERT vs CPM

Evolution

PERT evolved as a suitable technique for research and development projects.

CPM is best suited for non-research projects like construction.

Orientation

While PERT is event-oriented because it focuses on time.

CPM is often activity-oriented.

PERT vs CPM

Model

PERT uses a probabilistic model.

CPM is a deterministic model.

Time Estimates

PERT has three-time estimates and helps make high-precision time estimates.

CPM has only a onetime estimate and is appropriate for reasonable time estimation.

PERT vs CPM

Activities Type

While the PERT technique manages unpredictable activities.

The CPM can manage predictable activities effectively.

Job Nature

PERT is beneficial for the non-repetitive types of jobs.

CPM involves jobs with a repetitive nature.

PERT vs CPM

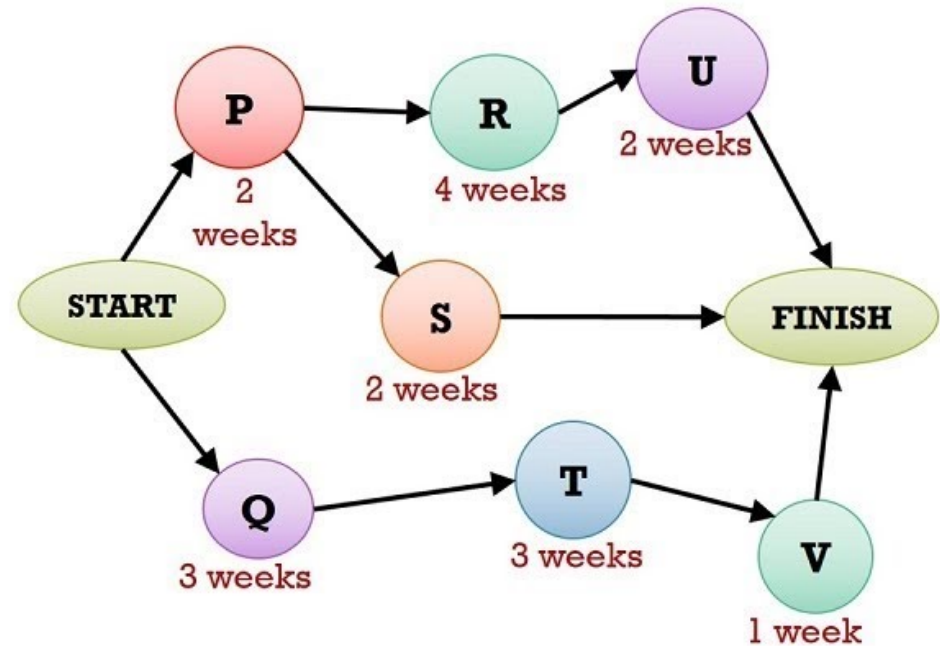
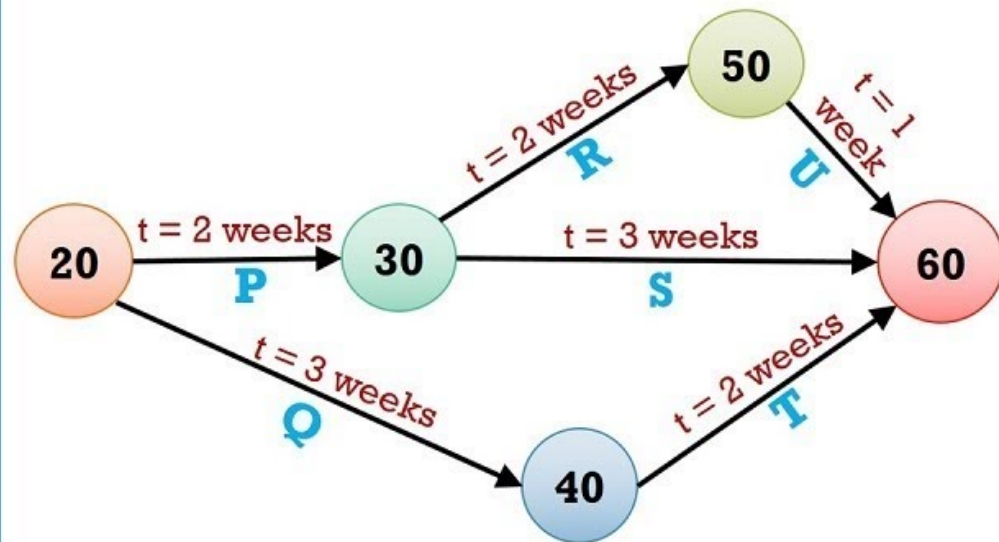
Crashing Concept

In comparison, the crashing concept does not apply to PERT as there's no time certainty.

Crashing is a compression technique used in CPM to reduce the project duration and associated costs.

Key Components and Features in CPM

PERT Vs CPM



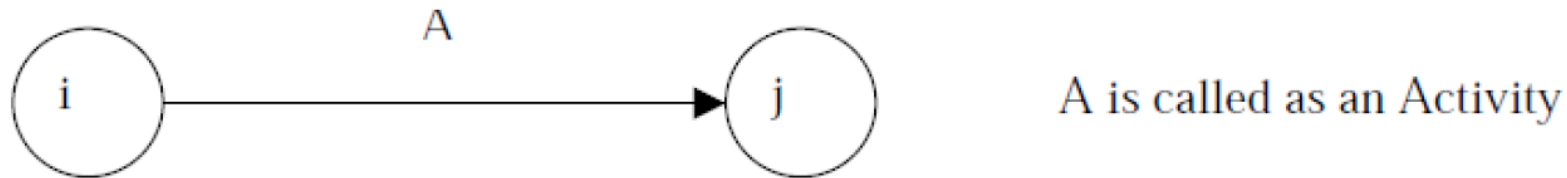
Key Differences

PERT / CPM Network Components

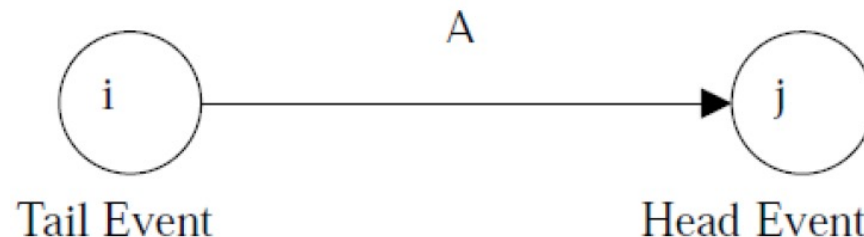
PERT / CPM networks contain two major components

1. Activities
2. Events

Activity: An **activity** represents an **action and consumption of resources** (time, money, energy) required to complete a portion of a project. **Activity** is represented by an **arrow**



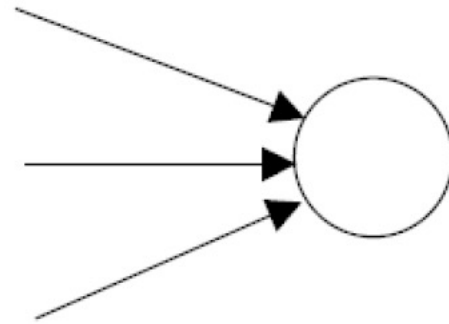
Event: An **event (or node)** will always occur at the **beginning and end of an activity**. The event has no resources and is represented by a **circle**. The **ith** event and **jth** event are the **tail event** and **head event** respectively



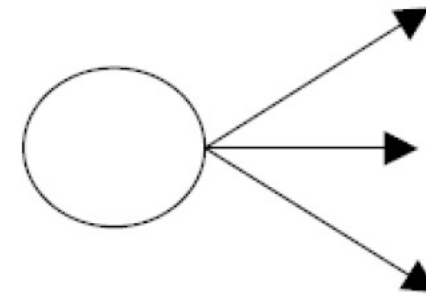
PERT / CPM Network Components

Merge and Burst Events

- ❖ One or more activities can start and end simultaneously at an event as shown below:



(a) Merge Event

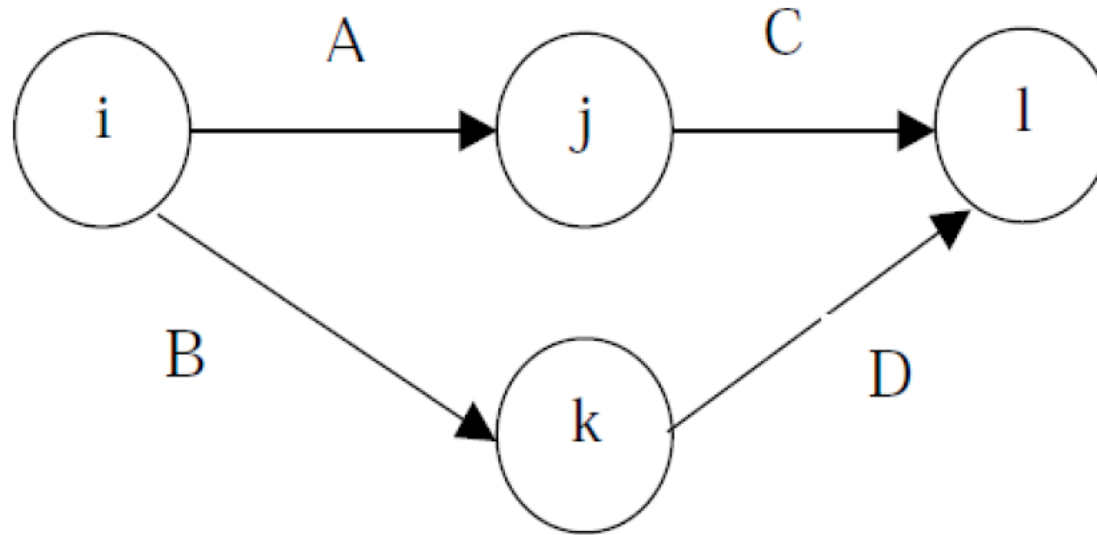


(b) Burst Event

PERT / CPM Network Components

Preceding and Succeeding Activities

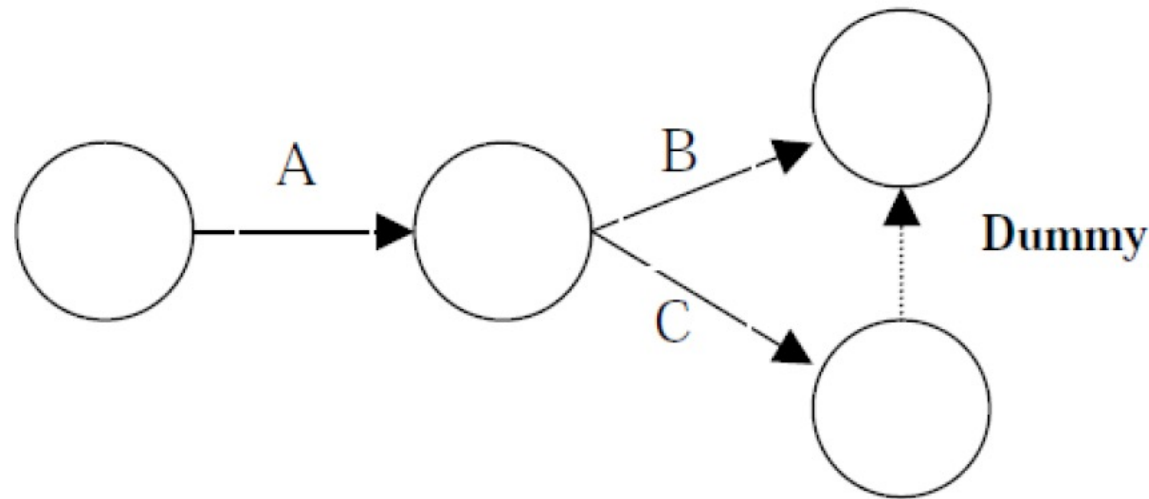
❖ **Activities performed before** given **events** are known as **preceding activities** and **activities performed after a given event** are known as **succeeding activities**. In the following figure, activities A and B precede activities C and D respectively.



PERT / CPM Network Components

Dummy Activity

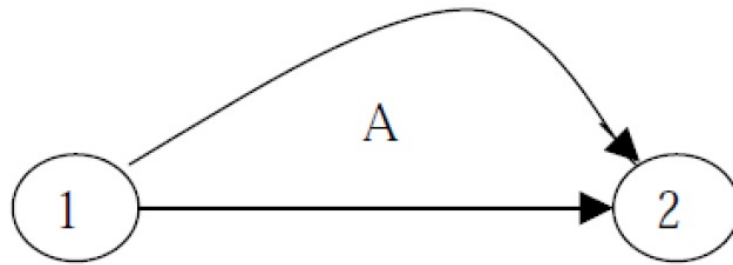
- ❖ An **imaginary activity which does not consume any resource and time is called a dummy activity**. Dummy activities are simply used to **represent a connection between events in order to maintain a logic in the network**. It is represented by a dotted line in a network, as shown be



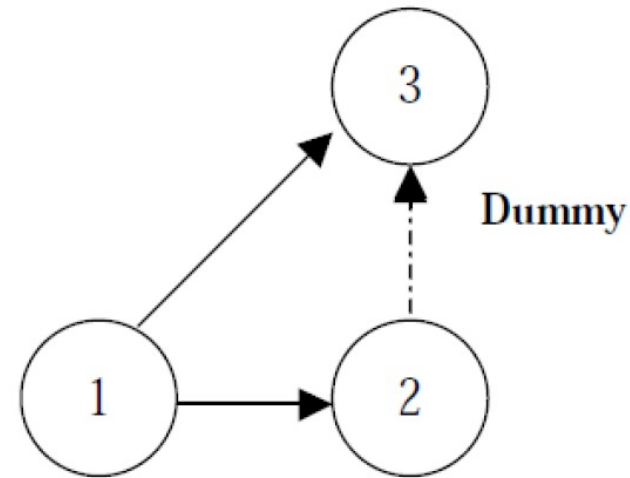
PERT / CPM Network Components

ERRORS TO BE AVOIDED IN CONSTRUCTING A NETWORK

1. **Two activities starting from a tail event must not have a same end event.** To ensure this, it is **absolutely necessary to introduce a dummy activity**, as shown:



Incorrect

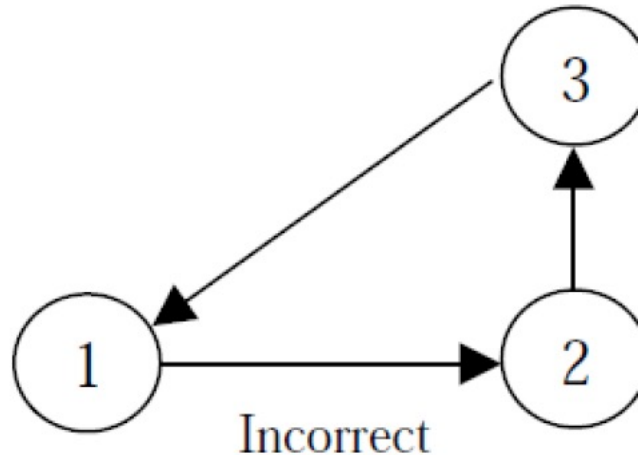


Correct

PERT / CPM Network Components

ERRORS TO BE AVOIDED IN CONSTRUCTING A NETWORK

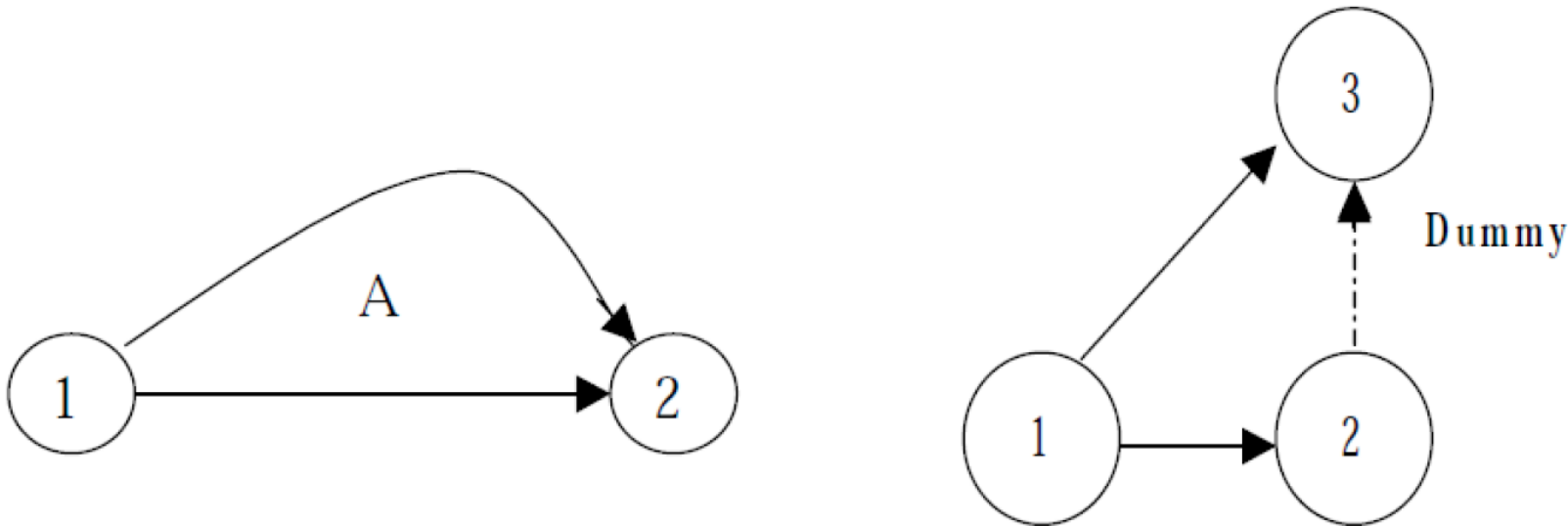
2. **Looping error should not be formed in a network**, as it represents performance of activities repeatedly in a cyclic manner, as shown below:



PERT / CPM Network Components

ERRORS TO BE AVOIDED IN CONSTRUCTING A NETWORK

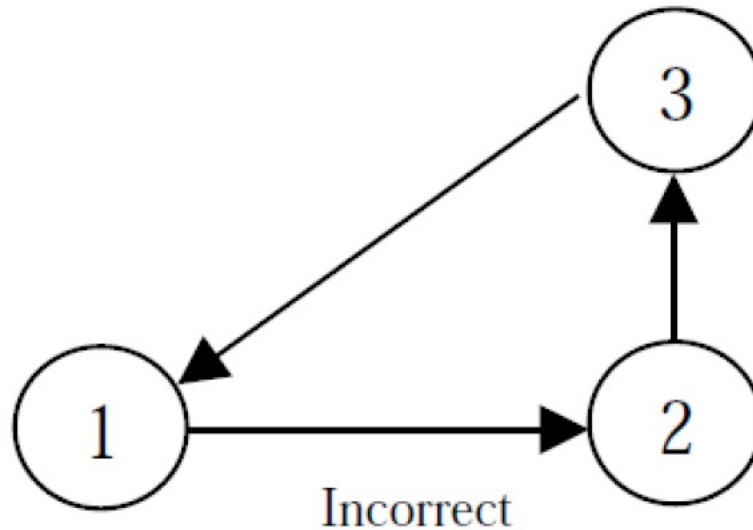
3. In a network, there should be only one start event and one ending event as shown below:



PERT / CPM Network Components

ERRORS TO BE AVOIDED IN CONSTRUCTING A NETWORK

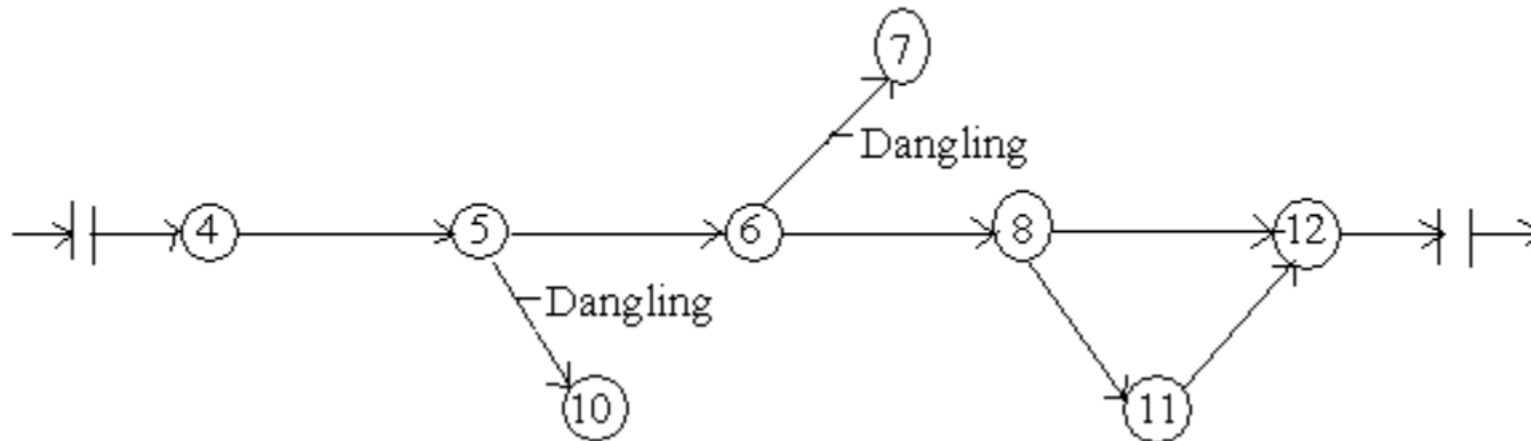
4. The direction of arrows should flow from left to right avoiding mixing of direction as shown:



PERT / CPM Network Components

ERRORS TO BE AVOIDED IN CONSTRUCTING A NETWORK

4. To disconnect an activity before the completion of all activities in a network diagram is known as dangling.



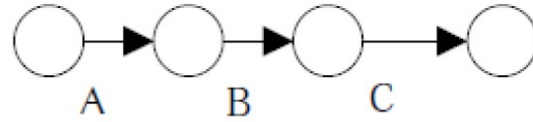
PERT / CPM Network Components

Rules for Constructing a Network Diagram

1. **No single activity can be represented more than once** in a network. The length of an arrow has no significance.
2. The **event numbered 1 is the start event** and an **event with highest number is the end event**. Before an activity can be undertaken, all activities preceding it must be completed. That is, the **activities must follow a logical sequence** (or – interrelationship) between activities.
3. In assigning numbers to events, **there should not be any duplication of event numbers** in a network.
4. **Dummy activities must be used only if it is necessary** to reduce the complexity of a network.
5. **A network should have only one start event and one end event.**

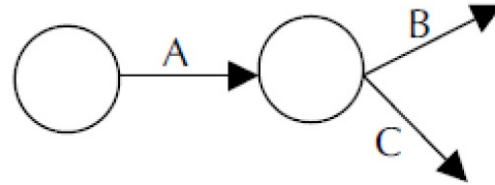
PERT / CPM Network Components

(a)



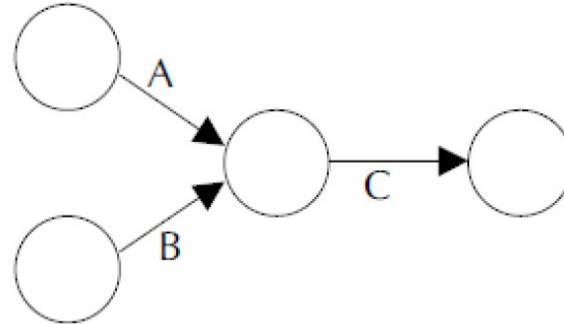
Activity B can be performed only after completing activity A, and activity C can be performed only after completing activity B.

(b)



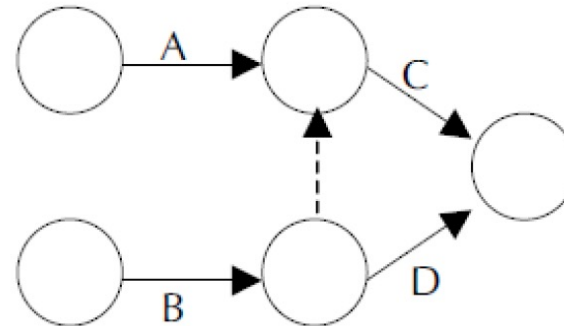
Activities B and C can start simultaneously only after completing A.

(c)



Activities A and B must be completed before start of activity C.

(d)



Activity C must start only after completing activities A and B. But activity D can start after completion of activity B.

**Rules for Constructing
a Network Diagram**

PERT / CPM Network Components

PROCEDURE FOR NUMBERING EVENTS USING FULKERSON'S RULE

Step1: Number the start or initial event as 1.

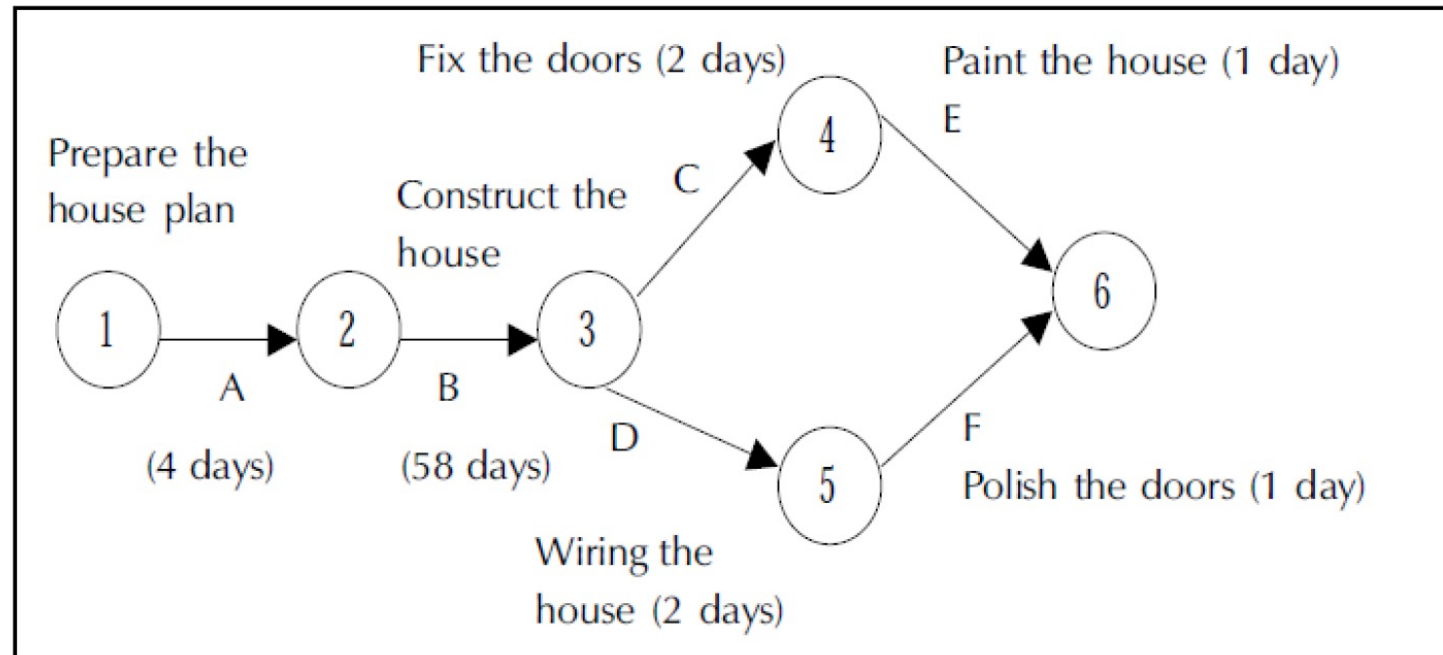
Step2: From event 1, strike off all outgoing activities. This would have made one or more events as initial events (event which do not have incoming activities). Number that event as 2.

Step3: Repeat step 2 for event 2, event 3 and till the end event. The end event must have the highest number.

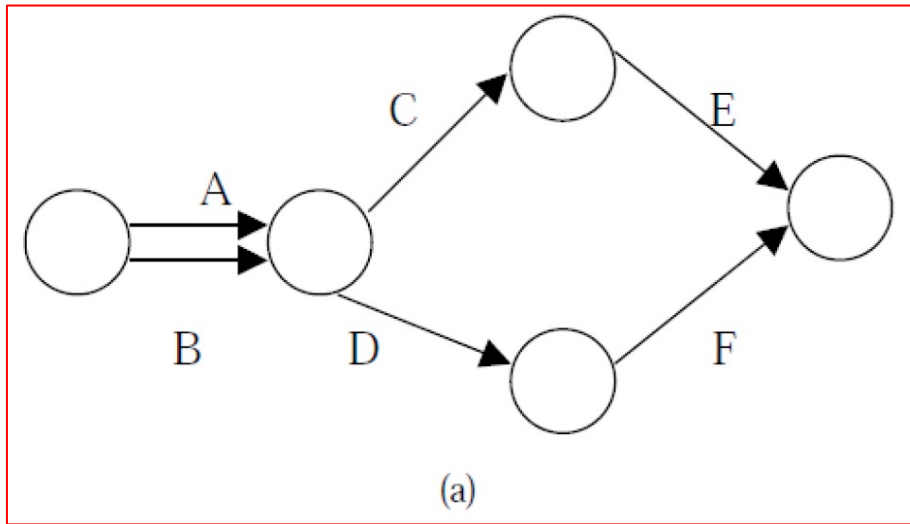
Example 1: Draw a network for a house construction project. The sequence of activities with their predecessors are given in following table:

Name of the activity	Starting and finishing event	Description of activity	Predecessor	Time duration (days)
A	(1,2)	Prepare the house plan	--	4
B	(2,3)	Construct the house	A	58
C	(3,4)	Fix the door / windows	B	2
D	(3,5)	Wiring the house	B	2
E	(4,6)	Paint the house	C	1
F	(5,6)	Polish the doors / windows	D	1

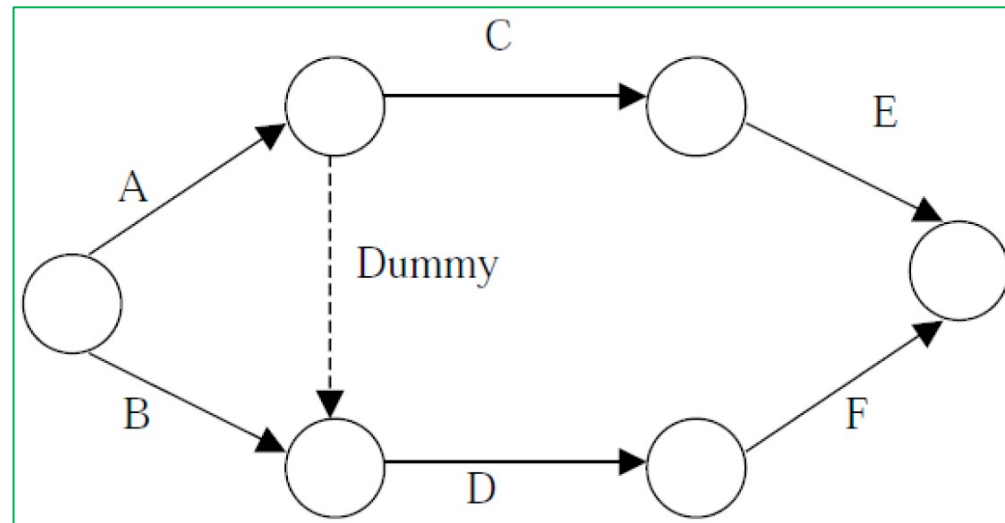
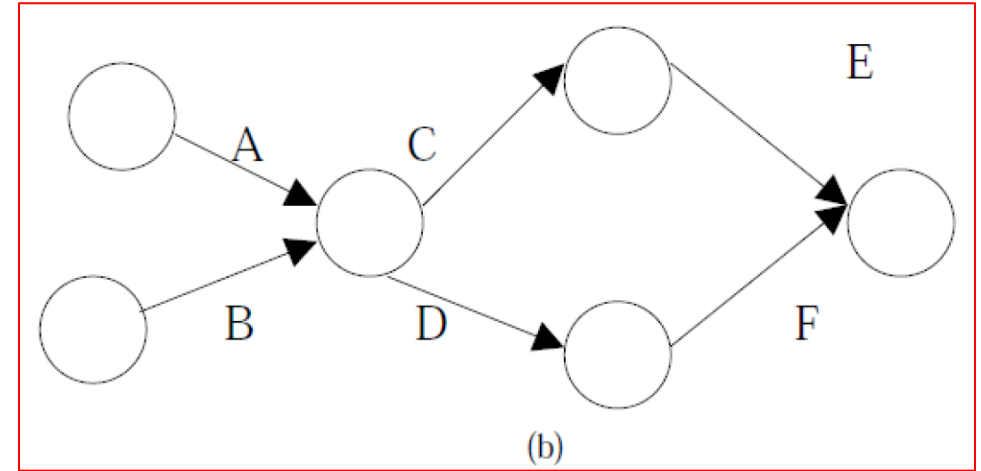
Solution:



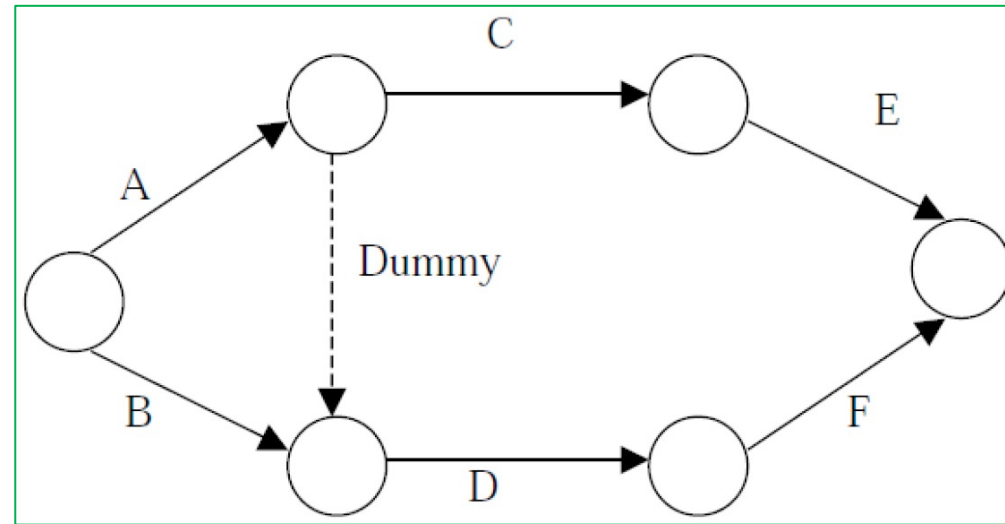
Example 2: Understand the need for Dummy activity and draw the table from the given Network Diagram



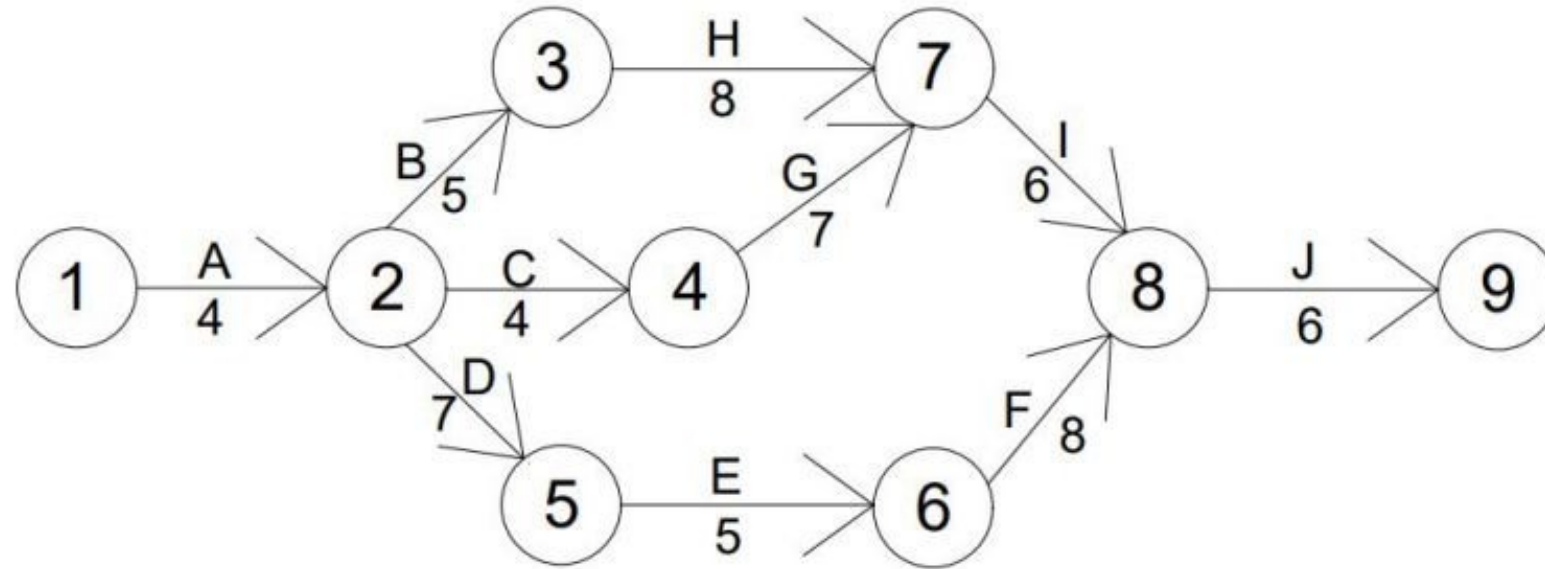
(a) and (b) are wrong.



Example 2: Understand the need for Dummy activity and draw the table from the given Network Diagram

[illegible]

Critical Path Method (CPM)



- ❖ This critical path method was developed in 1957 and
- ❖ Initially, it was suitable for both civil and mechanical engineering projects.
- ❖ This technique is useful **to determine** how best **to reduce the time required to perform production, maintenance and construction and also to minimize the direct and indirect cost** of the project.

Critical Path Method (CPM)

Using this critical path method C.P.M, a **planning engineer** will come to know the **sequence of the various activities** of the project.

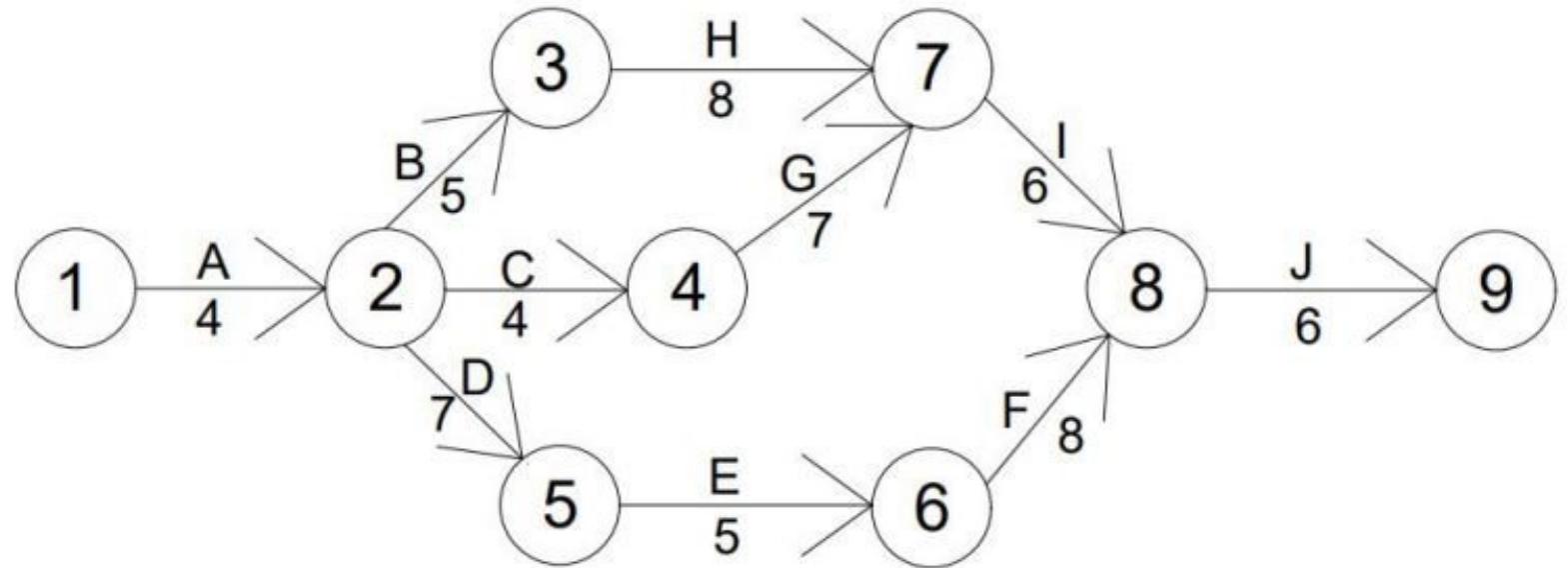
❖ Critical path network can be prepared using the below steps :

1. Preparation of network
2. Estimation of the expected time to perform each activity.
3. Determination of critical path schedule.
4. Interpretation of the results.

Critical Path Method Terminologies

CPM terms:

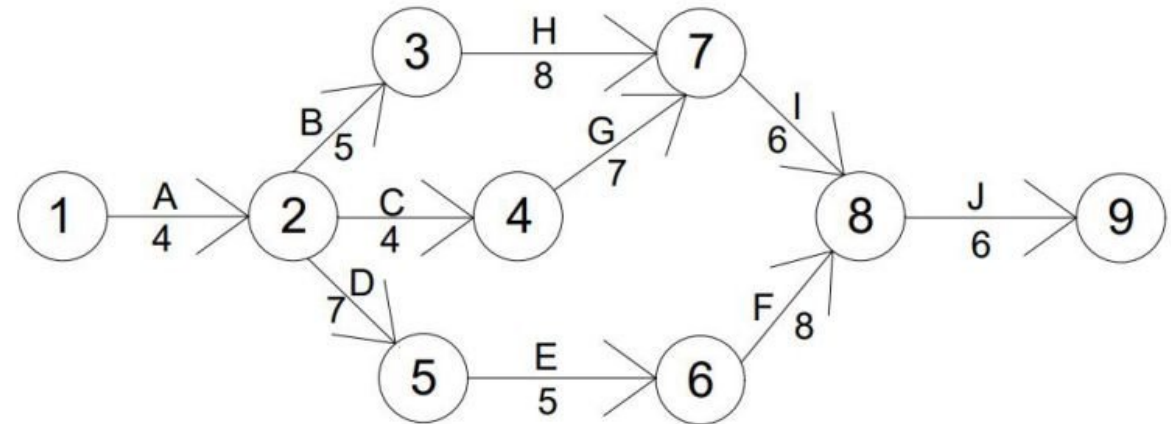
1. Activity
2. Dummy activity
3. Event
4. Network
5. Early start time (E.S.T)
6. Early finish time (E.F.T)
7. Late start time
8. Late finish time
9. Total float
10. Free float
11. Critical activities
12. Critical path



1. Activity

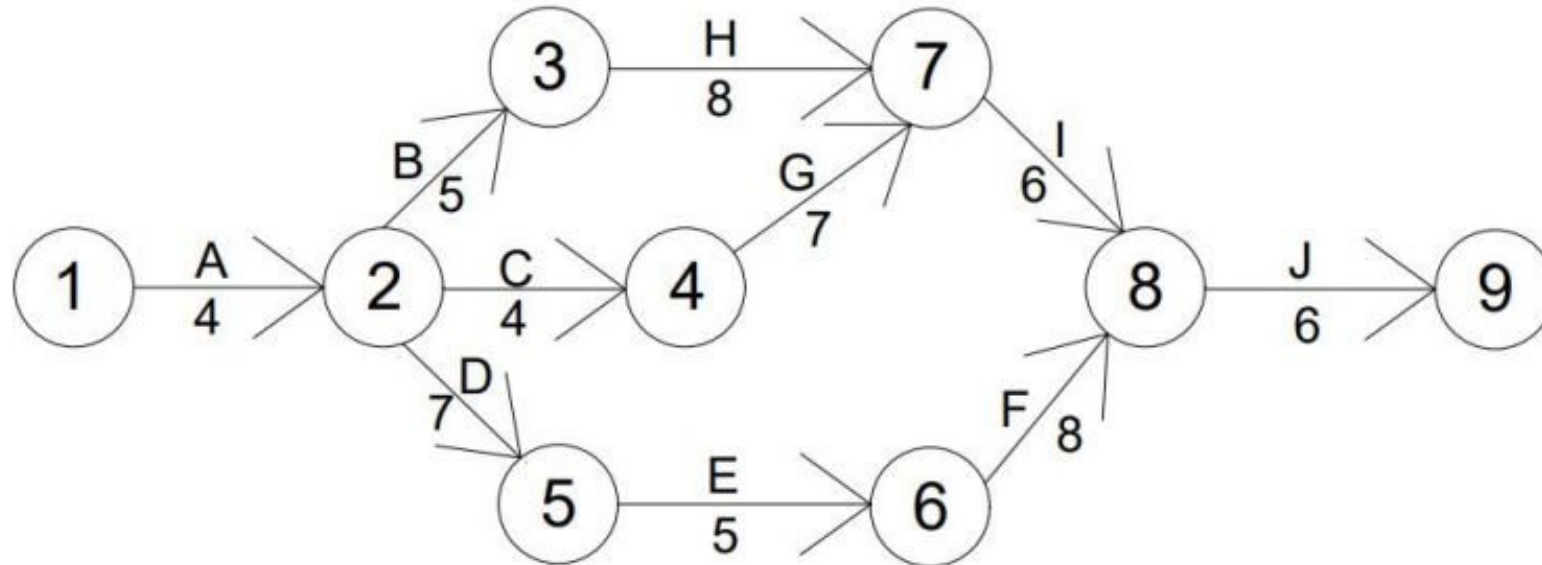
- ❖ **Activity** is part of the project.
- ❖ The network of the project is denoted by an **arrow**.
- ❖ The **start of the activity** is indicated by the **tail of the arrow** and the **end of the activity** is indicated by the **head of the arrow**.
- ❖ For a **given duration** of an **activity only one arrow** is utilised.
- ❖ The **arrows on the activity are not represented to scale**.
- ❖ Along with the arrows, the duration of the activity is shown.
- ❖ For example:

Between **2** and **4** arrows activity **C** is shown ,and its **4** units of time duration is shown beneath the arrow.



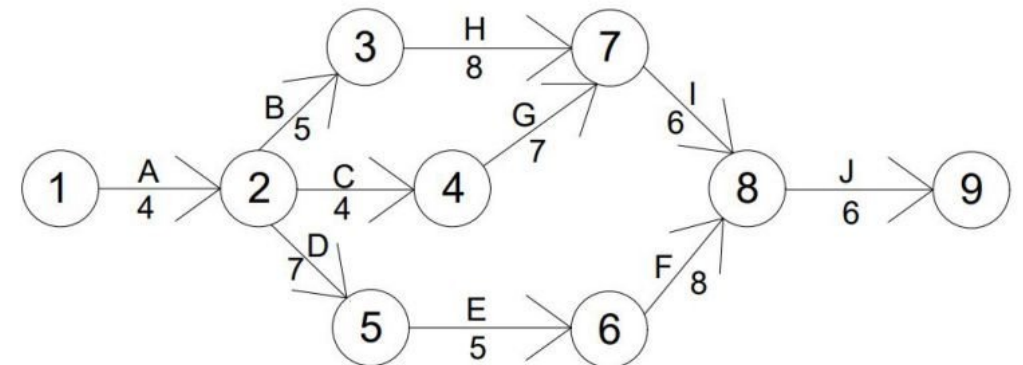
2.Dummy activity

- ❖ The dummy activity is the activity which does not utilise any time or any resources for the completion of activity .



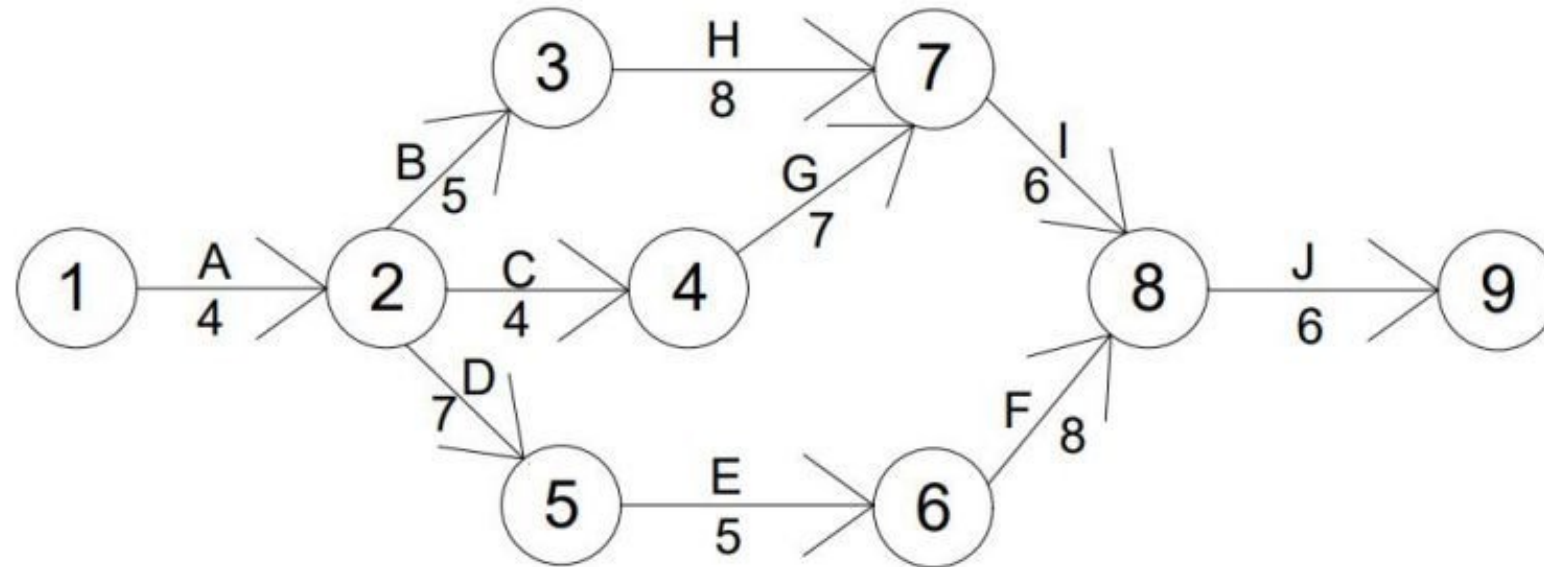
3.Event

- ❖ An **event** is the **stage or point** at which **all previous completed activities merge** and **jobs coming out of this point** needs to be **finished**.
- ❖ At the junctions of arrows, they are represented by **circles** or **nodes**.
- ❖ In sequential order, they are **serially numbered**.



4. Network

- ❖ The representation of the **entire project activities** in **diagrammatically** is termed as **network**.
- ❖ On this diagram, The various projects jobs which are expected to be completed are shown in the order.



Other CPM Terminologies from 5 to 12

5. Early start time

- An early start is the **earliest possible time** at which an activity may start.

6. Early finish time

- Early finish time is the **sum** of the **earliest start time of activity** and the **time required for its completion**.

7. Late start time

- **Without delaying** the project date, the **most possible time at which an activity may start** is termed as late start time.

8. Late finish time

- The **sum** of the **time required for completion** and the **late start time of activity** is termed as late finish time.

Other CPM Terminologies from 5 to 12

9. **Total float**

- The difference between the estimated duration and maximum allowed time for an activity is termed as total float.
- Without disturbing the project schedule it is the span of time by which the activity can start.
- It is expressed by letter S.

10. **Free float**

- The span of time without disturbing the start of succeeding activities by which the activity completion can be delayed is termed as Free float.
- It is expressed by S.F.

11. **Critical activities**

- The events which have no float are termed as critical activities.
- All the critical activities need to be finished on schedule.

12. **Critical path**

- The joining of the critical events in the network path is termed as critical path .

Basic rules for critical path method network

1. The **flow direction** is indicated by **arrows**, which is normally accepted from **left to right**.
2. The **length duration of the activity** is **not correspond to length of the arrow**. It is just geometrical configuration and does not have any importance.
3. The **activities merging** at it **must be completed before starting any new activity**.
4. The **indication of the logical conditions of dependency** in a network is **done** by an **arrow**.
5. The **construction of network** is done **logically** on the **principle of dependency**.
6. For **starting the project** and also for **closing the project**, only one event must be there in every network.
7. These **connection of any two events** may be **done directly** but **not with more than one activity**.
8. **Serial numbers** should be **given to events** and those **numbers should not done duplication** in a network.

Drawing a Network in critical path method

1. **List all the activities** required for completion of a project.
2. **Ascertain the dependence** of the activities on each other.
3. **Draw a network of the activities** roughly with a soft pencil so that it can be easily erased.
4. The **duration of each activity** can be **estimated** by keeping in mind about the availability of **equipment, manpower** and also the **likelihood of activities delays**.
5. The **estimate time** of each activity **needs to be noted**.
6. For **deciding the critical path**, the latest and earliest allowable start and finish times for every activity to determined.
7. In the **field** need to **finalise the network** for use.

Advantages of critical path method

1. It is useful to **ascertain the time schedule**
2. It helps to have **proper control of the project.**
3. It enables **better and detailed planning.**
4. It is very helpful **to communicate** the cost, project schedule, project planning ,performance ,time and project planning.
5. The **critical activities** which are **highlighted along the critical path should be attended with care until their completion.**

Despite the great planning cost of a project, **Critical path proves to be very useful for focusing the attention of project engineers** on the critical path activities and thus **avoids unnecessary expenditure.**

Expected length of the project , the critical path

1. Critical path :

- The **longest duration path** of a network.
- The **sum of the expected times of the total activities is maximum** in this path.

2. Length of the project

- The sum of the expected times of all the activities along the critical path of the network of a project is called the length of the project.

3. The variance of the project

- The variance of the project is the **total sum of the variance along the critical path the activities** of the network of a project.

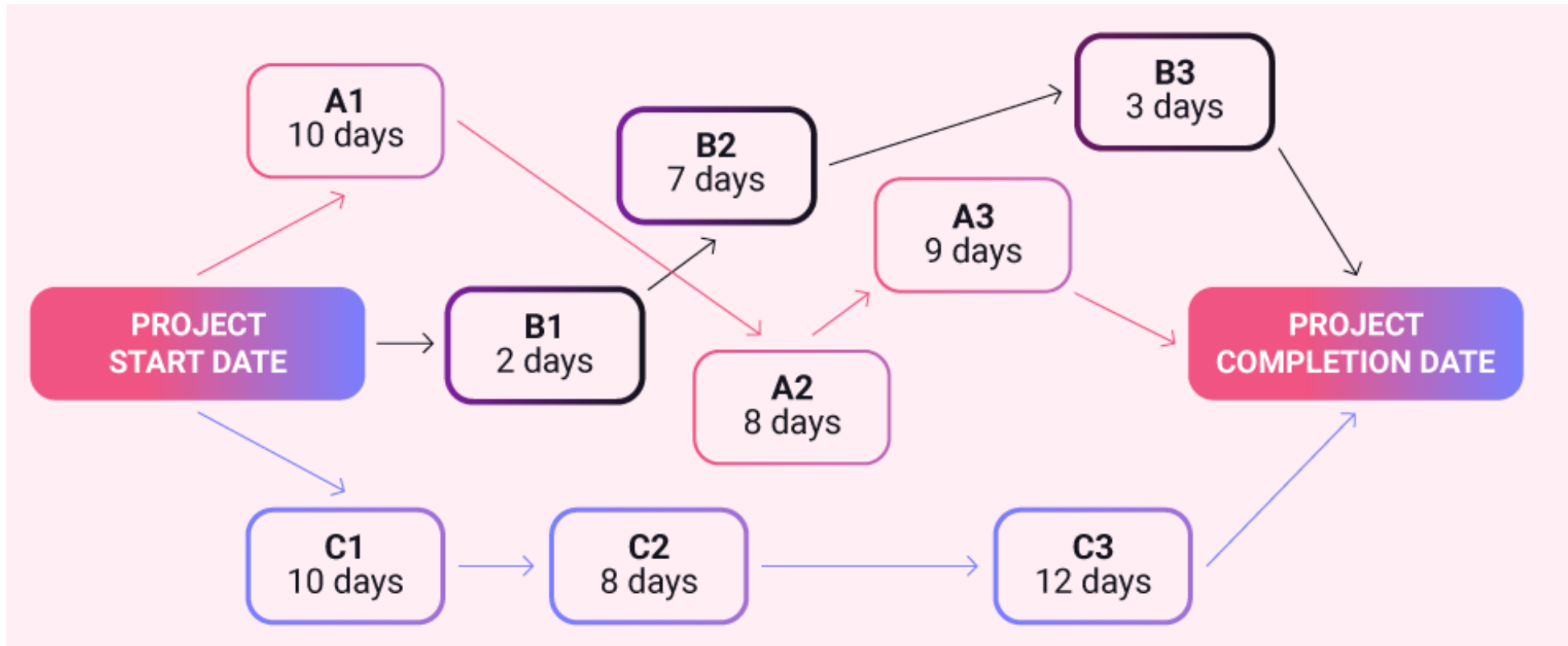
4. The standard deviation of the project

- The standard deviation is the **square root of the variance** which is calculated along the network critical path of a project.

Critical Path Method Terminologies

1: Critical path

The critical path is the sequence of dependent activities with the longest overall duration.



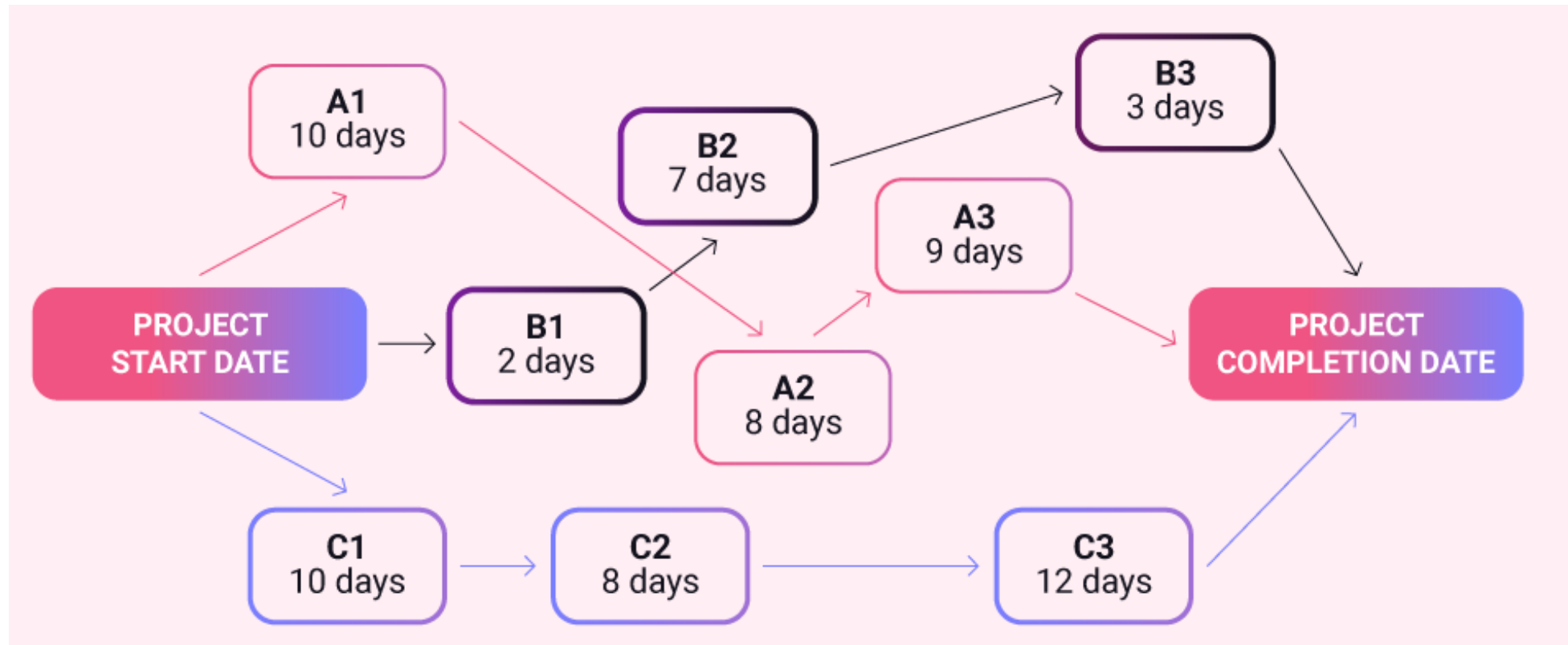
C1-C2-C3 string of activities constitutes the critical path, as they take 30 days to complete.

Critical Path Method Terminologies

2: Activity nodes

Activity nodes are rectangles used in CPM diagrams designed to hold all the crucial information needed for charting a project's critical path.

Each task required for project completion gets its own activity node. The illustration below features 9 activity nodes: **A1, A2, A3, B1, B2, B3, C1, C2, and C3**.



Critical Path Method Terminologies

3: Early start time (ES)

- Early start time denotes the **earliest possible point at which you can start working on a task.**
- The ES for the first task(s) in a project is set to **0**.
- To calculate the ES for all other tasks, just **copy the EF (early finish time) of the preceding dependent task.**
 - If a task is dependent on 2 or more previous activities, **refer to the activity with the longer ES.**

Critical Path Method Terminologies

4: Duration (Dur)

Duration is self-explanatory – it denotes how long you expect a task will take to complete.

5: Early finish time (EF)

- **Early finish time denotes the earliest moment at which it is possible to finish a task.**
- To calculate the early finish time for a task, you take its ES and add the task duration (Dur).

$$EF = ES + Dur$$

- Since the first activity in a project generally has an ES of 0, the EF will be identical to the duration of the first activity.
- All dependent tasks that follow immediately after it will take the first task's EF as their ES, and add their duration to it to calculate their EF.

Critical Path in Network Analysis

Basic Scheduling Computations

The notations used are

❖ (i, j) = Activity with **tail event** i and **head event** j

❖ E_i = **Earliest occurrence time** of event i

❖ L_j = **Latest allowable occurrence time** of event j

❖ D_{ij} = **Estimated completion time** of activity (i, j)

❖ $(Es)_{ij}$ = **Earliest starting time** of activity (i, j)

❖ $(Ef)_{ij}$ = **Earliest finishing time** of activity (i, j)

❖ $(Ls)_{ij}$ = **Latest starting time** of activity (i, j)

❖ $(Lf)_{ij}$ = **Latest finishing time** of activity (i, j)

Critical Path in Network Analysis

1. Determination of Earliest time (E_j): Forward Pass computation

- **Step 1**

The computation begins from the start node and move towards the end node. For easiness, the forward pass computation starts by assuming the earliest occurrence time of zero for the initial project event.

- **Step 2**

- i. Earliest starting time of activity (i, j) is the earliest event time of the tail end event i.e. $(Es)_{ij} = E_i$
- ii. Earliest finish time of activity (i, j) is the earliest starting time + the activity time i.e. $(Ef)_{ij} = (Es)_{ij} + D_{ij}$ or $(Ef)_{ij} = E_i + D_{ij}$
- iii. Earliest event time for event j is the maximum of the earliest finish times of all activities ending in to that event i.e. $E_j = \max [(Ef)_{ij}]$ for all immediate predecessor of (i, j) or $E_j = \max [E_i + D_{ij}]$

Critical Path in Network Analysis

2. Backward Pass computation (for latest allowable time)

- **Step 1**

For ending event assume $E = L$. Remember that all E 's have been computed by forward pass computations.

- **Step 2**

Latest finish time for activity (i, j) is equal to the latest event time of event j i.e.

$$(Lf)_{ij} = L_j$$

- **Step 3**

Latest starting time of activity $(i, j) =$ the latest completion time of $(i, j) -$ the activity time or $(Ls)_{ij} = (Lf)_{ij} - D_{ij}$ or $(Ls)_{ij} = L_j - D_{ij}$

- **Step 4**

Latest event time for event 'i' is the minimum of the latest start time of all activities originating from that event i.e. $L_i = \min [(Ls)_{ij}]$ for all immediate successor of (i, j) $= \min [(Lf)_{ij} - D_{ij}] = \min [L_j - D_{ij}]$

Critical Path in Network Analysis

3. Determination of floats and slack times

There are three kinds of floats

- **Total float** – The amount of time by which the completion of an activity could be delayed beyond the earliest expected completion time without affecting the overall project duration time.

Mathematically

$$(Tf)_{ij} = (\text{Latest start} - \text{Earliest start}) \text{ for activity } (i - j)$$

$$(Tf)_{ij} = (Ls)_{ij} - (Es)_{ij} \text{ or } (Tf)_{ij} = (L_j - D_{ij}) - E_i$$

- **Free float** – The time by which the completion of an activity can be delayed beyond the earliest finish time without affecting the earliest start of a subsequent activity.

Mathematically

$$(Ff)_{ij} = (\text{Earliest time for event } j - \text{Earliest time for event } i) - \text{Activity time for } (i, j)$$

$$(Ff)_{ij} = (E_j - E_i) - D_{ij}$$

Critical Path in Network Analysis

3. Determination of floats and slack times

There are three kinds of floats

- **Independent float** – The amount of time by which the start of an activity can be delayed without effecting the earliest start time of any immediately following activities, assuming that the preceding activity has finished at its latest finish time.

Mathematically

$$(If)_{ij} = (E_j - L_i) - D_{ij}$$

The negative independent float is always taken as zero.

- **Event slack** - It is defined as the difference between the latest event and earliest event times.

Mathematically

$$\text{Head event slack} = L_j - E_j, \text{ Tail event slack} = L_i - E_i$$

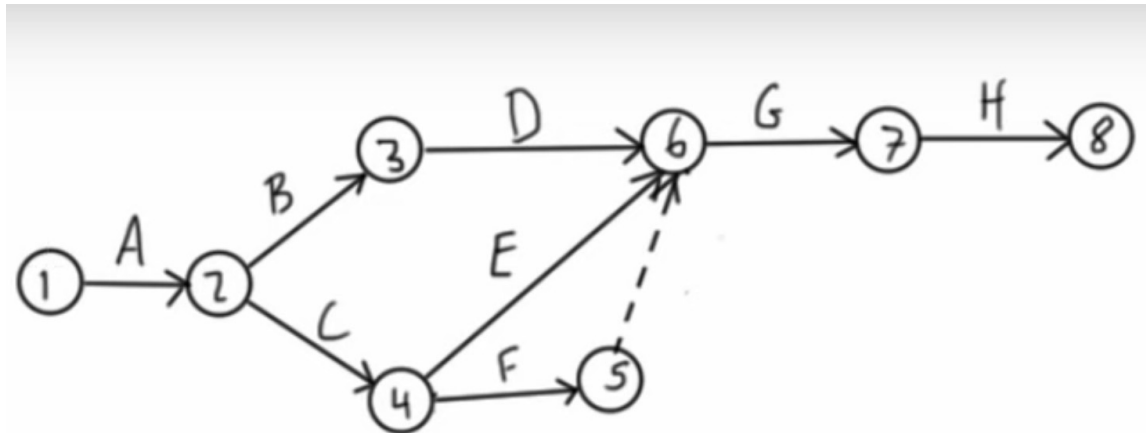
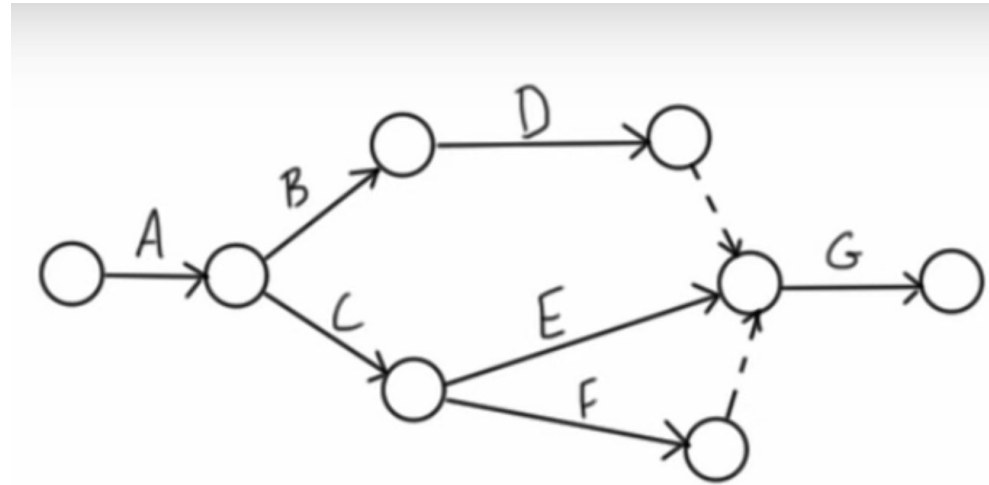
Critical Path in Network Analysis

4. Determination of Critical Path

- **Critical event** – The events with zero slack times are called critical events. In other words the event i is said to be critical if $E_i = L_i$
- **Critical activity** – The activities with zero total float are known as critical activities. In other words an activity is said to be critical if a delay in its start will cause a further delay in the completion date of the entire project.
- **Critical path** – The sequence of critical activities in a network is called critical path. The critical path is the longest path in the network from the starting event to ending event and defines the minimum time required to complete the project.

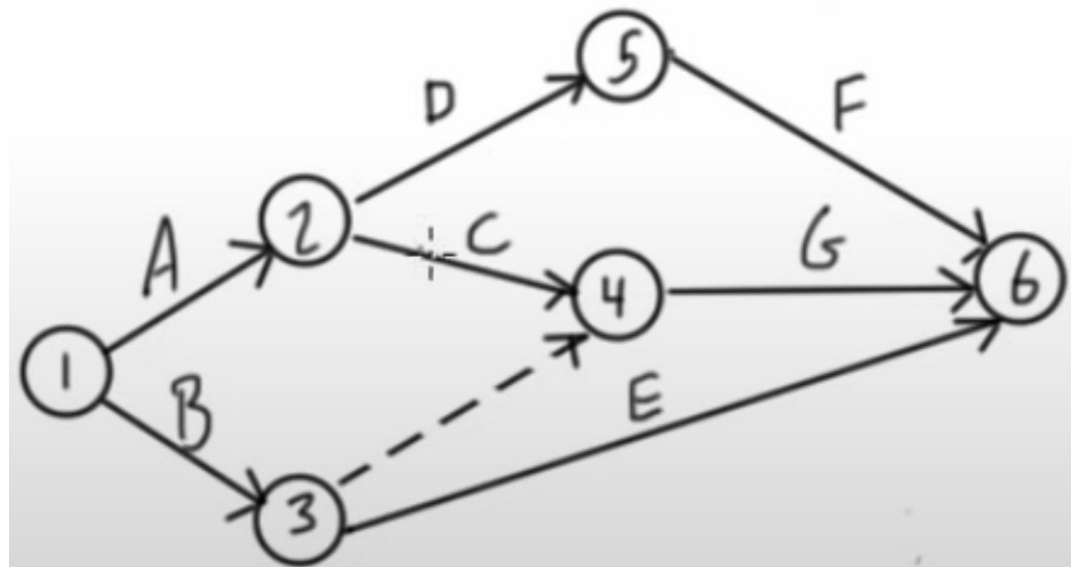
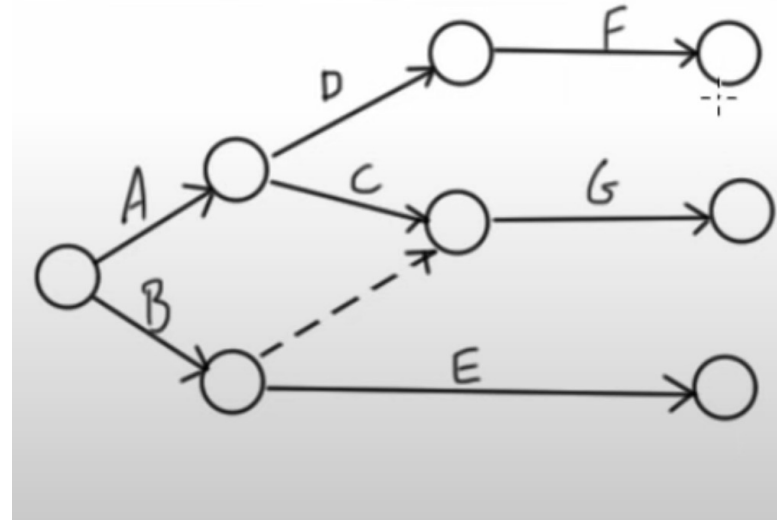
How to Draw Network Diagram

Activity	Predecessor
A	-
B	A
C	A
D	B
E	C
F	C
G	D,E,F
H	G



How to Draw Network Diagram

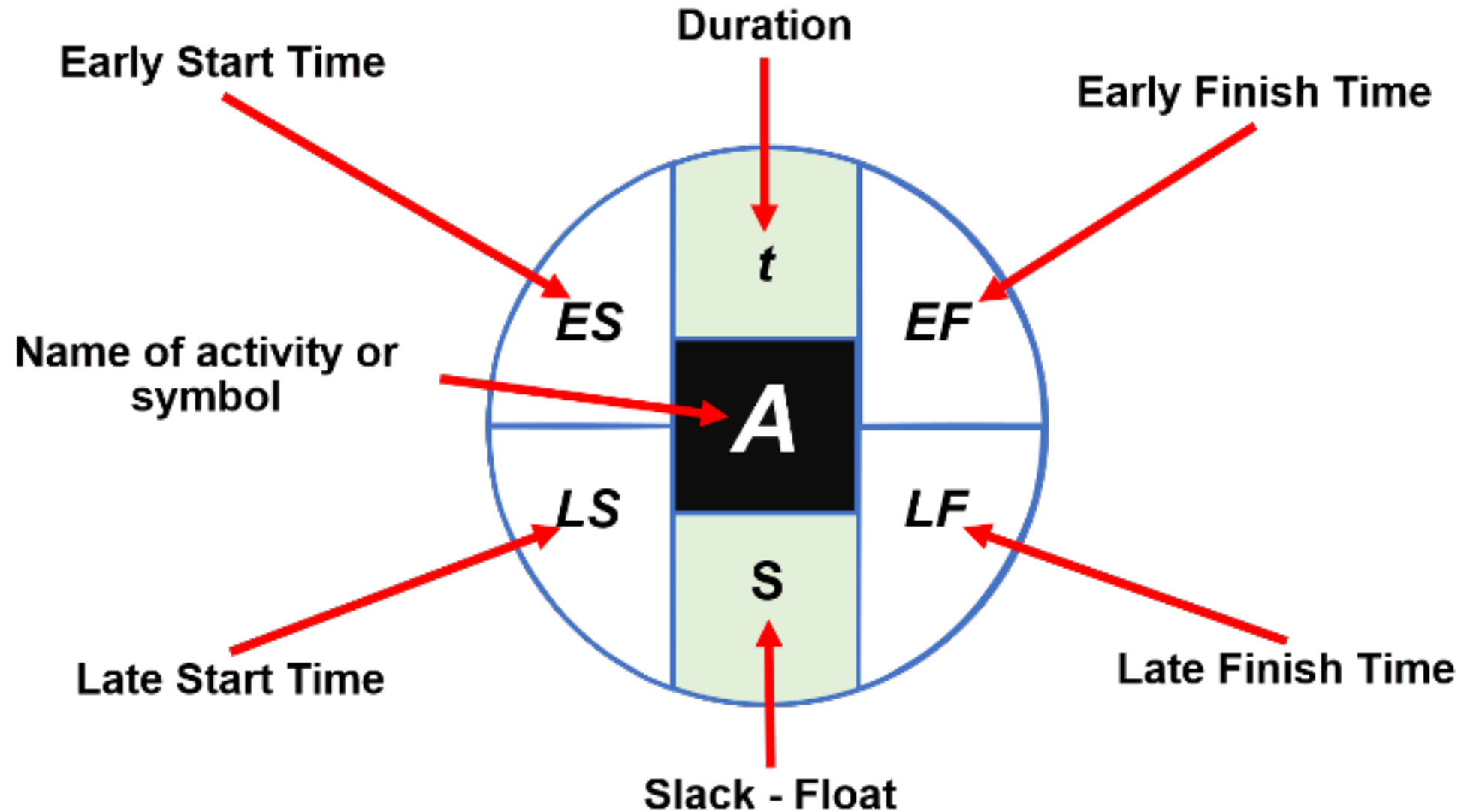
Activity	Predecessor
A	-
B	-
C	A
D	A
E	B
F	D
G	B,C

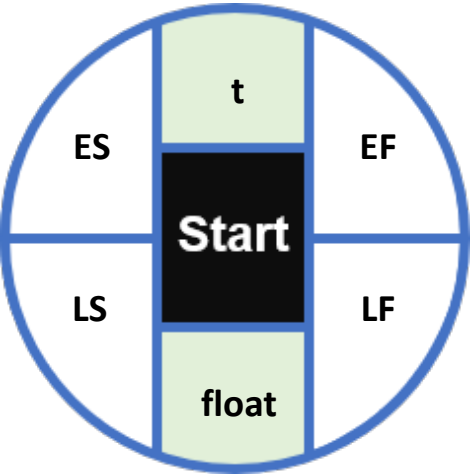


Critical Path in Network Analysis

- ❖ **Step 1:** Obtain the project data. Make a list of all the activities of the project along with their dependencies and their specific times.
- ❖ **Step 2:** Elaborate the network diagram. We have written a post that explains how to elaborate the project network diagram step by step.
- ❖ **Step 3:** Calculate the Early Start and Late Start Times. Determine the Early Start and Late Start Times for each activity.
- ❖ **Step 4:** Calculate the Early Finish and Late Finish Times. Determine the Early Finish and Late Finish Times for each activity.
- ❖ **Step 5:** Calculate the slack time. The critical path must be determined by finding out the slack for each activity of the process. The activities where there is no slack are the ones making up the critical path.

How to Calculate Early Start, Late Start, Early Finish, Late Finish and Slack (Float)





CPM

❖ Duration (t):

- ❖ Indicates the time it takes to complete the activity.

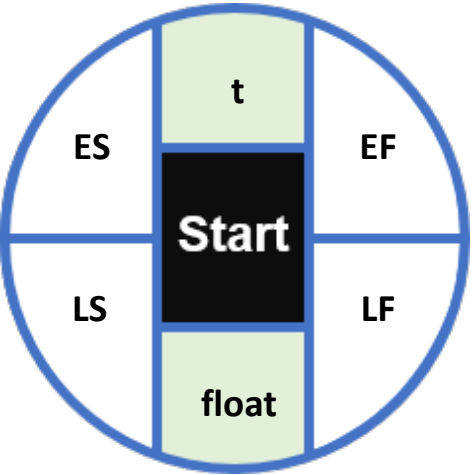
❖ Early finish Time (EF):

- ❖ This is the **earliest time that an activity can finish**. It is equal to the early start time plus its estimated duration (t):

$$EF = ES + t$$

❖ Early Start Time (ES):

- ❖ This is the **earliest time that an activity can be started** assuming all previous activities have been completed beforehand.
- ❖ For **activities that have more than one precedent**, the **ES is the greatest of the late start time of their precedents**.



CPM

❖ Late Finish Time (LF):

- ❖ This is the latest time at which an activity can be completed without delaying the entire project. It is obtained by equaling the late start time of the activity that immediately follows.
- ❖ If activities have more than one task immediately following them, the LF will be the least of the late start time of those activities.

❖ Late Start Time (LS):

- ❖ It is the **latest time an activity can begin without delaying** the whole project. It is **equal to the Late Finish Time minus the expected duration** of that activity (t):

$$LS = LF - t$$

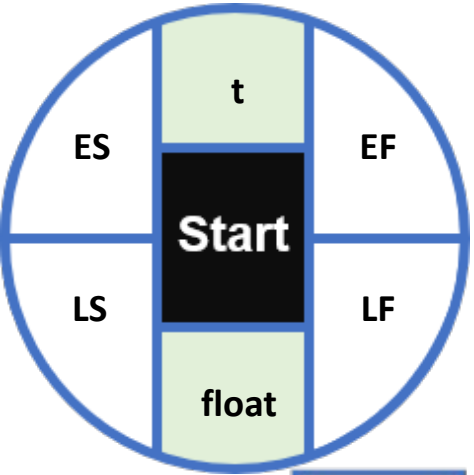
❖ Slack – Float (S):

- ❖ A period of time when an activity can be delayed without causing the entire project to be delayed. All activities contained in the critical path have zero slack.
- ❖ Here is how it is calculated mathematically:

$$S = LS - ES = LF - EF$$

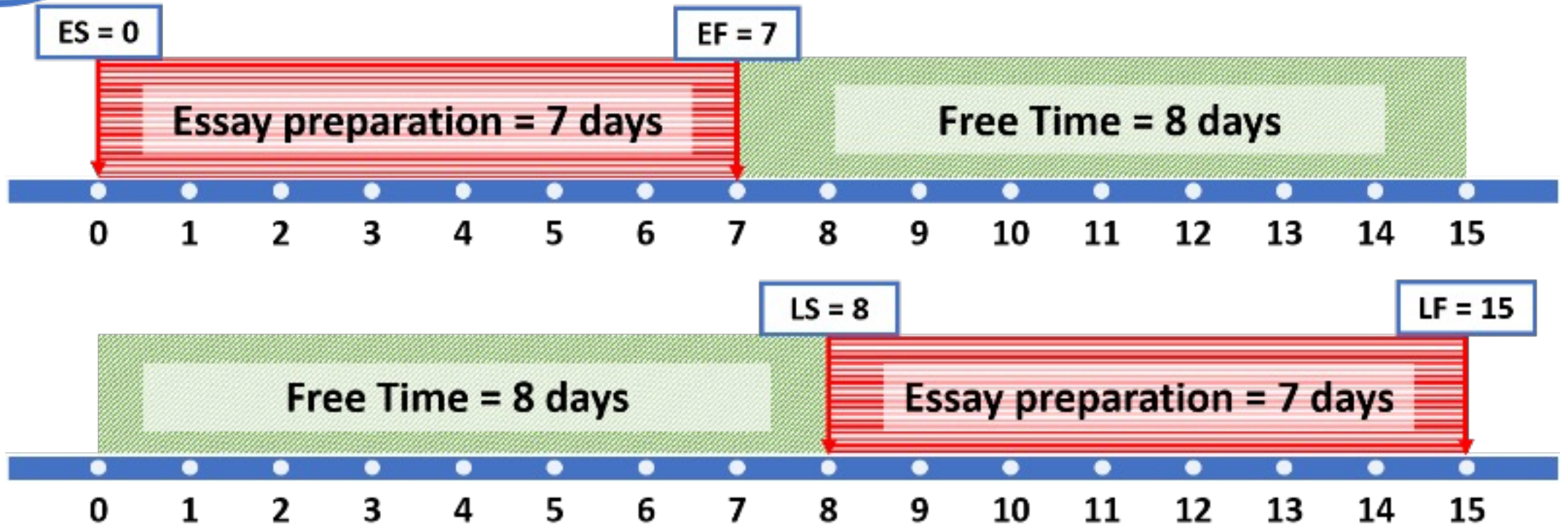
CPM

- **Early Start (ES):** It is equal to the Early Finish to the activity's precedent. If it has more than one precedent, the highest value is taken.
- **Early Finish (EF):** It is equal to the Early Start of the activity plus its duration (t).
$$EF = ES + t$$
- **Late Start (LS):** It is equal to the Late Finish minus its duration (t).
$$LS = LF - t$$
- **Late Finish (LF):** It is equal to the late start of the activity that follows. If it has more than one successor, the lowest value is taken.
- **Slack (S):** It can be calculated in two ways.
$$S = LS - ES = LF - EF$$
- Activities with zero clearance make up the critical path.

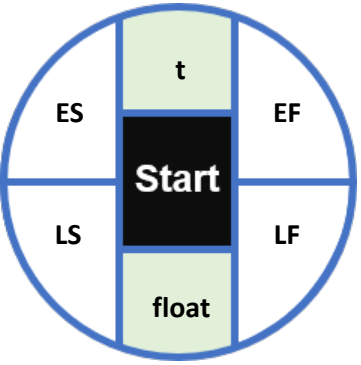


CPM

Imagine that your professor asks you to write an essay to be submitted in 15 days.

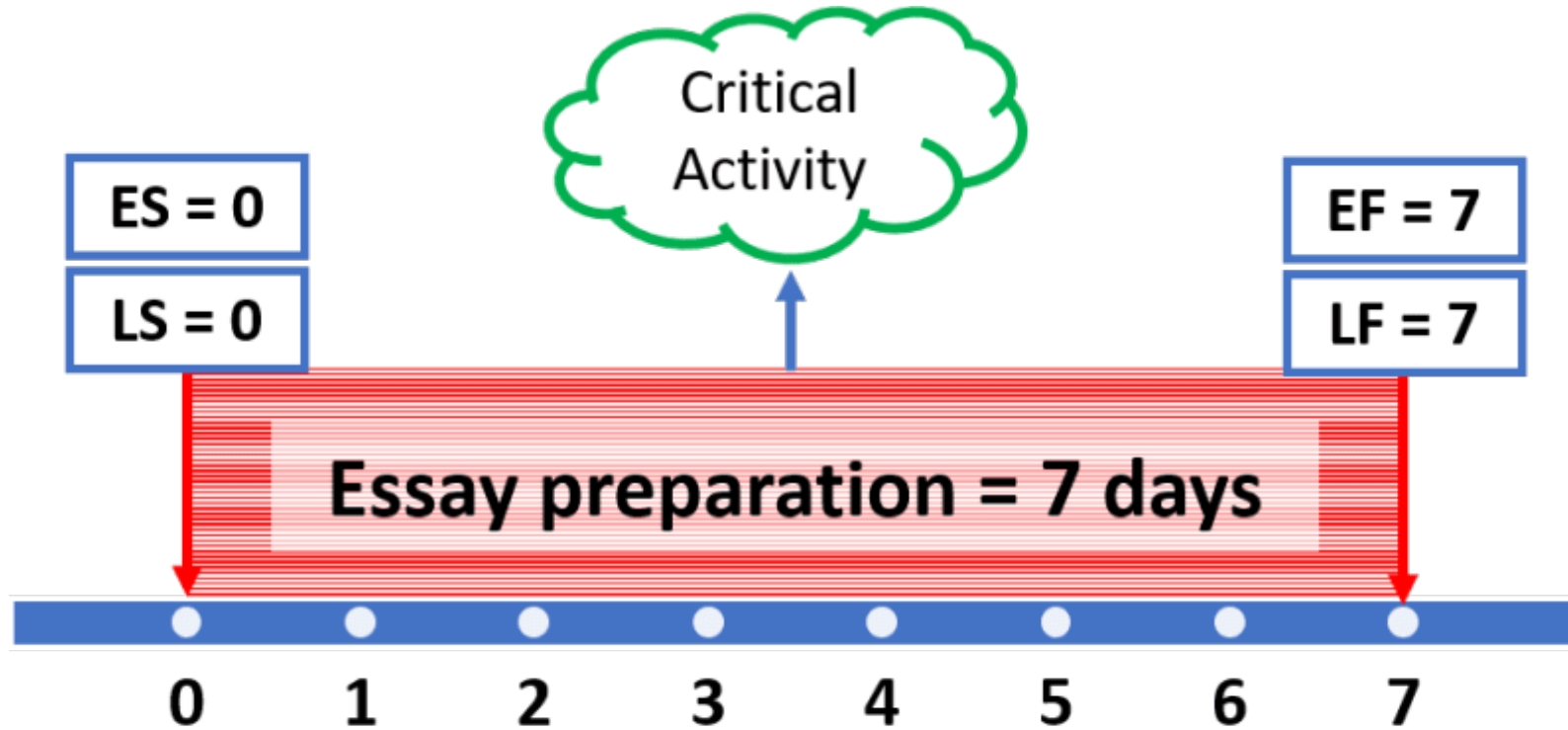


$$\text{Slack} = 8 - 0 = 15 - 7 = 8$$



CPM

If we change the scenario and our teacher gives us only the **one-week** deadline to submit the essay.



$$\text{Slack} = 0 - 0 = 7 - 7 = 0$$

CPM Example

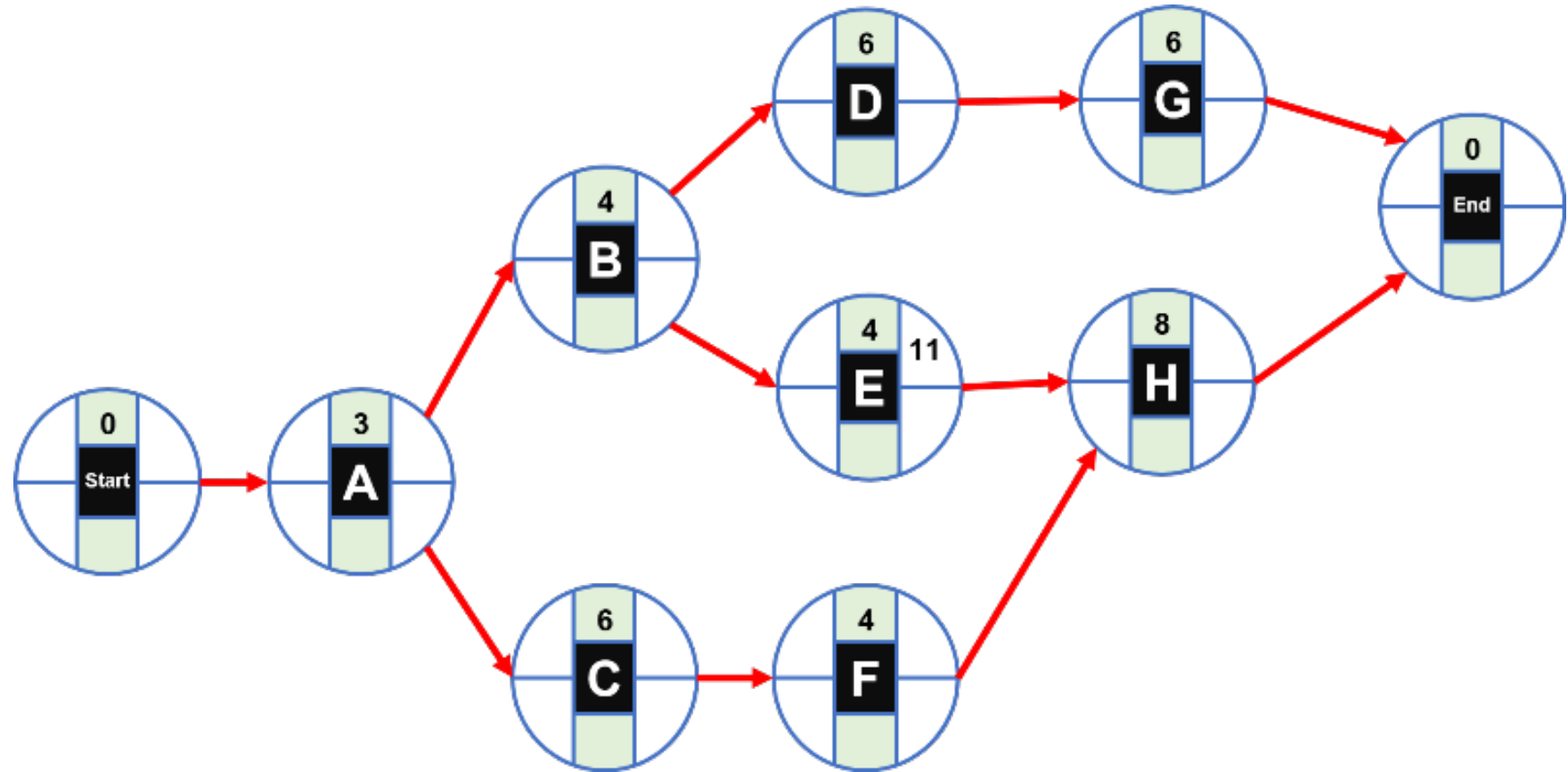
❖ Draw the activity-on-node (AON) project network associated with the following activities for Dave Carhart's consulting company project.

Activity	Immediate Predecessor(s)	Time (Days)
A	—	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8

1. How long should it take Dave and his team to complete this project?
2. What are the critical path activities?

CPM Example -Solution

Activity	Predecessor (s)	Time (Days)
A	—	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8

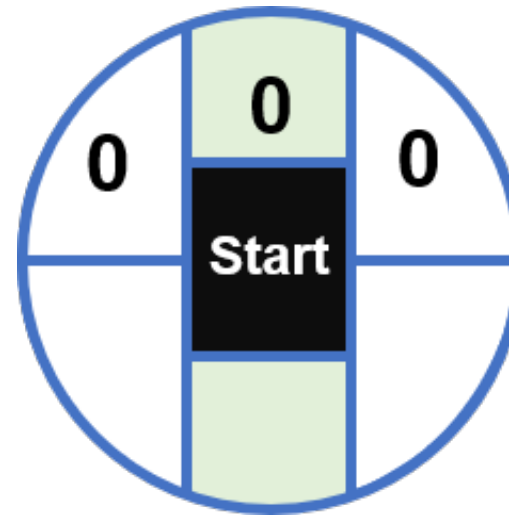


CPM Example –Solution (Forward Path)

Activity	Predecessor (s)	Time (Days)
A	–	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8

Start node:

This dummy node has all values equal to zero.



CPM Example –Solution (Forward Path)

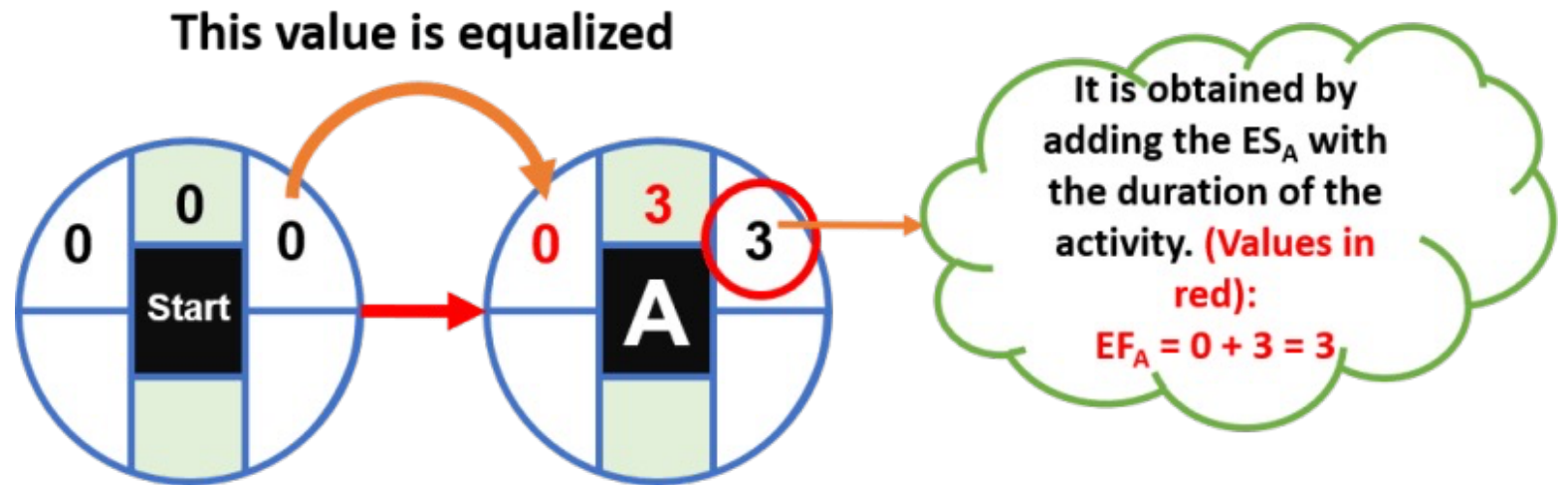
Activity	Predecessor (s)	Time (Days)
A	–	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8

Activity A:

Since it is the first activity, its **ES** will be **equal** to the **EF** of the **starting node** (**zero**); the EF is calculated as follows:

$$EF = ES + \text{activity time}$$

$$EF_A = 0 + 3 = 3$$

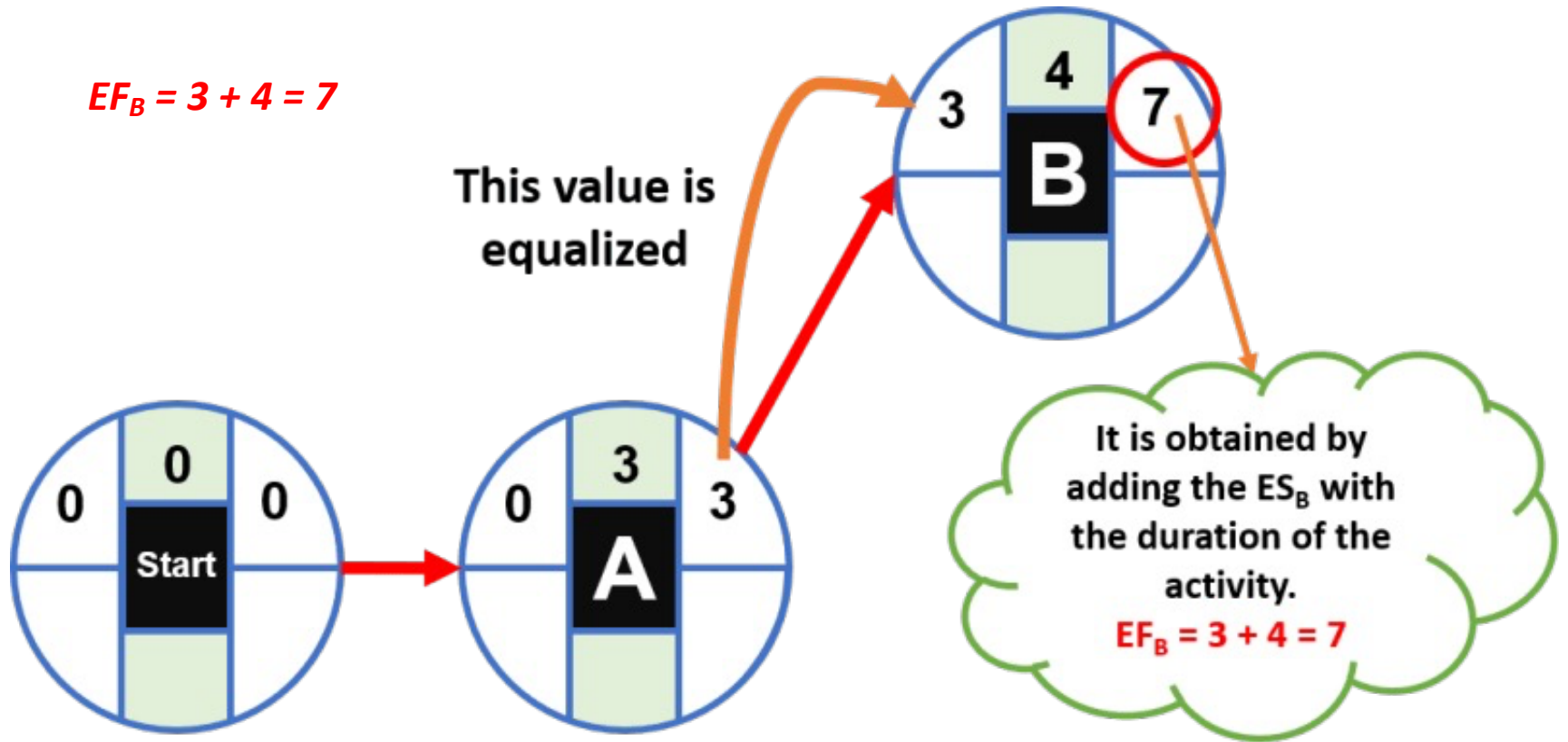


CPM Example –Solution (Forward Path)

Activity B:

It has as precedent only activity A; therefore its **ES** will be equal to the **EF** of activity A. In the same way as the previous node, the EF of activity B is calculated by adding its ES + the corresponding time:

$$EF_B = 3 + 4 = 7$$

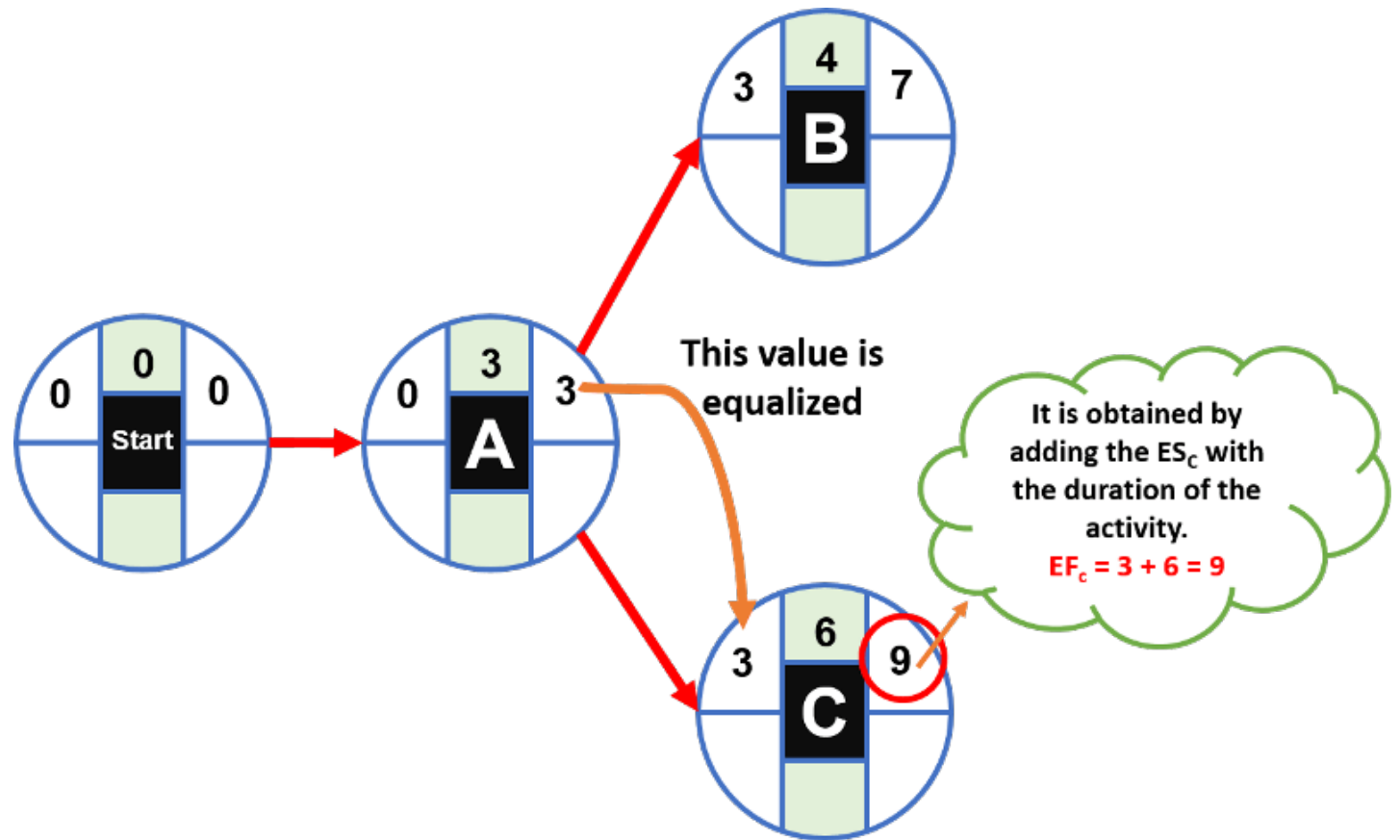


CPM Example –Solution (Forward Path)

Activity C:

It has as precedent only activity A; therefore its ES will be equal to the EF of activity A.
The EF of activity C is calculated by adding its ES + the corresponding time:

$$EF_C = 3 + 6 = 9$$

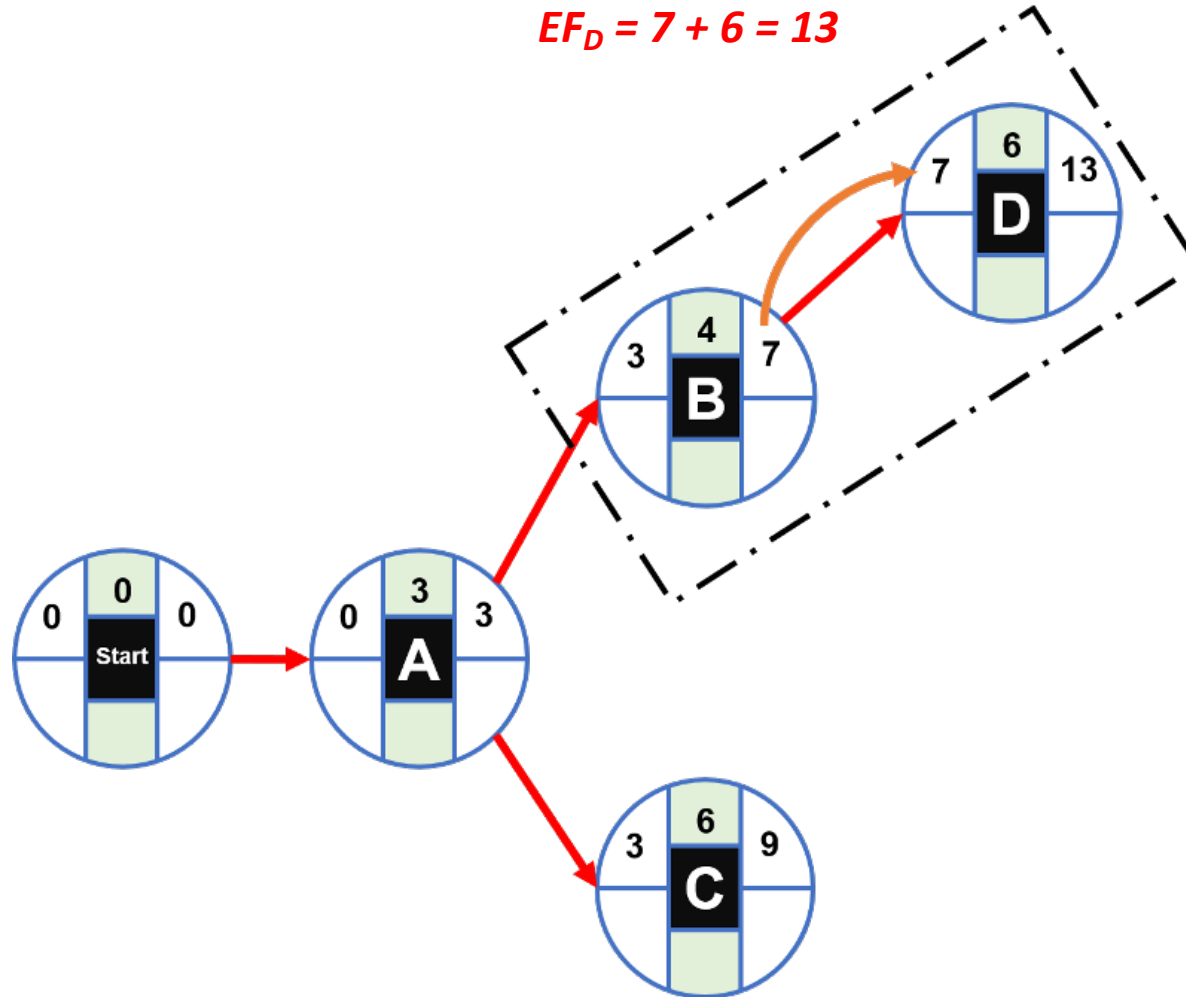


CPM Example –Solution (Forward Path)

Activity D:

It has as a precedent only activity B; therefore its ES will be equal to the EF of activity B. The EF of activity D is calculated by adding its ES + the corresponding time:

$$EF_D = 7 + 6 = 13$$



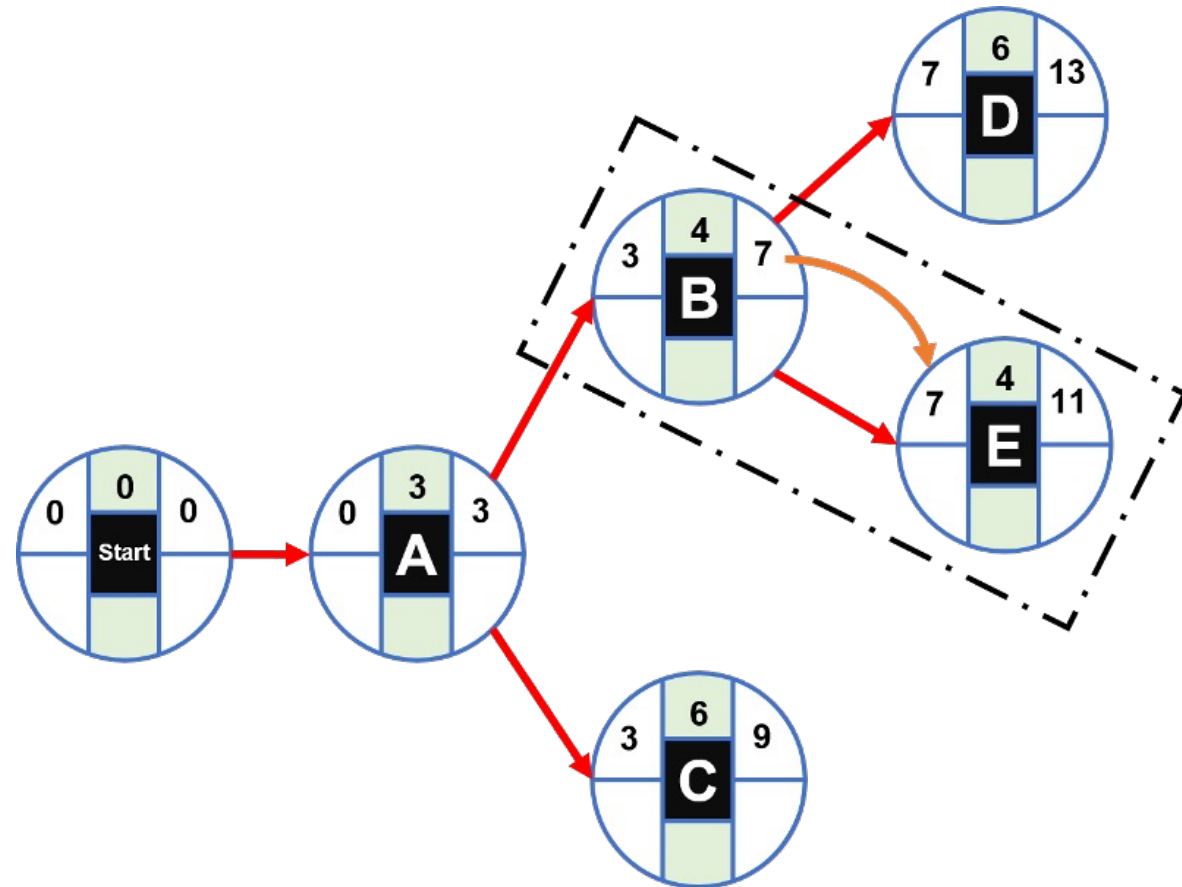
CPM Example –Solution (Forward Path)

Activity E:

It has as a precedent only activity B; therefore its ES will be equal to the EF of activity B. The EF of activity E is calculated by adding its ES + the corresponding time:

$$EF_E = 7 + 4 = 11$$

Activity	Predecessor (s)	Time (Days)
A	—	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8



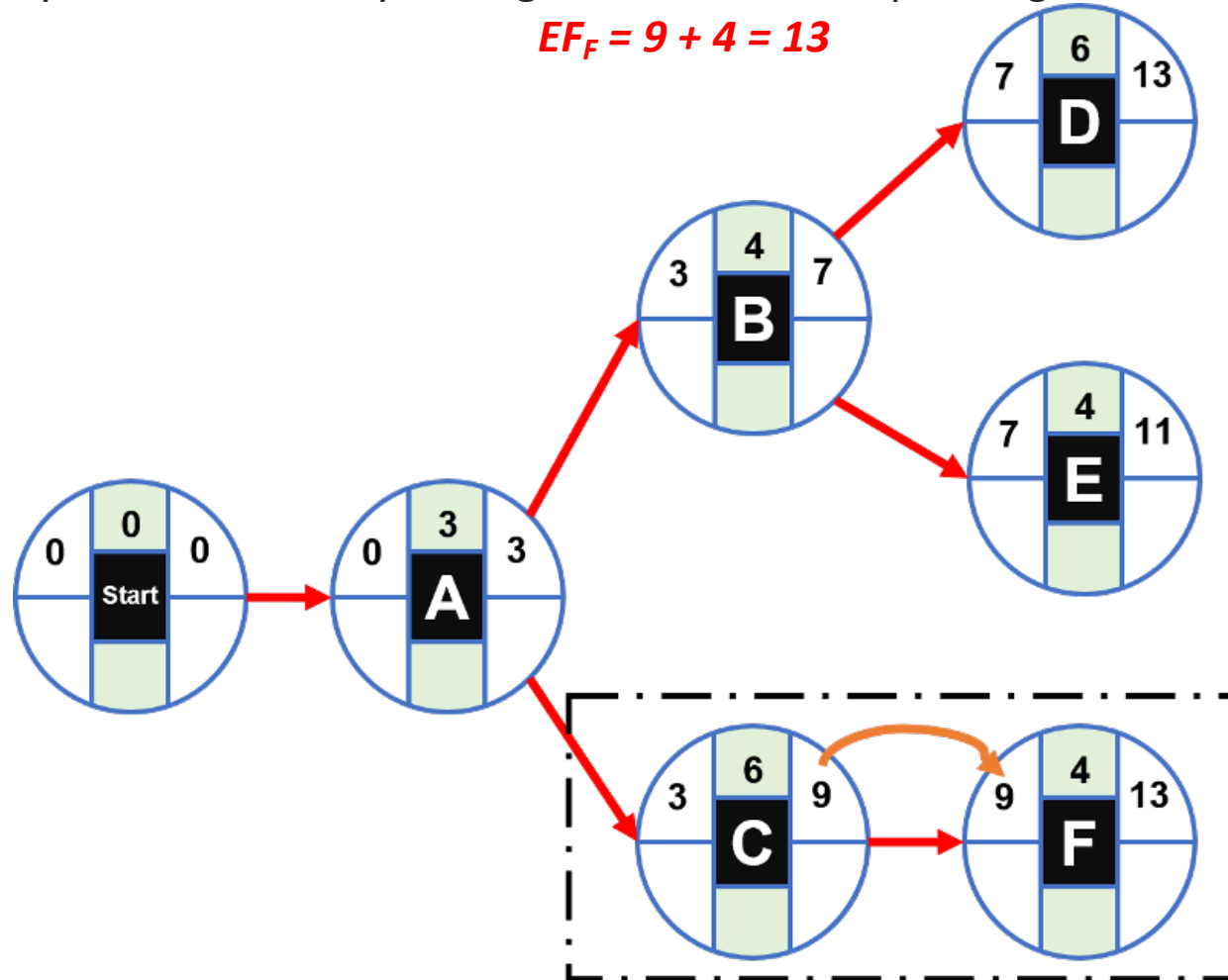
CPM Example –Solution (Forward Path)

Activity F:

It has as precedent only activity C; therefore its ES will be equal to the EF of activity C. The EF of activity F is calculated by adding its ES + the corresponding time:

$$EF_F = 9 + 4 = 13$$

Activity	Predecessor (s)	Time (Days)
A	–	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8



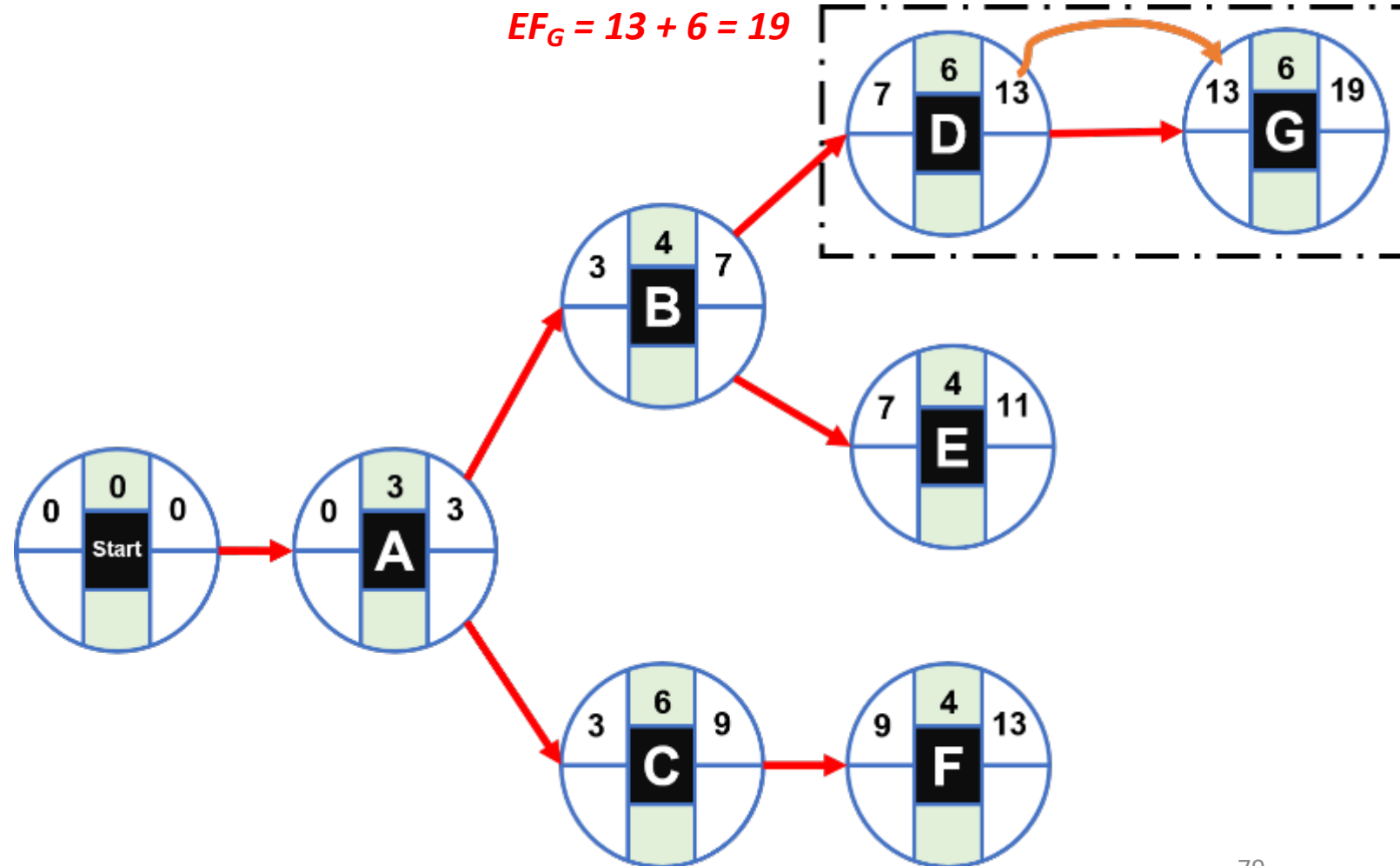
CPM Example –Solution (Forward Path)

Activity G:

It has as precedent only activity D; therefore its ES will be equal to the EF of activity D.

The EF of activity G is calculated by adding its ES + the corresponding time:

$$EF_G = 13 + 6 = 19$$

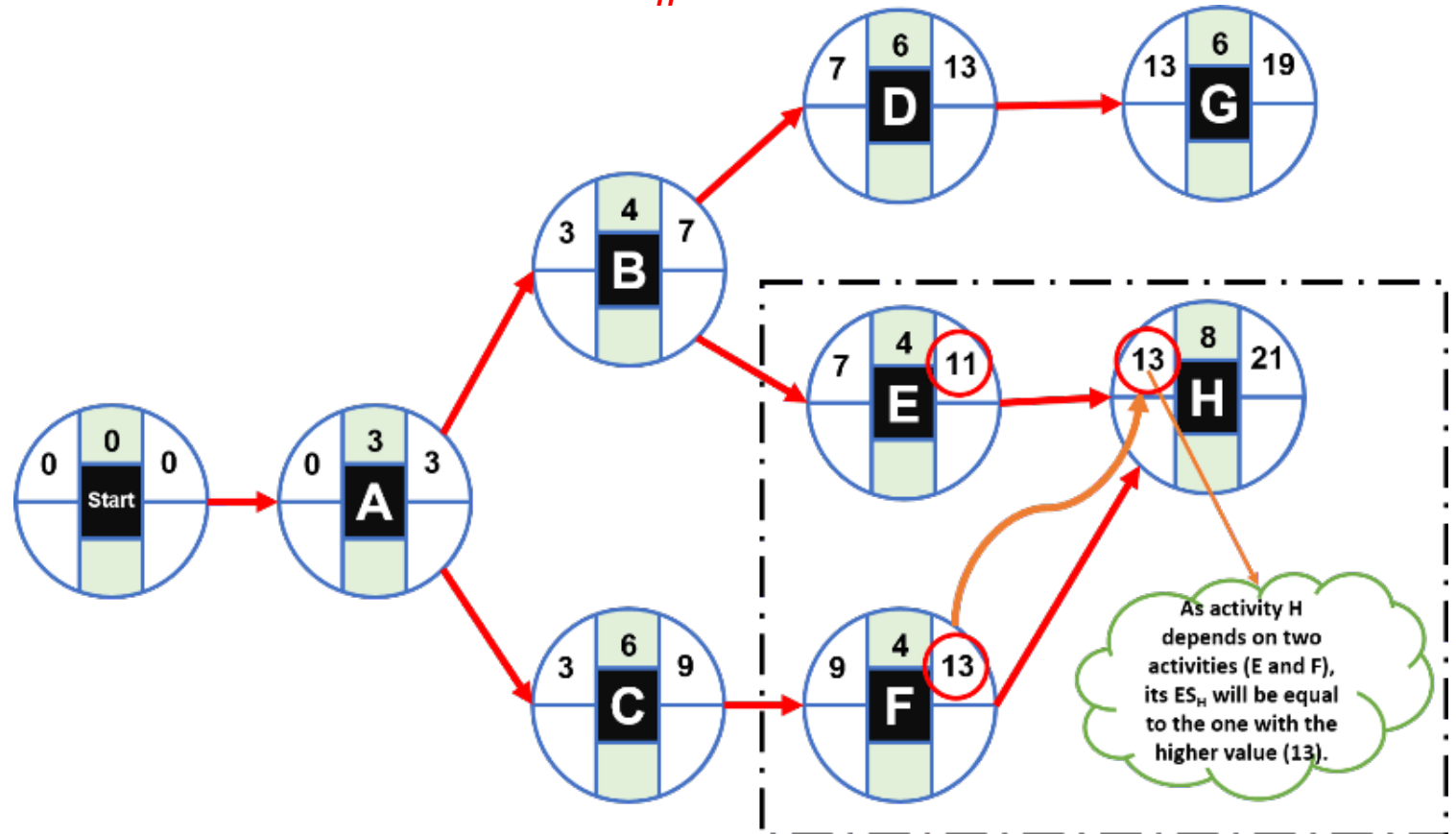


CPM Example –Solution (Forward Path)

Activity H:

This activity has two precedents: E and F; therefore its ES will be equal to the highest EF of both activities. In this case, activity F has the highest value with 13. The EF of activity H is calculated by adding its ES + the corresponding time:

$$EF_H = 13 + 8 = 21$$

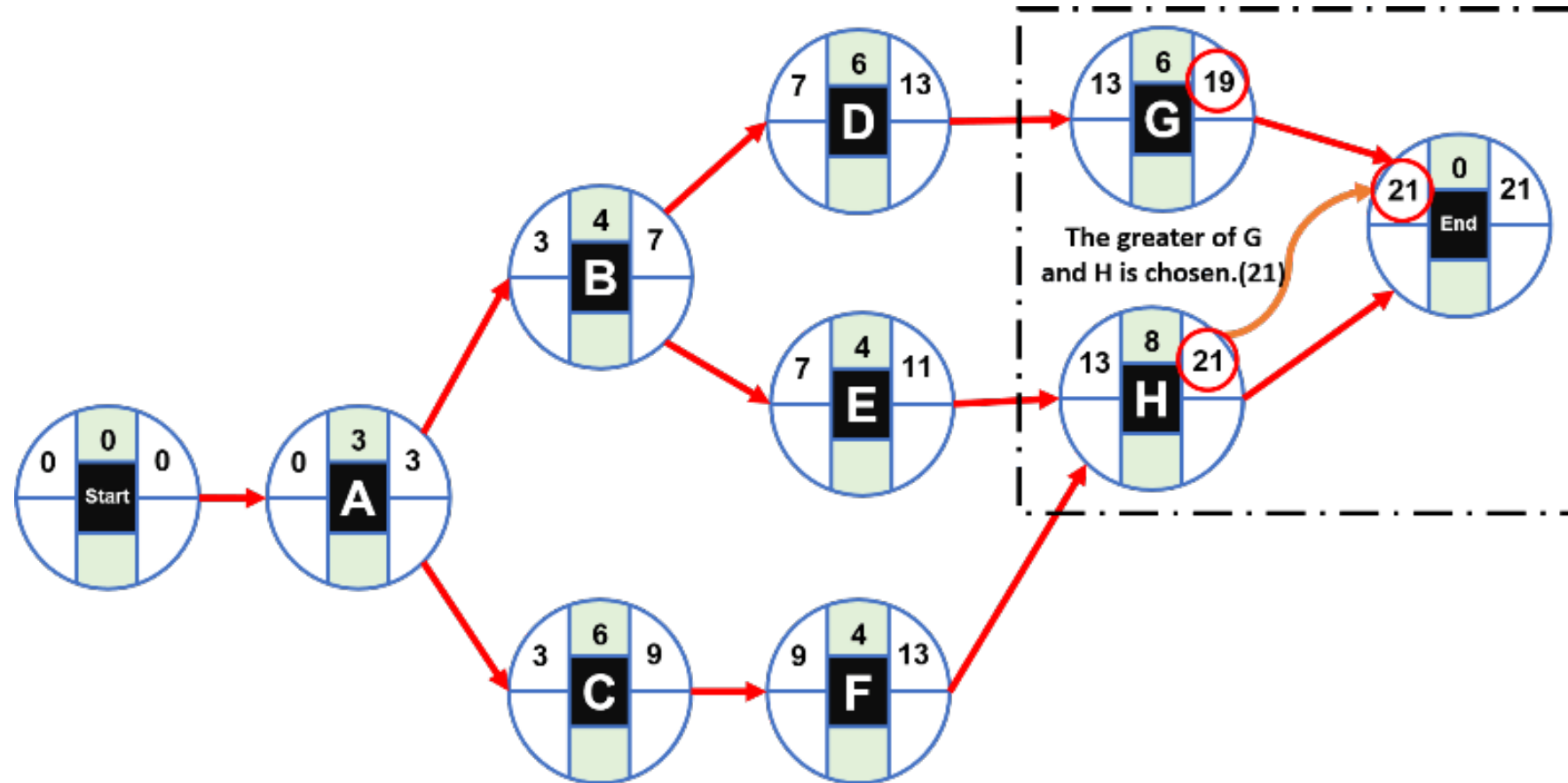


CPM Example –Solution (Forward Path)

End Node:

The end fictitious node is joined with the last activities G and H; and the highest value of the EF of both activities is placed as ES: 21. This value represents the total duration of the project. As this node has zero duration (because it is fictitious) its EF will be equal to $21 + 0 = 21$.

Activity	Predecessor (s)	Time (Days)
A	—	3
B	A	4
C	A	6
D	B	6
E	B	4
F	C	4
G	D	6
H	E, F	8



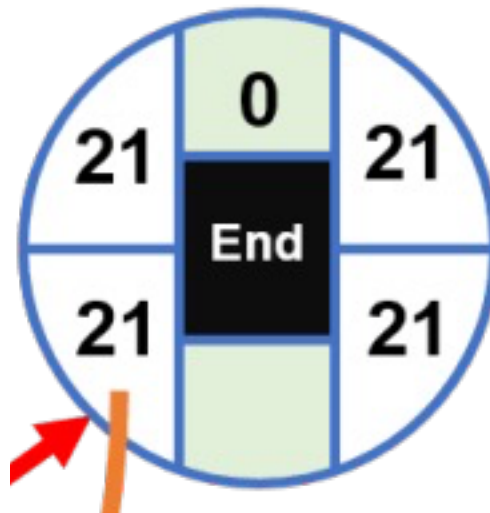
CPM Example –Solution (Backward Path)

To finalize the critical path calculation we will perform the backward traversal to calculate the LF and LS, starting from the final node; placing the values at the bottom of the node as follows:

End Node:

For the end node the LF value is equal to the project duration (21). The LS is calculated by subtracting the LF minus the duration (zero).

$$LS_{End} = 21 - 0 = 21$$

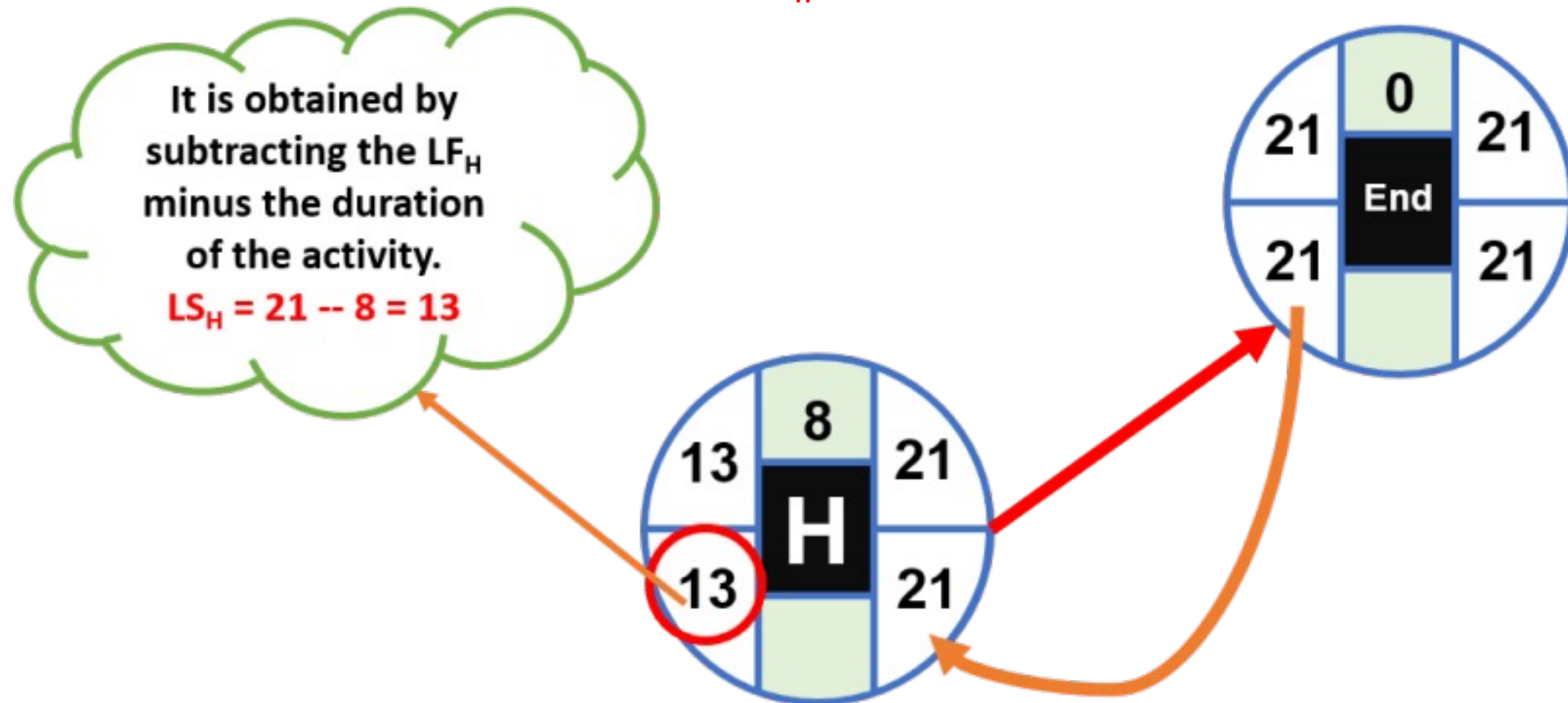


CPM Example –Solution (Backward Path)

Activity H:

Since the final node is the only successor to activity H, its LF will be equal to the LS of the final node (21). The LS of activity H is calculated by subtracting its LF minus its duration:

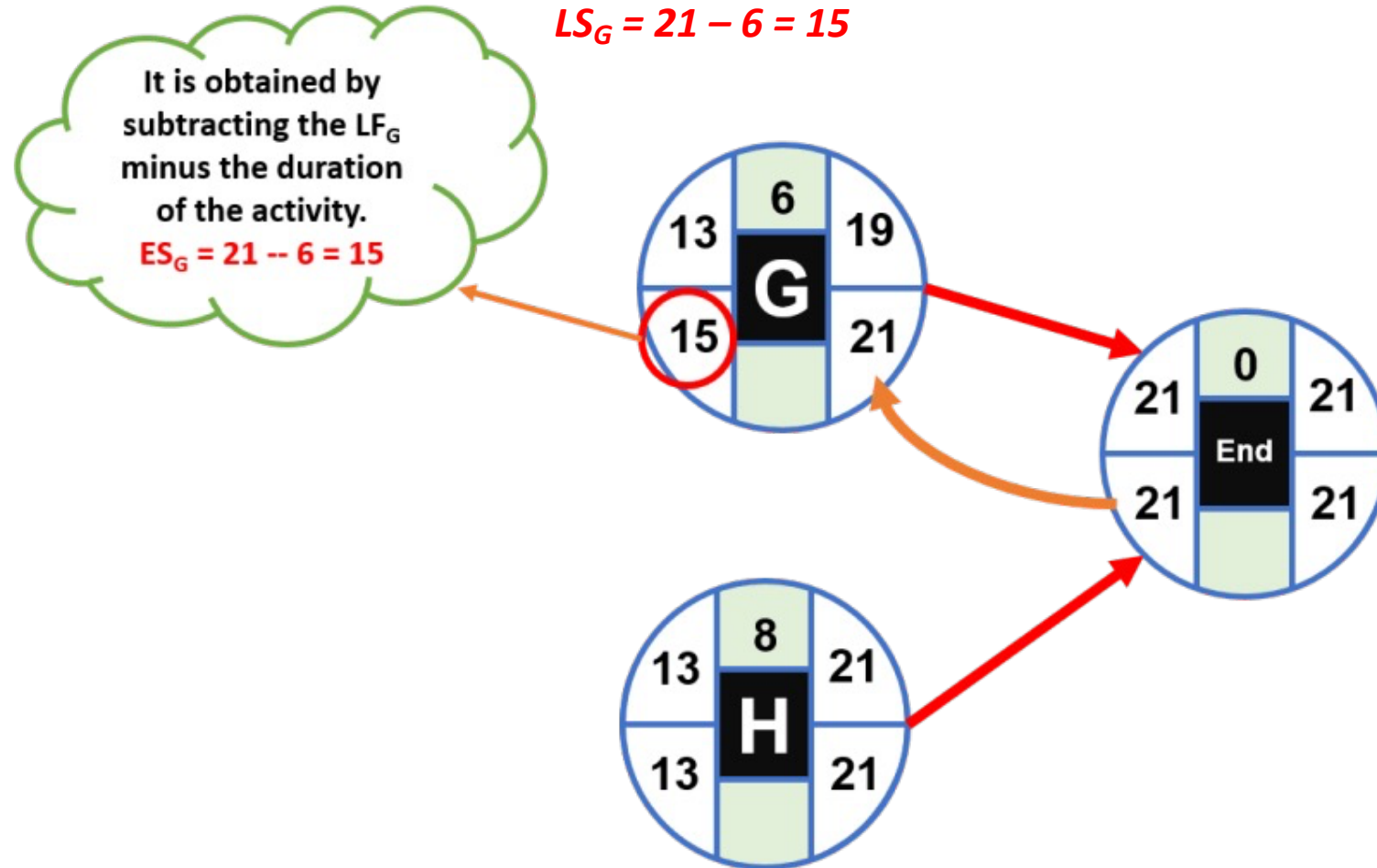
$$LS_H = 21 - 8 = 13$$



CPM Example –Solution (Backward Path)

Activity G:

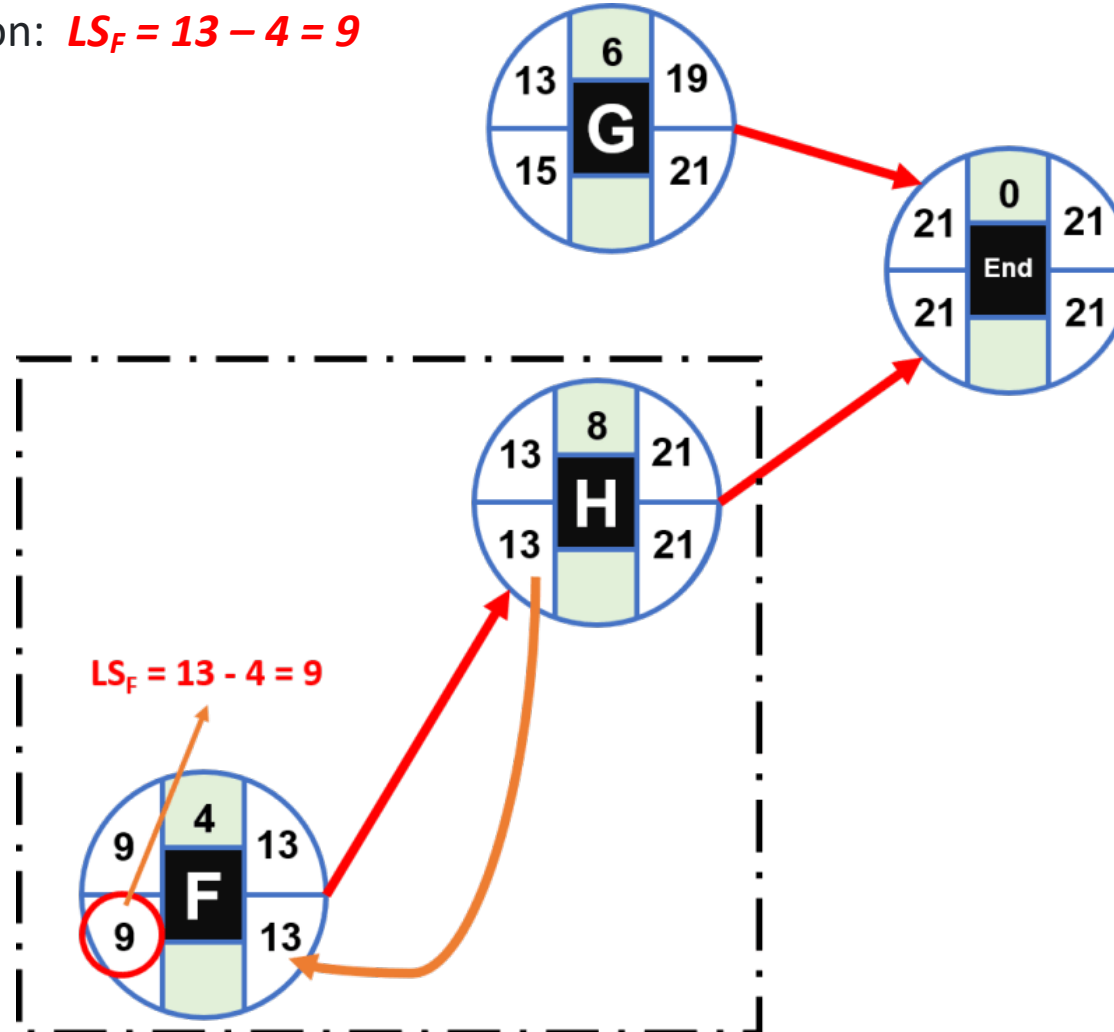
Since the end node is the only successor of activity G, its LF will be equal to the LS of the end node (21). The LS of activity G is calculated by subtracting its LF minus its duration:



CPM Example –Solution (Backward Path)

Activity F:

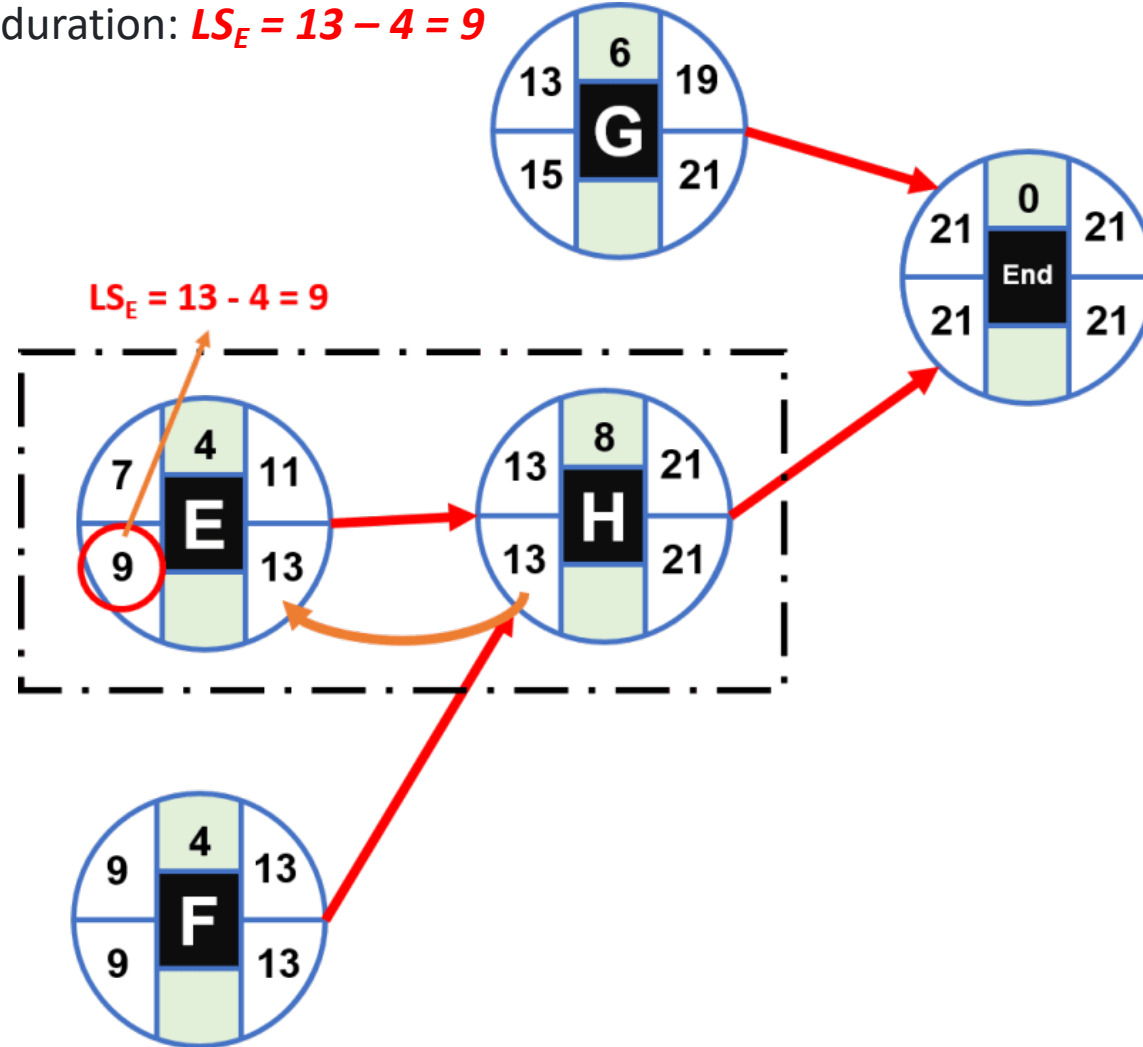
Since activity H is the only successor of activity F, its LF will be equal to the LS of activity H (13). The LS of activity F is calculated by subtracting its LF minus its duration: $LS_F = 13 - 4 = 9$



CPM Example –Solution (Backward Path)

Activity E:

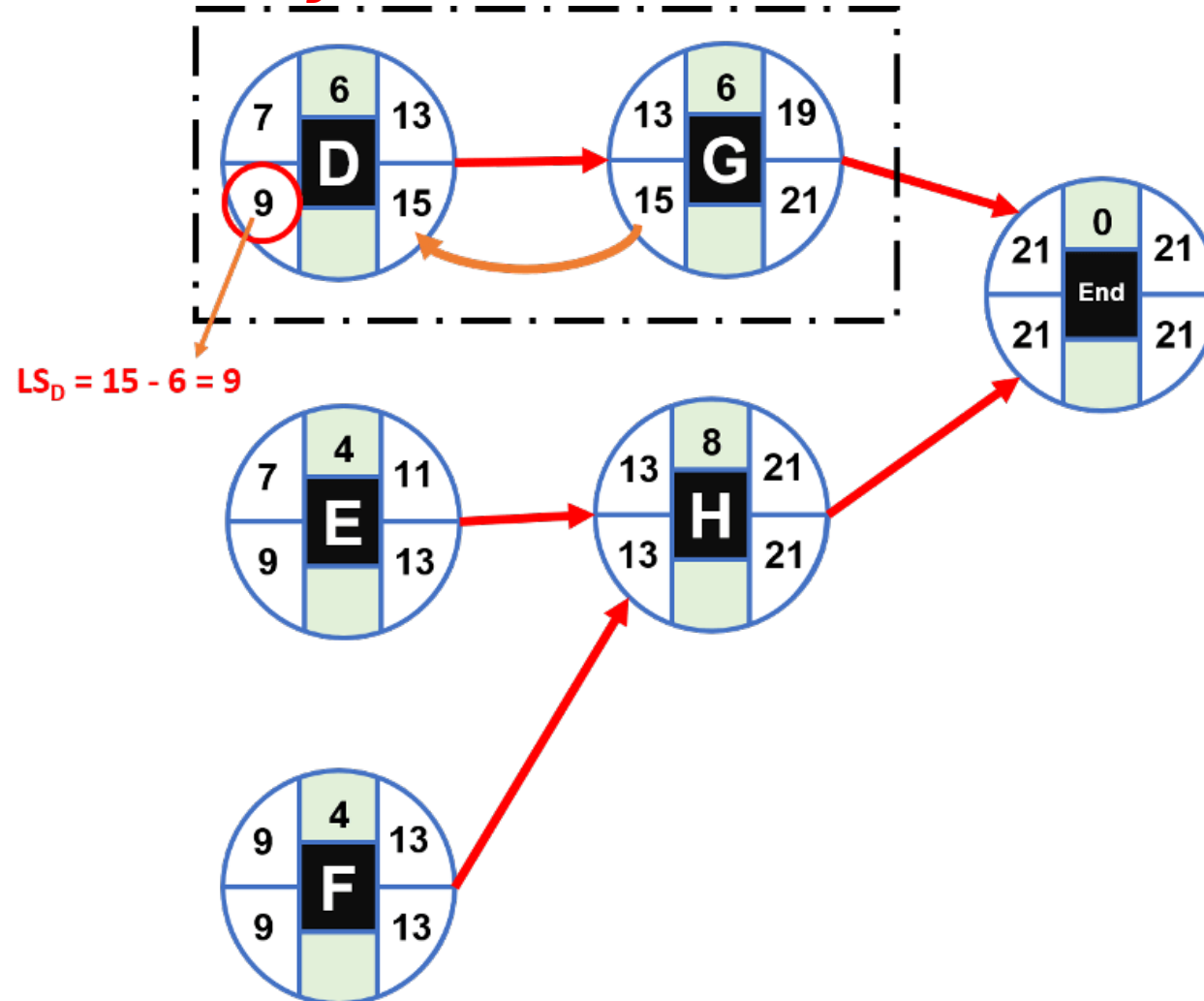
Since activity H is the only successor of activity E, its LF will be equal to the LS of activity H (13). The LS of activity E is calculated by subtracting its LF minus its duration: $LS_E = 13 - 4 = 9$



CPM Example –Solution (Backward Path)

Activity D:

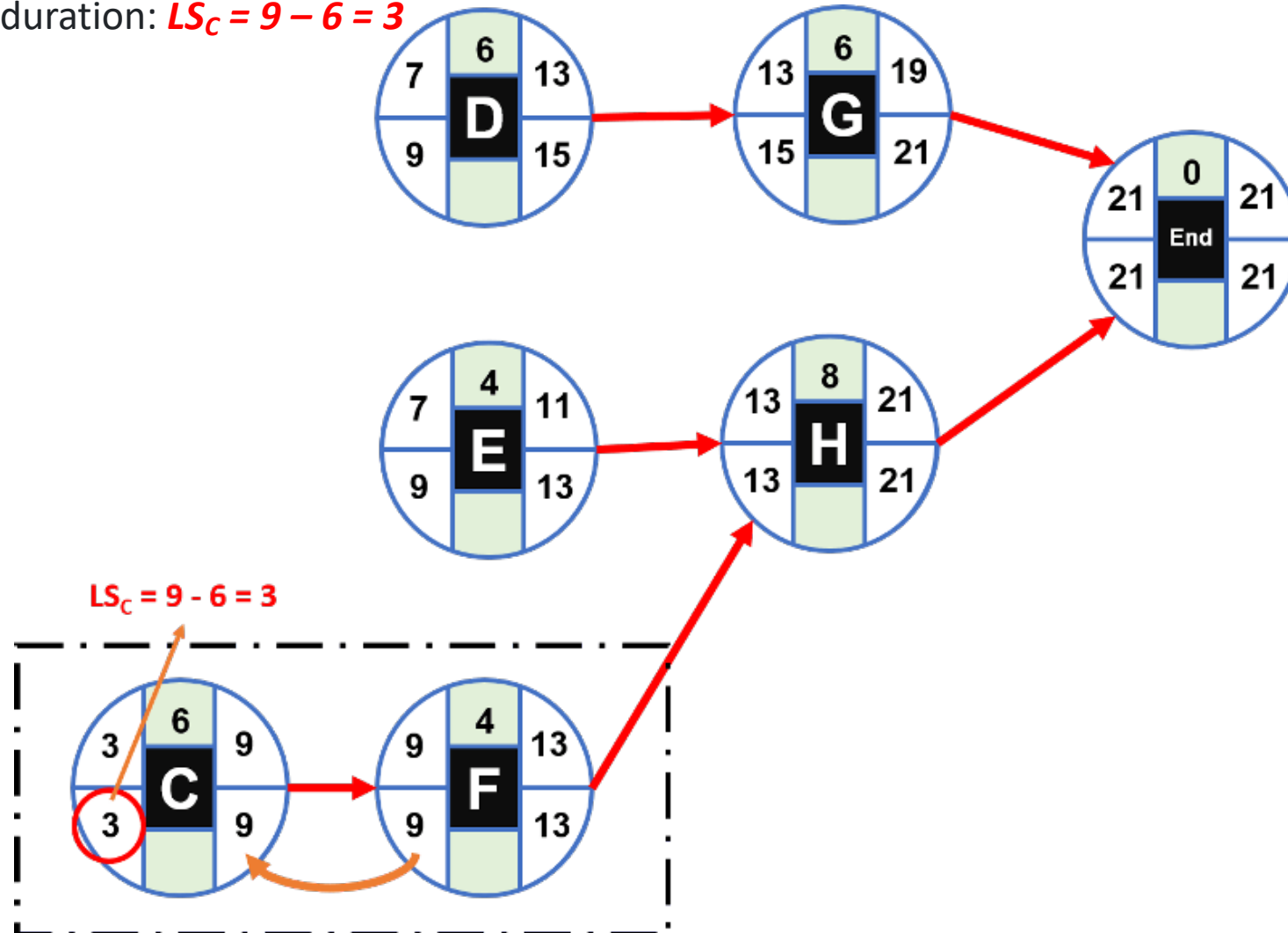
Since activity G is the only successor of activity D, its LF will be equal to the LS of activity G (15). The LS of activity D is calculated by subtracting its LF minus its duration: $LS_D = 15 - 6 = 9$



CPM Example –Solution (Backward Path)

Activity C:

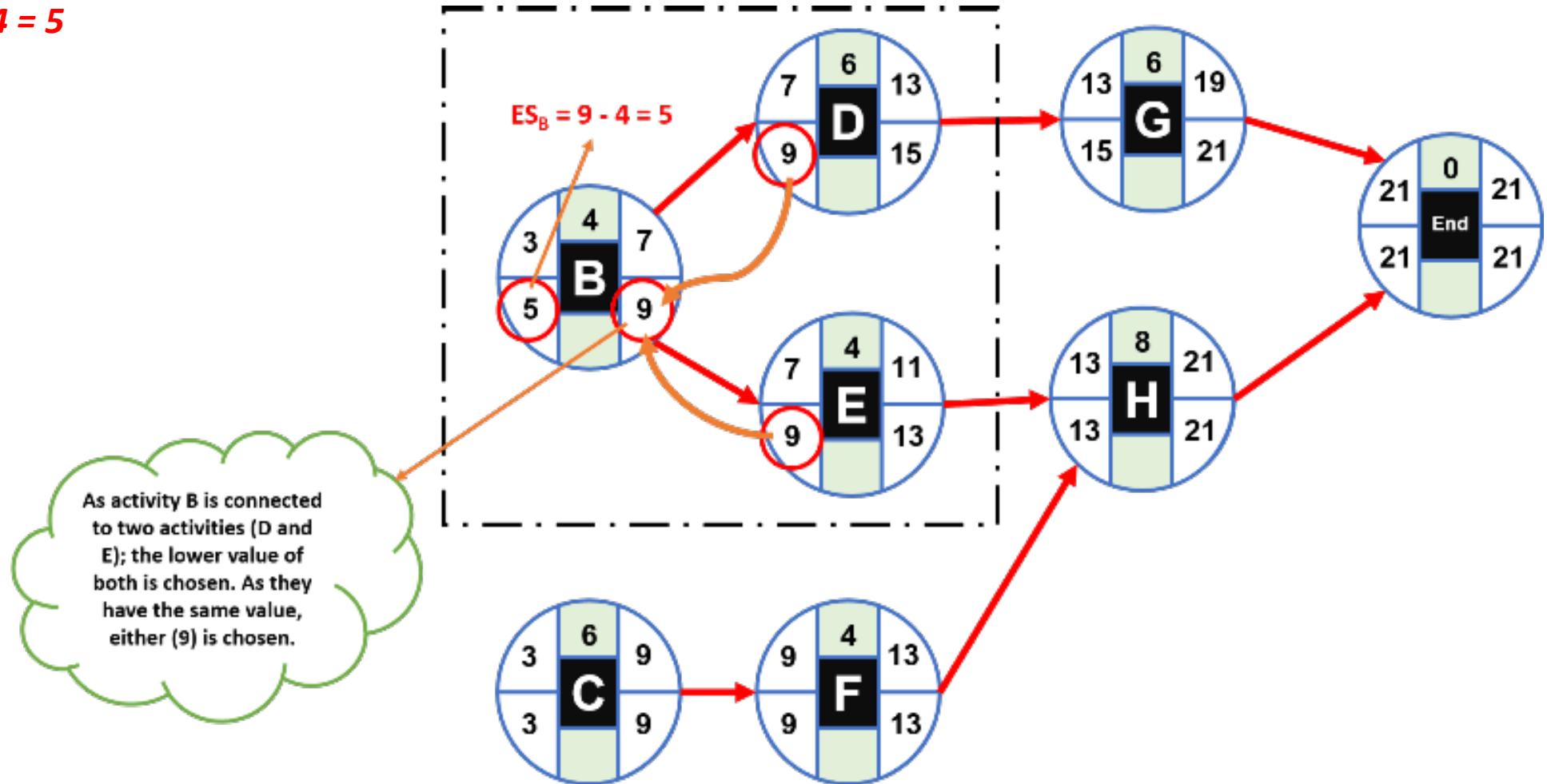
Since activity F is the only successor of activity C, its LF will be equal to the LS of activity F (9). The LS of activity C is calculated by subtracting its LF minus its duration: $LS_C = 9 - 6 = 3$



CPM Example –Solution (Backward Path)

Activity B:

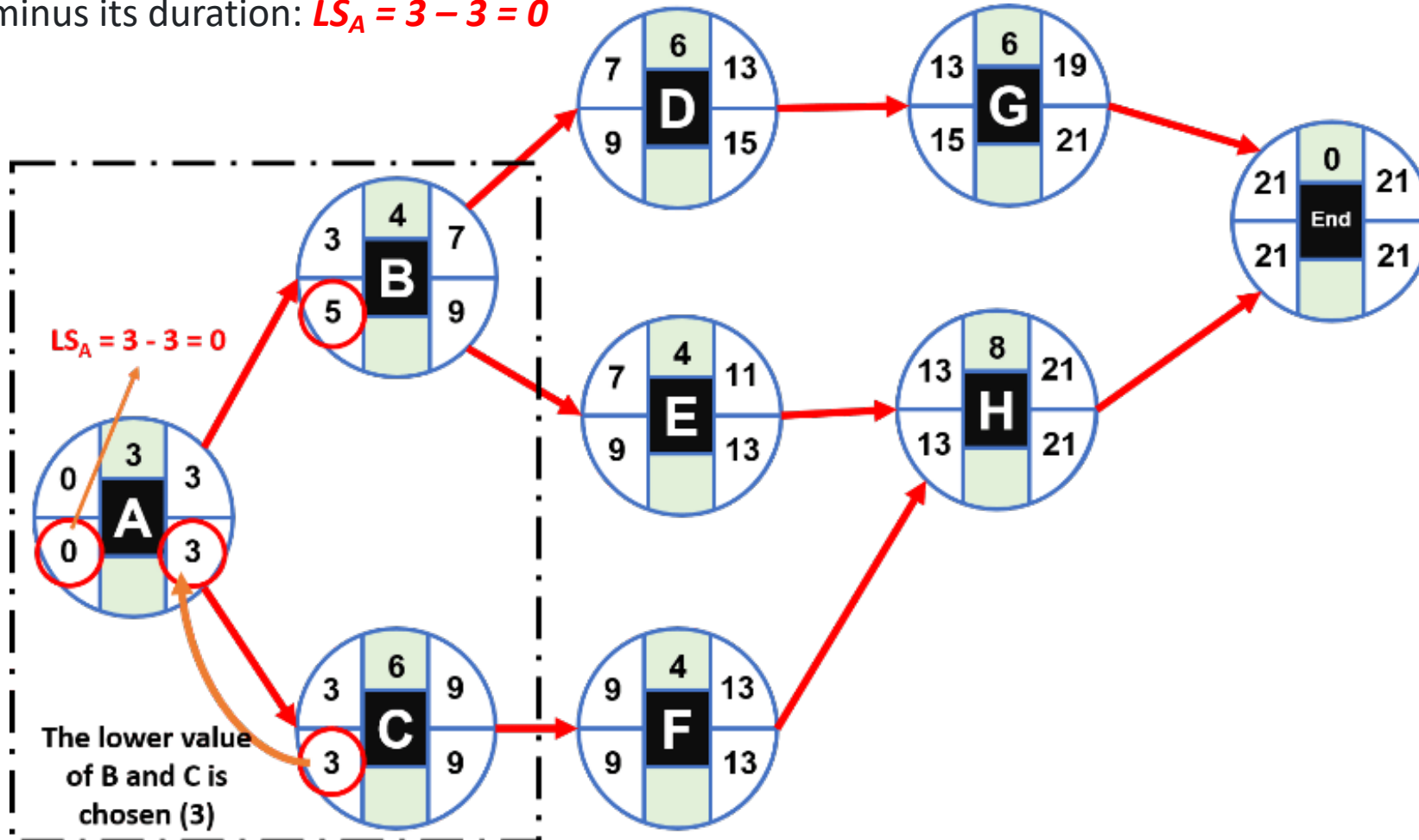
How activity B has as successors activities D and E, its LF will be equal to the smaller value of the LS of both. In this case, since both have a value of 9; that value will be the LF of activity B. The LS of activity B is calculated by subtracting its LF minus its duration: $LS_B = 9 - 4 = 5$



CPM Example –Solution (Backward Path)

Activity A:

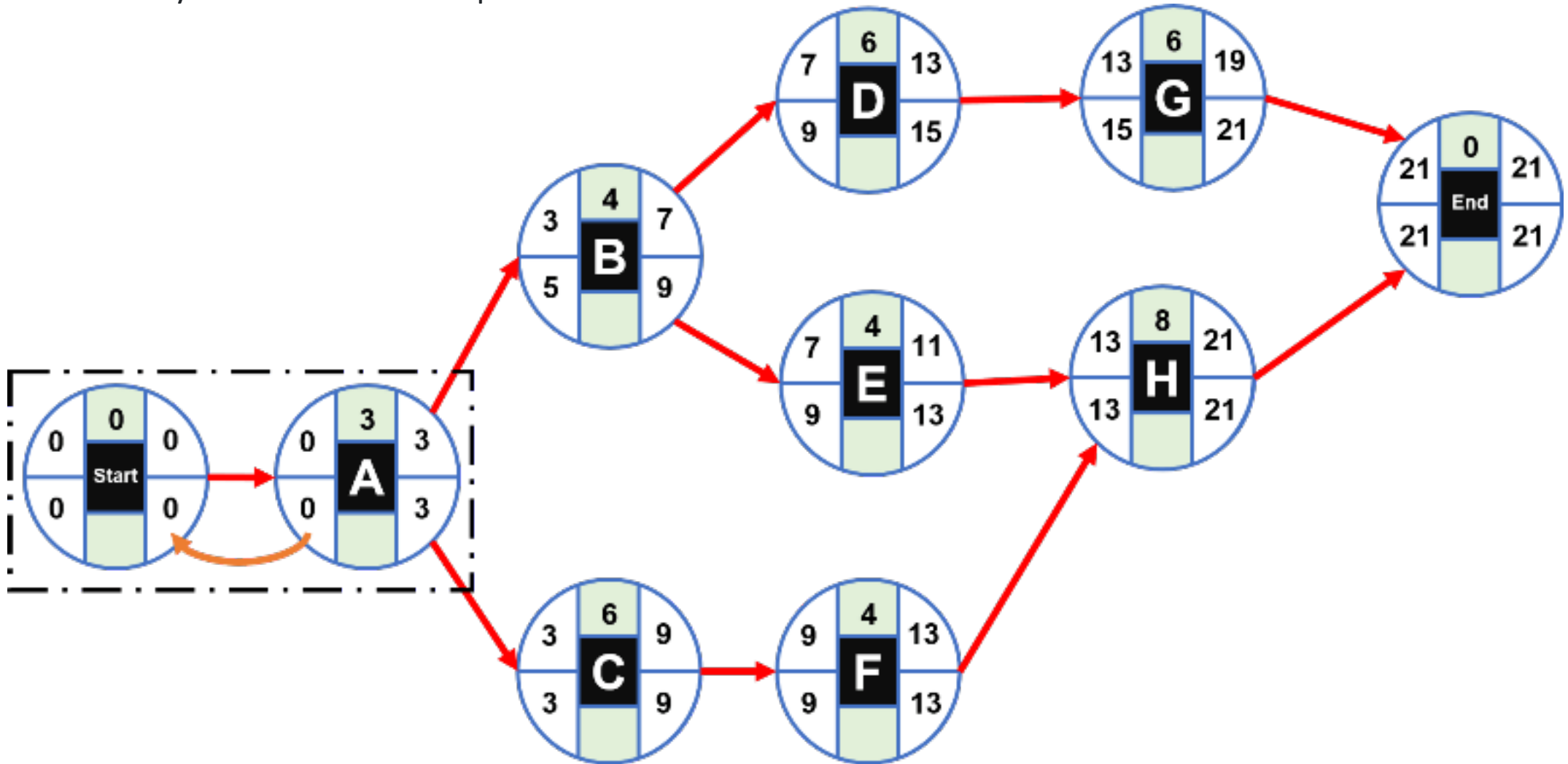
How activity A has as successors activities B and C, its LF will be equal to the smaller value of the LS of both. In this case, the lowest value is that of activity C (3); therefore, that value will be the LF of activity A. The LS of activity A is calculated by subtracting its LF minus its duration: $LS_A = 3 - 3 = 0$



CPM Example –Solution (Backward Path)

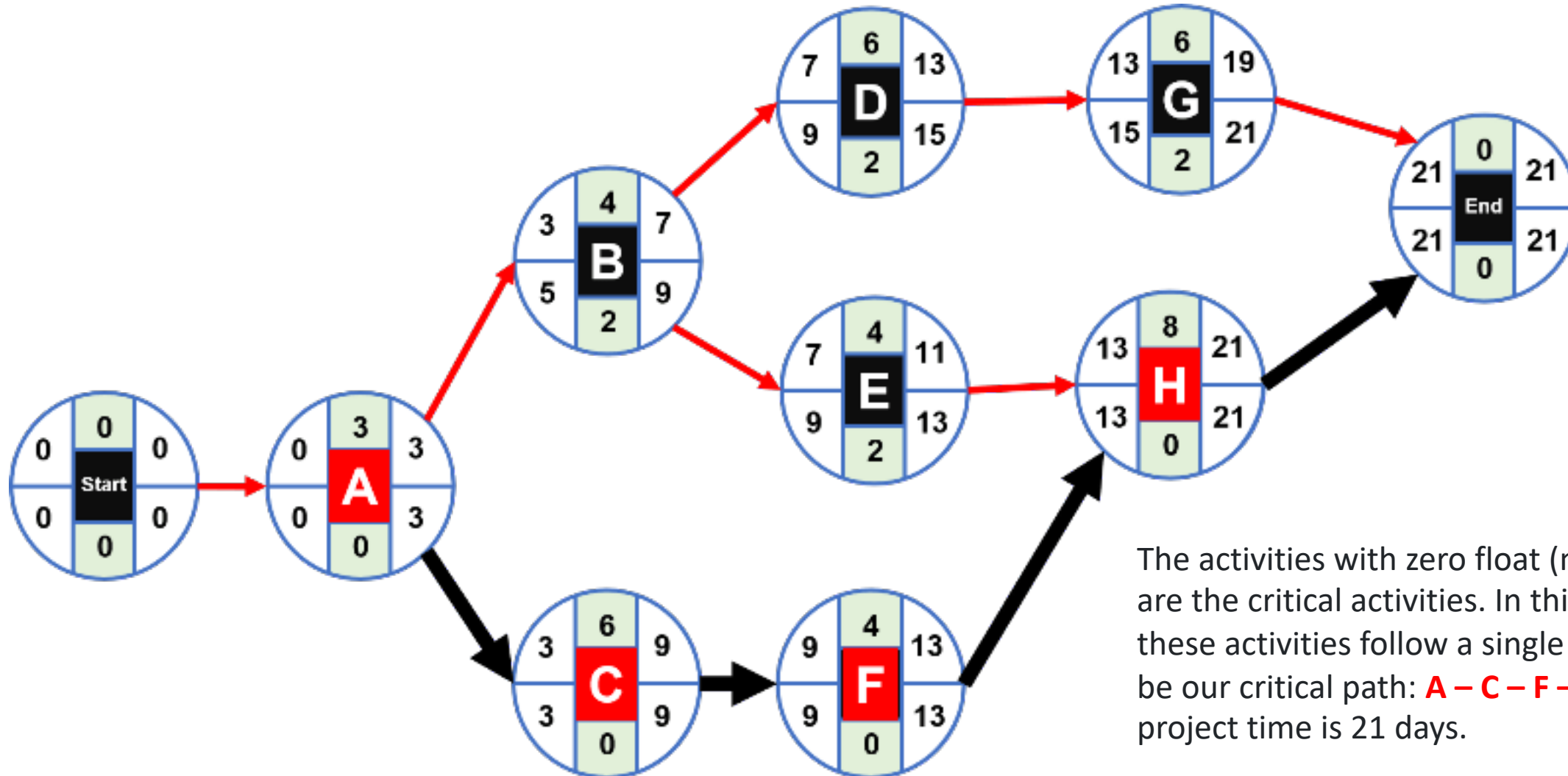
Initial Node:

Using the same analysis as above we complete the values of the initial node with zero.



CPM Example –Solution (Slack)

Finally, we calculate the slack for each node with the following formula: $Slack = LS - ES = LF - EF$



The activities with zero float (marked in red) are the critical activities. In this example, these activities follow a single path that will be our critical path: **A - C - F - H**. Total project time is 21 days.

Example 2

❖ Shirley Hopkins is developing a program in leadership training for middle-level managers. Shirley has listed a number of activities that must be completed before a training program of this nature can be conducted. The activities, immediate predecessors, and times appear in the accompanying table.

Example 2

Activity	Immediate Predecessor	Time (days)
A	—	2
B	—	5
C	—	1
D	B	10
E	A,D	3
F	C	6
G	E,F	8

Provide the following:

1. AON diagram for these precedencies
2. What is the critical path?
3. What is the total project completion date?
4. What is the slack time for each individual activity?

Example 2 - Solution

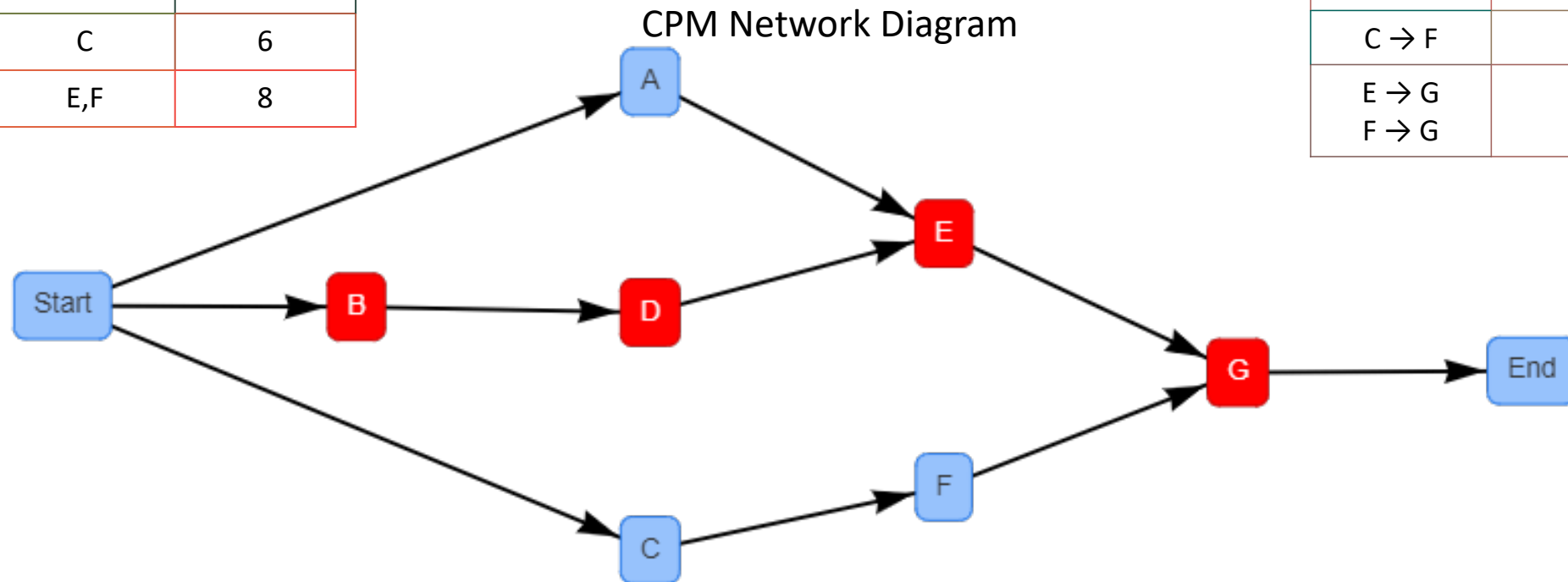
Each activity is broken down by its precedents and descendants:

Predecessor	Activity	Successor
Start → A	A	A → E
Start → B	B	B → D
Start → C	C	C → F
B → D	D	D → E
A → E D → E	E	E → G
C → F	F	F → G
E → G F → G	G	G → End

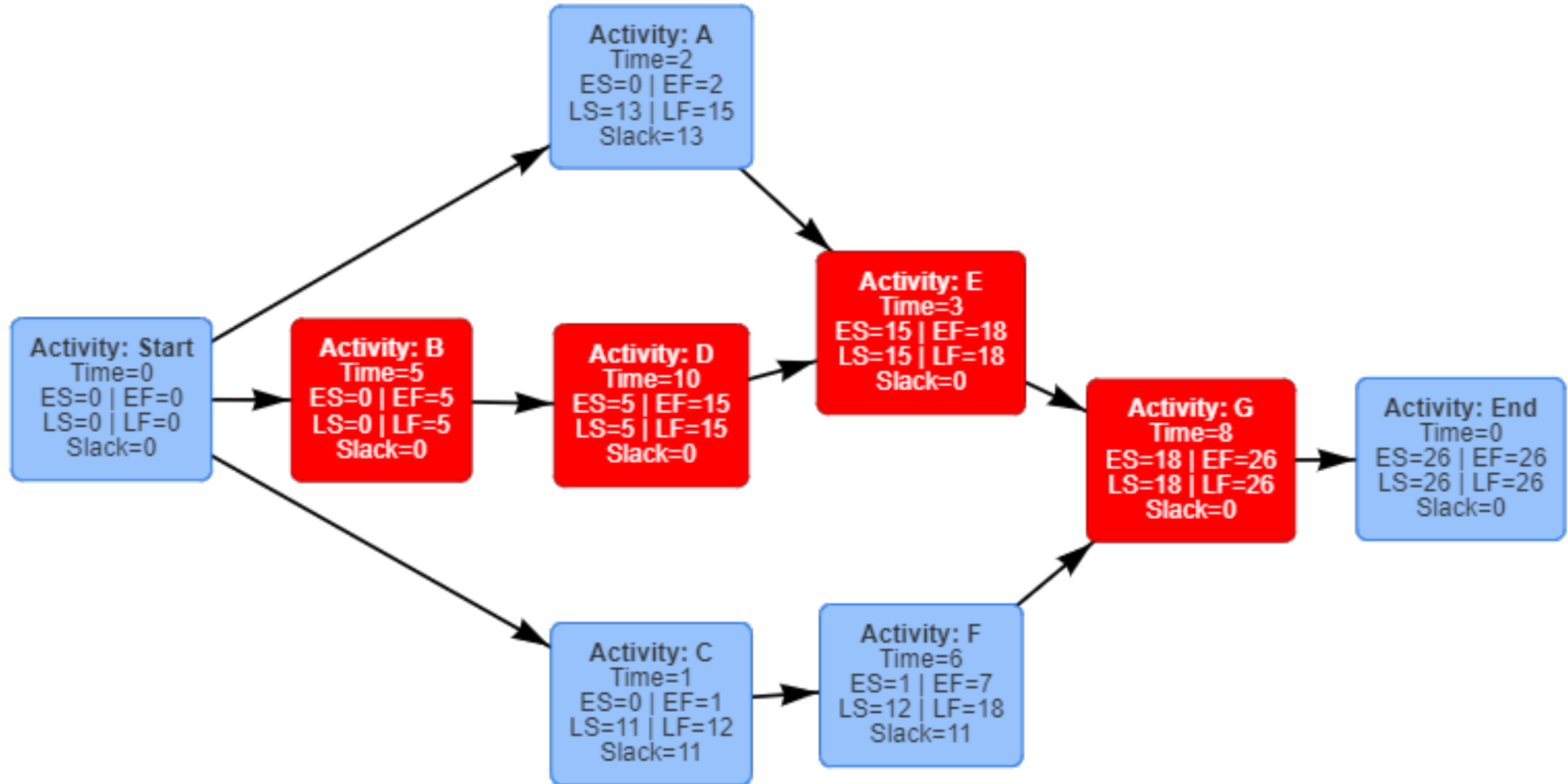
Example 2 - Solution

Activity	Immediate Predecessor	Time (days)
A	—	2
B	—	5
C	—	1
D	B	10
E	A,D	3
F	C	6
G	E,F	8

Predecessor	Activity	Successor
Start → A	A	A → E
Start → B	B	B → D
Start → C	C	C → F
B → D	D	D → E
A → E D → E	E	E → G
C → F	F	F → G
E → G F → G	G	G → End



Example 2 - Solution

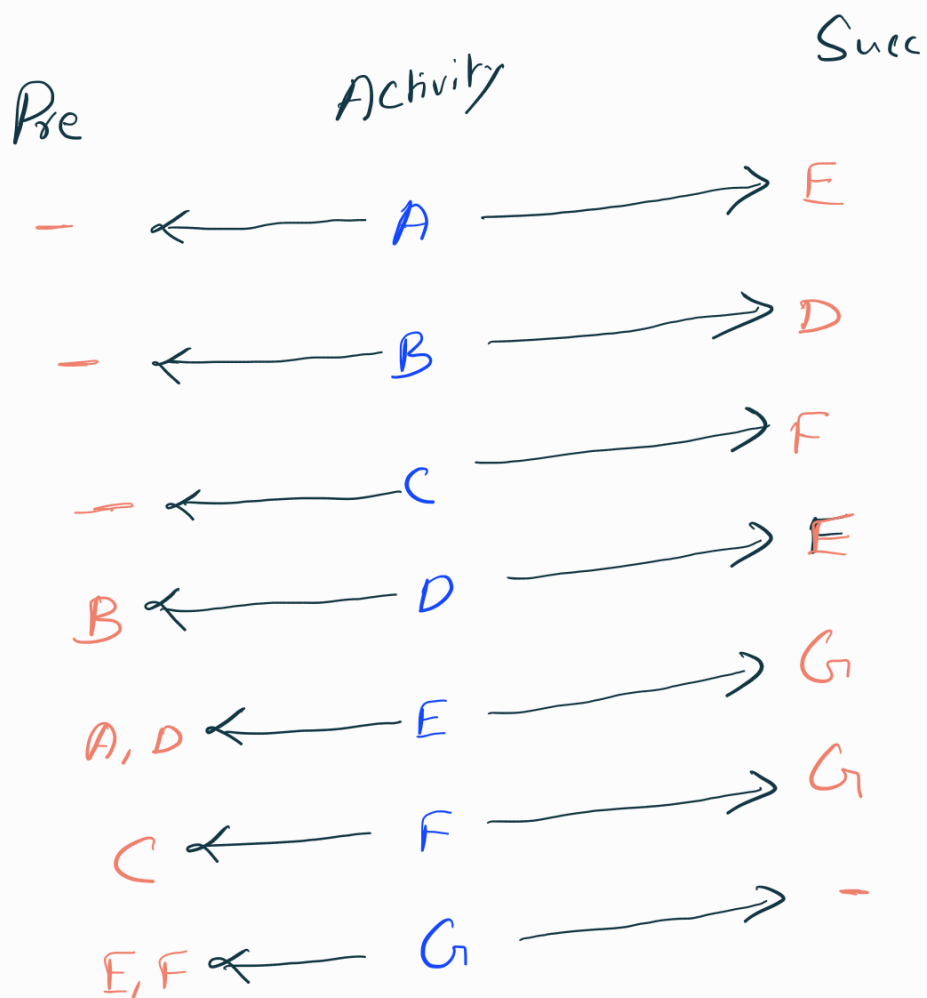
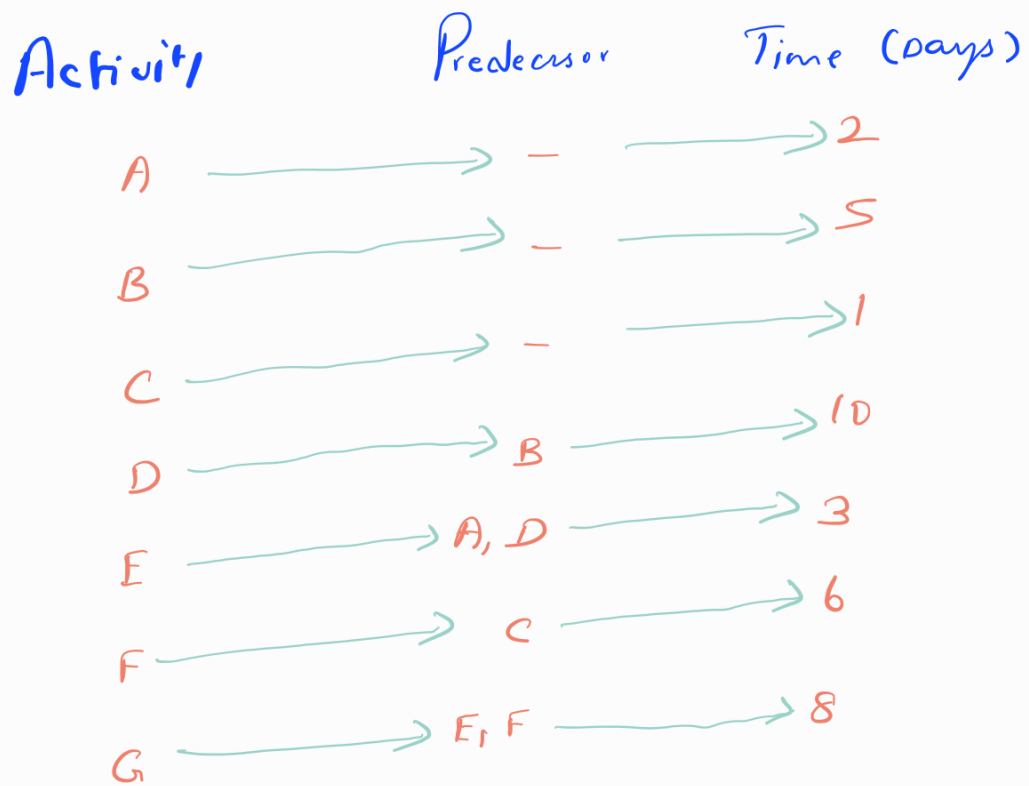


Example 2 - Solution

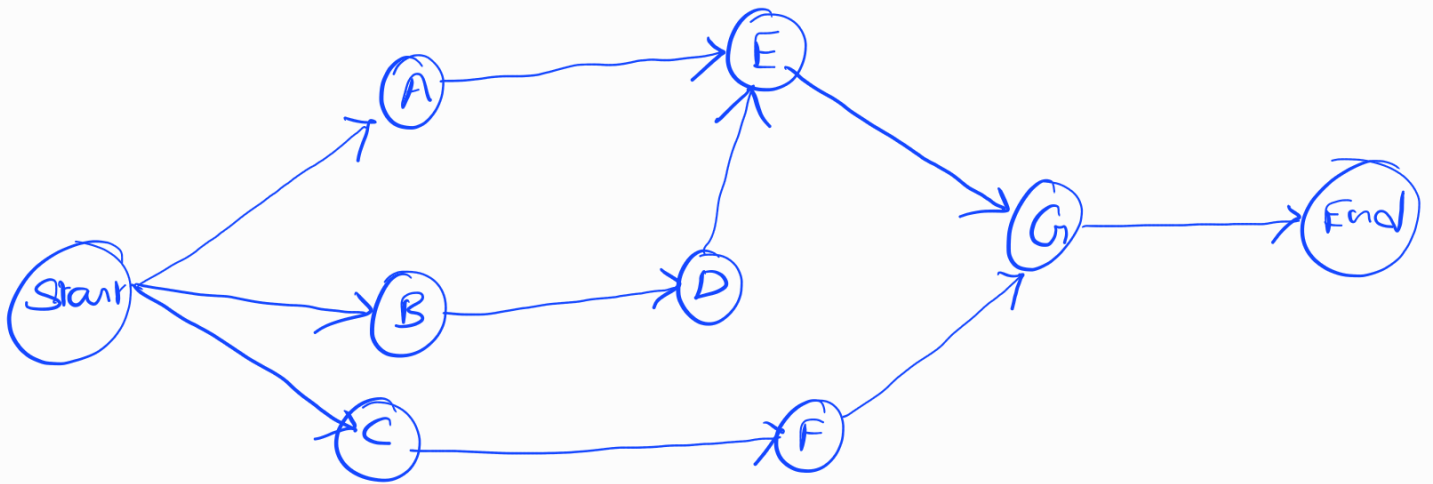
Activity	Time	Early Start (ES)	Early Finish (EF)	Late Start (LS)	Late Finish (LF)	Slack (S)
A	2	0	2	13	15	13
B	5	0	5	0	5	0
C	1	0	1	11	12	11
D	10	5	15	5	15	0
E	3	15	18	15	18	0
F	6	1	7	12	18	11
G	8	18	26	18	26	0

The critical path is: **B → D → E → G**

The total project time is: **26 hours**

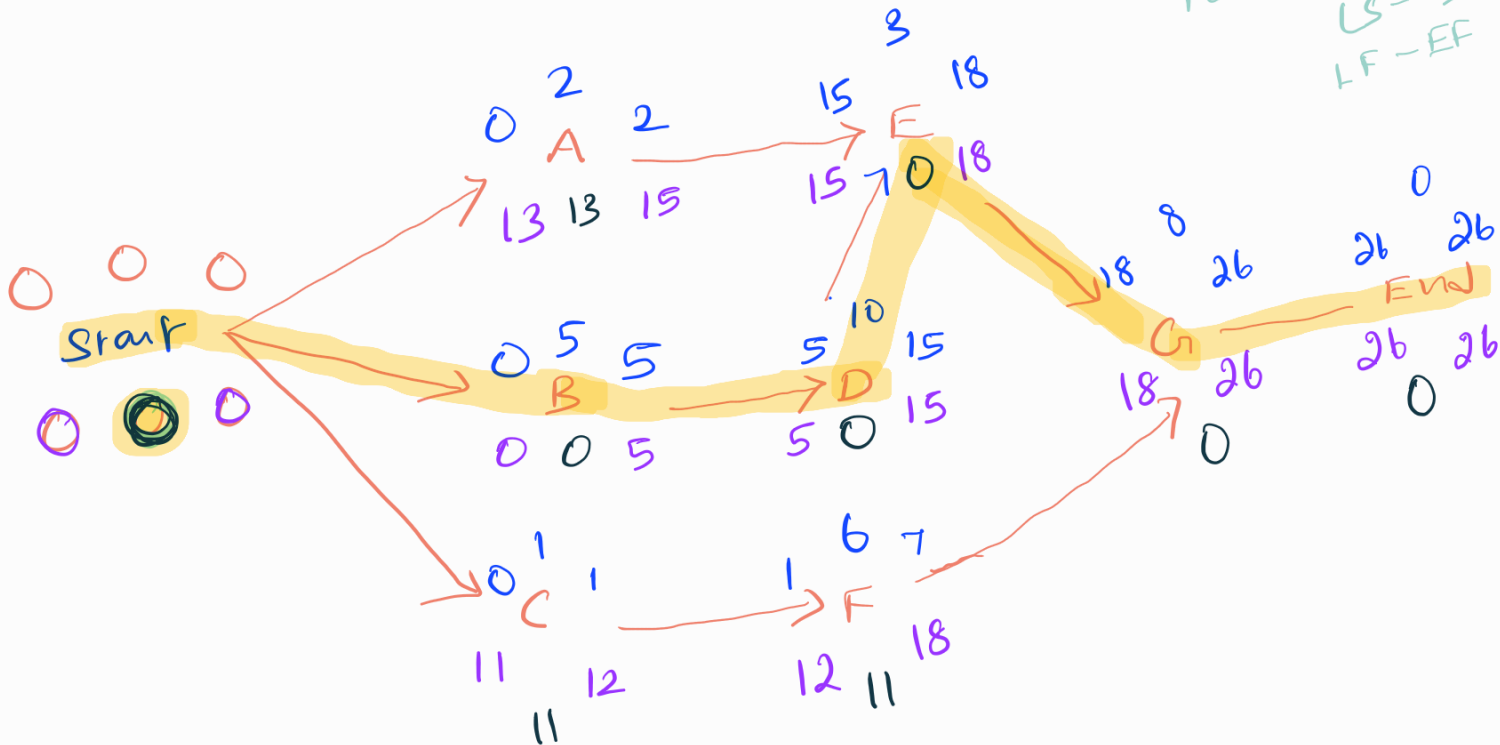


13
26
15



FWD $\rightarrow EF = ES + E$
 Backward $\rightarrow LS = LF - E$

Float =
 $LS - ES$
 $LF - EF$



Critical path $\rightarrow B \rightarrow D \rightarrow E \rightarrow G = 5 + 5 + 3 + 8 = 26$ Days

Unit 1 – Lecture 5

Importance of Float/Slack and Crashing of an Activity

Float Calculation and its Importance

➤ Float / Slack = **Amount of time** a task or activity **can be delayed without affecting the overall project timeline.**

➤ Types of Float

➤ Total Float (TF)

➤ Total Float = Late Start (LS) – Early Start (ES)

(or)

➤ Total Float = Late Finish (LF) – Early Finish (EF)

➤ Free Float (FF)

➤ Free Float = Early Start of Next Activity – Early Finish of Current Activity

➤ Independent Float

➤ Interfering Float

Importance of Float in PERT and CPM

1. Critical Path Identification
2. Resource Allocation
3. Risk Management
4. Schedule Flexibility
5. Project Optimization
6. Decision-Making

Example Float Calculation

Activity	Predecessor	Duration (Days)	Early Start (ES)	Early Finish (EF)	Late Start (LS)	Late Finish (LF)
A	-	3	0	3	0	3
B	A	4	3	7	3	7
C	B	2	7	9	7	9
D	A	5	3	8	4	9

- **Total Float for Activity D:**

- **Total Float = Late Start (LS)–Early Start (ES) = 4–3 = 1 day**

- or

- **Total Float = Late Finish (LF)–Early Finish (EF) = 9–8 = 1 day**

- **Free Float for Activity D:**

- **Free Float = Early Start of Next Activity–Early Finish of Current Activity=9–8=1**

Example Float Calculation

Activity	Predecessor	Duration (Days)	Early Start (ES)	Early Finish (EF)	Late Start (LS)	Late Finish (LF)
A	-	3	0	3	0	3
B	A	4	3	7	3	7
C	B	2	7	9	7	9
D	A	5	3	8	4	9

Activity	Predecessor	Duration (Days)	ES	EF	LS	LF	Total Float (TF)	Free Float (FF)
A	-	3	0	3	0	3	0	0
B	A	4	3	7	3	7	0	0
C	B	2	7	9	7	9	0	0
D	A	5	3	8	4	9	1	1

Cost reduction by Crashing of activity in PERT and CPM

- **Crashing is a technique** used in project management, particularly in PERT (Program Evaluation and Review Technique) and CPM (Critical Path Method), **to reduce the project duration by shortening the time required to complete critical activities.**
- This is achieved by **allocating additional resources** (e.g., labor, equipment, or funds) to these activities.
- However, **crashing comes at an additional cost**, and the goal is **to minimize the total project cost while achieving the desired reduction in project duration.**

Concepts in Crashing

1. Normal Time (NT)
2. Crash Time (CT)
3. Normal Cost (NC)
4. Crash Cost (CC)
5. Cost Slope

$$\text{Cost Slope} = \frac{\text{Crash Cost (CC)} - \text{Normal Cost (NC)}}{\text{Normal Time (NT)} - \text{Crash Time (CT)}}$$

Steps to Reduce Project Duration by Crashing

1. Identify the Critical Path
2. Calculate Cost Slope for Critical Activities
3. Crash the Least Expensive Critical Activities First
4. Recalculate the Critical Path
5. Repeat the Process

Example of Crashing

Activity	Predecessor	Normal Time (NT)	Crash Time (CT)	Normal Cost (NC)	Crash Cost (CC)	Cost Slope
A	-	3	2	\$100	\$150	\$50/day
B	A	4	3	\$200	\$300	\$100/day
C	B	2	1	\$50	\$100	\$50/day
D	A	5	4	\$300	\$400	\$100/day

- **Critical Path:** A → B → C (Total Duration = 9 days)
- **Total Normal Time** = 3 + 4 + 2 = 9 days
- **Total Normal Cost:** 100+200 + 50 = \$350

Example of Crashing

Activity	Predecessor	Normal Time (NT)	Crash Time (CT)	Normal Cost (NC)	Crash Cost (CC)	Cost Slope
A	-	3	2	\$100	\$150	\$50/day
B	A	4	3	\$200	\$300	\$100/day
C	B	2	1	\$50	\$100	\$50/day
D	A	5	4	\$300	\$400	\$100/day

Step 1: Crash Activity A (Cost Slope = \$50/day)

- Reduce A from **3 → 2 days**
- New total duration: **8 days** (New Critical Path: A → B → C (Total Duration = 8 days))
- Additional cost: **\$50**
- Total cost after crashing A: $(\$150 + \$200 + \$50) \rightarrow \mathbf{\$350 + \$50 = \$400}$

Example of Crashing

Activity	Predecessor	Normal Time (NT)	Crash Time (CT)	Normal Cost (NC)	Crash Cost (CC)	Cost Slope
A	-	3	2	\$100	\$150	\$50/day
B	A	4	3	\$200	\$300	\$100/day
C	B	2	1	\$50	\$100	\$50/day
D	A	5	4	\$300	\$400	\$100/day

Step 2: Crash Activity C (Cost Slope = \$50/day)

- Reduce C from **2 → 1 day**
- New total duration: **7 days**
- Additional cost: **\$50**
- Total cost after crashing C: **\$400 + \$50 = \$450**

Example of Crashing

Activity	Predecessor	Normal Time (NT)	Crash Time (CT)	Normal Cost (NC)	Crash Cost (CC)	Cost Slope
A	-	3	2	\$100	\$150	\$50/day
B	A	4	3	\$200	\$300	\$100/day
C	B	2	1	\$50	\$100	\$50/day
D	A	5	4	\$300	\$400	\$100/day

Step 3: Crash Activity B (Cost Slope = \$100/day)

- Reduce B from **4 → 3 days**
- New total duration: **6 days**
- Additional cost: **\$100**
- Total cost after crashing B: **\$450 + \$100 = \$550**

Example of Crashing

The final project duration is 6 days, and the total project cost after crashing is \$550.

Step	Activity Crashed	Duration Reduced	New Total Duration	Additional Cost	Total Cost
1	A	1 day	8 days	\$50	\$400
2	C	1 day	7 days	\$50	\$450
3	B	1 day	6 days	\$100	\$550

Importance of Crashing

1. Meeting Deadlines
2. Cost-Benefit Analysis
3. Resource Optimization
4. Risk Mitigation
5. Competitive Advantage

Limitations of Crashing

1. Increased Costs
2. Diminishing Returns
3. Resource Constraints
4. Quality Risks

Example: Free Float and Total Float Calculation

Problem Statement

An IT project has the following tasks, durations, and dependencies:

Task	Predecessors	Duration (Days)
A	-	3
B	A	5
C	A	4
D	B, C	2

1. Draw the project network diagram.
2. Calculate the **Early Start (ES)**, **Early Finish (EF)**, **Late Start (LS)**, and **Late Finish (LF)** for each task.
3. Calculate the **Total Float** and **Free Float** for each task.

Solution

Step 1: Draw the Project Network Diagram

The network diagram is as follows:

A → B → D

A → C → D

Step 2: Calculate ES, EF, LS, and LF

Forward Pass (Calculate ES and EF):

- Start with **Task A**:
 - ES = 0
 - EF = ES + Duration = 0 + 3 = 3
- **Task B**:
 - ES = EF of A = 3
 - EF = ES + Duration = 3 + 5 = 8
- **Task C**:
 - ES = EF of A = 3
 - EF = ES + Duration = 3 + 4 = 7
- **Task D**:
 - ES = Max(EF of B, EF of C) = Max(8, 7) = 8
 - EF = ES + Duration = 8 + 2 = 10

Backward Pass (Calculate LS and LF):

- Start with **Task D**:
 - LF = EF of D = 10
 - LS = LF - Duration = 10 - 2 = 8

- **Task B:**
 - $LF = LS \text{ of D} = 8$
 - $LS = LF - \text{Duration} = 8 - 5 = 3$
- **Task C:**
 - $LF = LS \text{ of D} = 8$
 - $LS = LF - \text{Duration} = 8 - 4 = 4$
- **Task A:**
 - $LF = \text{Min}(LS \text{ of B}, LS \text{ of C}) = \text{Min}(3, 4) = 3$
 - $LS = LF - \text{Duration} = 3 - 3 = 0$

Step 3: Calculate Total Float and Free Float

Total Float:

Total Float = $LS - ES$ or $LF - EF$

- Task A:
Total Float = $0 - 0 = 0$ or $3 - 3 = 0$
- Task B:
Total Float = $3 - 3 = 0$ or $8 - 8 = 0$
- Task C:
Total Float = $4 - 3 = 1$ or $8 - 7 = 1$
- Task D:
Total Float = $8 - 8 = 0$ or $10 - 10 = 0$

Free Float:

Free Float = $ES \text{ of Successor} - EF \text{ of Current Task}$

- Task A:
Free Float = $\text{Min}(ES \text{ of B}, ES \text{ of C}) - EF \text{ of A} = 3 - 3 = 0$
- Task B:
Free Float = $ES \text{ of D} - EF \text{ of B} = 8 - 8 = 0$
- Task C:
Free Float = $ES \text{ of D} - EF \text{ of C} = 8 - 7 = 1$
- Task D:
Free Float = 0 (No successor)

Summary of Results

Task	ES	EF	LS	LF	Total Float	Free Float
A	0	3	0	3	0	0
B	3	8	3	8	0	0
C	3	7	4	8	1	1
D	8	10	8	10	0	0

Assumption:

- **Total Float** is the flexibility of a task without delaying the project.
- **Free Float** is the flexibility of a task without delaying the start of its successor tasks.
- Tasks on the critical path ($A \rightarrow B \rightarrow D$) have zero total float and zero free float.
- Task C has a total float of 1 day and a free float of 1 day, meaning it can be delayed by 1 day without affecting the project or its successor.

Example 2:

Project Overview:

Imagine a project with the following four activities and their respective time estimates. We are given the **early start (ES)**, **early finish (EF)**, **late start (LS)**, and **late finish (LF)** for each activity.

Activity	Predecessor	Duration (Days)	ES	EF	LS	LF
A	-	3	0	3	0	3
B	A	4	3	7	3	7
C	B	2	7	9	7	9
D	A	5	3	8	4	9

Solution:

1. Total Float (TF):

- **Formula for Total Float (TF):**

$$TF = LS - ES \text{ or } TF = LF - EF$$

- **Interpretation:**

Total Float is the amount of time an activity can be delayed without delaying the project's overall completion date.

Let's calculate the **Total Float (TF)** for each activity:

- **Activity A:**

$$TFA = LSA - ESA = 0 - 0 = 0 \text{ or } TFA = LFA - EFA = 3 - 3 = 0$$

Total Float for A = 0 days (critical path)

- **Activity B:**

$$TFB = LSB - ESB = 3 - 3 = 0 \text{ or } TFB = LFB - EFB = 7 - 7 = 0$$

Total Float for B = 0 days (critical path)

- **Activity C:**

$$TFC = LSC - ESC = 7 - 7 = 0 \text{ or } TFC = LFC - EFC = 9 - 9 = 0$$

Total Float for C = 0 days (critical path)

- **Activity D:**

$$TFD = LSD - ESD = 4 - 3 = 1 \text{ or } TFD = LFD - EFD = 9 - 8 = 1$$

Total Float for D = 1 day (non-critical path)

2. Free Float (FF):

- **Formula for Free Float (FF):**

$$FF = ES_{\text{next}} - EF$$

- **Interpretation:**

Free Float is the amount of time an activity can be delayed without delaying the **earliest start** of its **successor activities**.

Let's calculate the **Free Float (FF)** for each activity:

- **Activity A:**

- Successors: **B** and **D**.
- **Earliest Start for Successors (B and D) = 3** (both start at Day 3).

$$FF_A = \min(ES_B, ESD) - EFA = \min(3, 3) - 3 = 3 - 3 = 0$$

- **Free Float for A = 0 days.**

- **Activity B:**

- Successor: **C**.
- **Earliest Start for Successor (C) = 7**.

$$FF_B = ESC - EFB = 7 - 7 = 0$$

- **Free Float for B = 0 days.**

- **Activity C:**

- Successor: **None**.
- Since C has no successor, the Free Float is the difference between its Late Finish (LF) and Early Finish (EF).

$$FF_C = LFC - EFC = 9 - 9 = 0$$

- **Free Float for C = 0 days.**

- **Activity D:**

- Successor: **None**.
- Since D has no successor, the Free Float is the difference between its Late Finish (LF) and Early Finish (EF).

$$FF_D = LFD - EFD = 9 - 8 = 1$$

- **Free Float for D = 1 day.**

Summary of Calculations:

Activity	Total Float (TF)	Free Float (FF)
A	0 days	0 days
B	0 days	0 days
C	0 days	0 days
D	1 day	1 day